

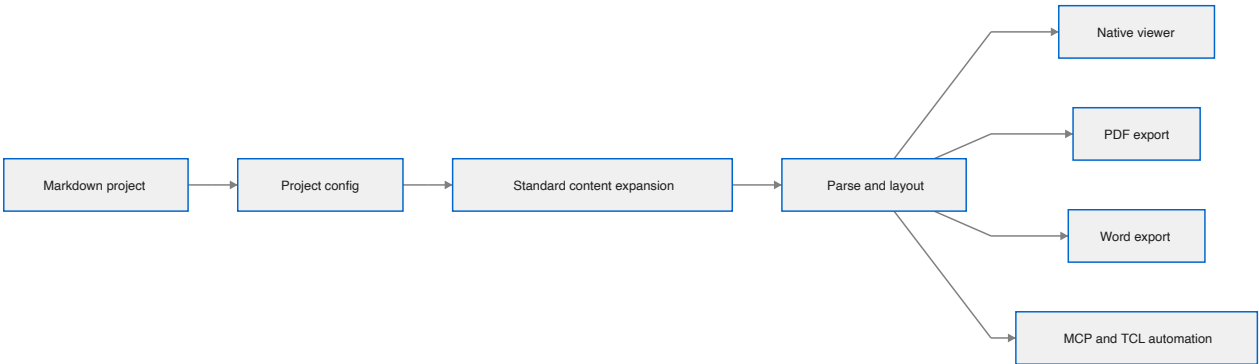
DocCrate

DocCrate is a native macOS application for database-backed and local Markdown projects. It uses Cocoa for the desktop interface, supports light and dark appearance, and works offline for normal document work.

DocCrate can browse, edit, validate, and export documentation projects. It supports CommonMark Markdown, syntax-highlighted code, native Mermaid diagrams, DocCrate `blockgraph` diagrams, custom vector shapes, reusable project variables, standard-content references, TCL automation, and MCP tooling for agents.

Asset Record

Field	Value
Asset	SecurityOperations
Asset area ID	18590
Asset area name	doccrate
Asset identity key	urn:asset:2249:securityoperations:18590:doccrate
Description	documentation utility



Documentation

Document	What it covers
Install on macOS	Install the notarized app, document library, signed connection package, Keychain credential, CLI, and MCP server.
User Guide	Current end-user guide: projects, authoring, diagrams, editing, exports, standard content, and automation.
Build This App	Source-repository and development-build notes.
Keyboard Shortcuts	Menu commands, keyboard shortcuts, mouse actions, and editor-mode controls.

Features	Rendered Markdown examples and feature tour.
Implementation	Current code architecture, layout pipeline, diagram engine, and export pipeline.
Architecture	Lower-level module notes and design constraints.
Licences and Permitted Use	Open-source origin and the restriction against redistribution or republication of this LexisNexis-branded fork.
Asset Metadata	SecurityOperations asset registration and repository metadata.
Block Views	DocCrate-only blockgraph cards, arrows, notes, and labels.
Custom Mermaid Shapes	Project-specific .shape icons and shape syntax.
Configuration	Runtime viewer themes, syntax highlighting, shapes, and project config files.
TCL Scripting	Automation scripts for project setup, validation, exports, and standard content.

Reference

Reference	What it covers
macOS Command Line	Installed CLI location, shared storage, validation, export, SQLite library, and Cocoa automation.
Connection String Protection	Password-free connection packages and one-time PAQ provisioning into the macOS Keychain.
MCP Server	Agent tools and resources exposed by doccrate-mcp.
AppleScript and Apple Events	Native macOS automation of the GUI, library, editors, export, and referrals.
Agent Discovery	MCP discovery resources and release-folder agent-card files.
Agent Referrals	App-generated clipboard prompts for handing brochure, saved DocCrate, pagination, and standard-content jobs to agents.
Project Config	.doccrate/project-config.md, variables, substitutions, and export banners/headers/footers.
Standard Content Library	Managed reusable text database, search, editing, tokens, and expansion.
Database-Backed DocCrates	Saved DocCrate roots, database documents, database shapes, recents, and sync design.
Database-Primary Brochure Storage	Design for database-authoritative brochures, managed local checkouts, local import/export, and agent workflows.
Menu And Editing Usability	Proposed menu restructuring, source-mode consolidation, contextual right-click labels, and governed library editing.
Spelling on macOS	Regional brochure profiles, native underlines, governed corrections, and project terminology.
Theming	Viewer theme JSON, color keys, and theme switching.

At A Glance

- **Native Markdown viewer:** headings, emphasis, code, blockquotes, lists, tables, images, links, horizontal rules, toolbar tables, and page breaks.
- **Project workflow:** File menu commands for new projects, open project, recent project, close project, exports, and TCL scripts.
- **Project config:** `.doccrate/project-config.md` variables such as `{{product}}`, `{{product_name}}`, `{{MONTHYEAR}}`, and export banner/header/footer settings.
- **Standard content:** searchable approved text database with editable records, `{{standard:n}}` tokens, validation, and recursive substitution.
- **Diagrams:** Mermaid, `blockgraph`, manual layout comments, custom shapes, and graphical editors for supported diagram families.
- **Exports:** PDF, rendered Word, editable Word, SVG, current-page export, and whole-document export. PDF output is compressed and protected from modification in conforming readers by default.
- **Agent-friendly:** stable diagram IDs, headless snapshots, lint/inspect, MCP tools, TCL scripting, and agent discovery resources.

Quick Start

Install the application, separate library, and connection package with [Install on macOS](#). Then browse a local root:

```
/Applications/DocCrate.app/Contents/Helpers/doc-crate ~/Documents/my-docs
```

Create a new project:

```
doc-crate --new-root ~/Documents/my-docs --name "My Docs"
```

Export a whole project:

```
doc-crate --export-pdf ~/Documents/my-docs ~/Documents/my-docs.pdf
doc-crate --export-pdf-layout ~/Documents/my-docs ~/Documents/my-docs-layout.png
doc-crate --export-editable-docx ~/Documents/my-docs ~/Documents/my-docs.docx
```

Omit the output path only when you want DocCrate to place the generated file in the generated cache folder rather than beside the source docs.

Agents should start with [Agent Discovery](#) and the [MCP Server](#).

Permitted Use

DocCrate originated as MIT-licensed open-source software. This branded fork is for the exclusive commercial use of LexisNexis and is not for public redistribution or republication. Third-party components retain their own licence terms. Read [Licences and Permitted Use](#) before copying or distributing any application, source, documentation, or branded asset.

Third-Party Notices

[Third-Party Licences and Acknowledgements](#) is the final page in this documentation set. It contains the product acknowledgement, complete locked macOS Rust dependency inventory, bundled Lato font notice, prominent Selkie and docx-rs copyrights, and links to the applicable licence texts.

Agent MCP Test Page

This page is a compact document for testing the DocCrate MCP workflow. It gives agents a known target for diagram discovery, snapshot rendering, PDF export, and manual viewer checks without touching product documentation.

Feature Sweep

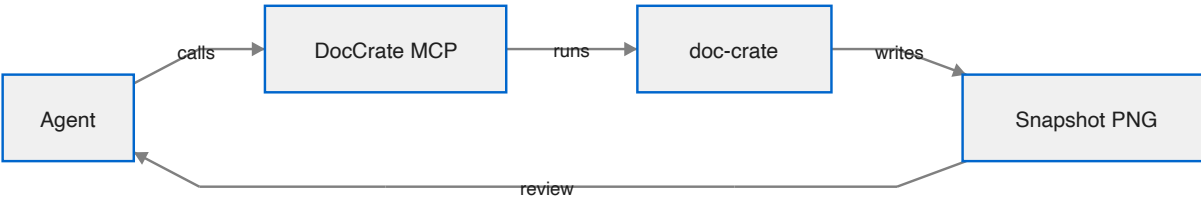
The document intentionally includes:

- A local link back to [Home](#)
- An external link to [Rust](#)
- Inline styles with **bold**, *italic*, ***bold italic***, and `code`
- A table with alignment and short wrapped content
- A highlighted Rust code block

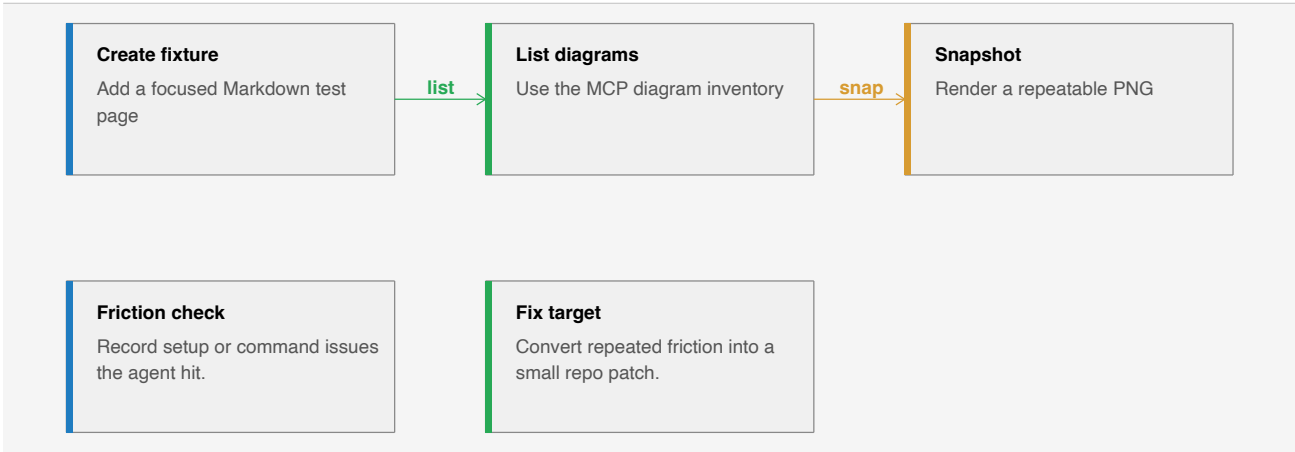
Surface	Expected check	Notes
MCP tools	Initialize, list tools, render snapshot	Paths should work with cwd
Diagram IDs	Stable explicit IDs	Prefer IDs over line numbers
Export	PDF command accepts this file	Output should stay outside docs

```
fn render_target(name: &str) -> String {
    format!("doccrate::{name}")
}
```

Mermaid Probe



Blockgraph Probe



Closing Check

If this page lists two diagrams and produces a PNG snapshot, the basic MCP agent loop is healthy.

Architecture

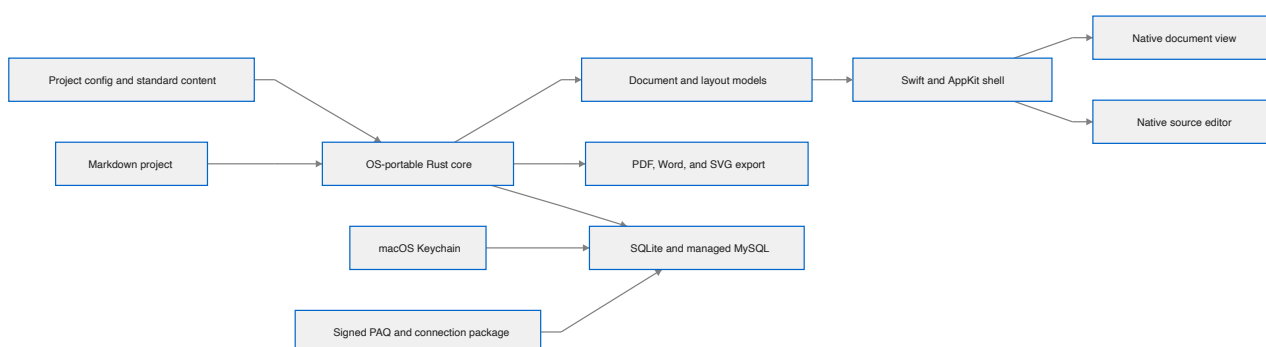
DocCrate is built around an OS-portable Rust core with native user-interface shells. On macOS, the shipping desktop application is a Swift/AppKit shell that links the Rust core through a small C ABI bridge. The earlier Windows shell uses Win32 and Direct2D/DirectWrite, but it is not the architecture of the Mac user interface.

This separation is deliberate: document meaning, validation, database access, diagram parsing, and export policy should behave consistently on every supported operating system, while each platform uses its own native windowing, accessibility, credential store, and drawing APIs.

Layers

Layer	macOS implementation	Responsibility
Native application	Swift, AppKit, Core Graphics, PDFKit, WebKit where required for preview	Application windows, menus, toolbar, sidebar, source editor, accessibility, native dialogs, Keychain integration, and user interaction.
Bridge	macos/CocoaDocCrate/RustBridge	Ownership-safe C ABI between Swift and Rust; converts core results into Cocoa-facing models.
Portable core	Rust modules under macos/src/ and shared Rust modules/crates	Project scanning, Markdown parsing, source mappings, layout data, pagination policy, exports, database access, credentials workflow, and automation services.
Diagram engine	DocCrate's fork of Selkie plus DocCrate diagram adapters	Parses Mermaid-compatible source, lays out supported diagram families, and supplies a platform-neutral diagram scene.
Storage and services	SQLite, MySQL over mandatory TLS, filesystem, Keychain adapter	Saved DocCrates, standard content, revisions, managed checkouts, connection packages, and secure credentials.

macOS Runtime Flow



The Cocoa application never needs to reimplement Markdown, database, or diagram semantics. It asks the bridge for project and document models, draws them with native Mac controls, and sends edits back through the same core save and validation paths used by automation.

Portable Rust Core

The core is portable by design. Platform-neutral responsibilities include:

- recursive project discovery and Markdown loading;
- CommonMark parsing with source-line and source-range tracking;
- project-variable and governed standard-content expansion;
- diagram parsing and platform-neutral scene construction;
- semantic pagination and page furniture;
- PDF, SVG, rendered Word, and editable Word generation;
- saved-DocCrate revisions and managed checkout rules;
- SQLite and mandatory-TLS MySQL access;
- signed connection-package and PAQ verification;
- MCP, TCL, and headless command services.

Platform-specific code is kept at the edge. The Mac adapter uses Keychain and macOS resolver/storage conventions. The legacy Windows adapter uses Windows Credential Manager and Win32 conventions. Neither credential-store API belongs in the document model or database protocol.

Cocoa Application Shell

The Mac shell is under `macos/CocoaDocCrate/`.

Area	Responsibility
DocumentWindowController.swift	Main window, toolbar, menus, project lifecycle, exports, database operations, and automation dispatch.
DocumentViewController.swift	Rendered/source modes, native text editing, find and replace, source tracking, and selection.
ProjectNavigatorViewController.swift	Recursive project navigation and document selection.
MacPageOutputModel.swift	Cocoa-side page source, pagination progress, and print/export coordination.
MacApplicationStorage.swift	Application Support, cache, connection package, CA certificate, and staged PAQ locations.
RustBridge/	Swift-compatible wrapper around the portable Rust core.

AppKit owns normal Mac behavior: menu commands, sheets, clipboard, keyboard shortcuts, light/dark appearance, VoiceOver metadata, security-scoped folder access, and Keychain-backed credentials. Generated error reports are presented in readable dialogs with a copy-to-clipboard action.

Document Pipeline

```
project scan
-> load Markdown source
```



```
-> apply project configuration and governed substitutions
-> parse blocks with source locations
-> parse diagrams into platform-neutral scenes
-> create layout and pagination models
-> AppKit viewer / PDF / SVG / Word / inspect / lint
```

Source locations survive parsing and layout. That allows the rendered document, source editor, graphical editors, agent referrals, and diagnostics to refer to the same section of Markdown without maintaining a second document format.

Mermaid Diagrams and the Selkie Fork

DocCrate uses a maintained fork of the MIT-licensed [Selkie](#) Rust project. Selkie provides a native Mermaid-compatible parser and layout engine without requiring JavaScript, Node.js, Chromium, or a web renderer at runtime.

The fork is vendored under `selkie/` and adapted for DocCrate. DocCrate uses it as a Rust library with optional CLI and bitmap features disabled in the desktop dependency. The integration:

1. reads a fenced `mermaid` block;
2. parses it with the Selkie fork;
3. lowers the result into DocCrate's platform-neutral diagram scene;
4. applies DocCrate annotations, manual layout overrides, and custom shapes;
5. renders the scene through the Mac canvas or an export backend.

This design keeps authoring source broadly Mermaid-compatible while allowing native Mac display, deterministic export, stable diagram IDs, and DocCrate's graphical editing features. The separate `blockgraph` format remains a DocCrate-specific card-and-edge DSL.

Selkie retains its own MIT notice and copyright. See [Licences and Permitted Use](#).

Pagination and Export

Pagination is a core service rather than a Cocoa view calculation. The Mac app requests a page-output model, shows progress in the toolbar, and removes the indicator when planning and page generation complete. Page furniture, explicit page breaks, keep-together ranges, and semantic break rules therefore apply to both preview and exported output.

PDF, SVG, and Word paths reuse the parsed document and diagram scenes. Platform views are not screen-captured to make the document export.

Data and Credential Boundaries

The Mac app, embedded CLI, and MCP server share these default locations:

Purpose	Location
Application data	~/Library/Application Support/Cocoa DocCrate/
Generated/cache data	~/Library/Caches/Cocoa DocCrate/

Database password	macOS login Keychain
Public connection metadata	Signed doccrate-db-connection.zip in Application Support
RDS trust certificate	rds-ca.pem in Application Support or an approved bundled fallback

The public connection package does not contain the database password. The signed PAQ is transient: after successful verification and Keychain storage it is removed from its staging location.

Automation

- The embedded `doc-crate` helper provides headless validation, inspection, database, and export commands.
- `doccrate-mcp` exposes agent-oriented tools and discovery resources.
- AppleScript and Apple Events automate the live AppKit application.
- TCL supplies repeatable local project workflows.

The old Windows live-viewer control channel is not used to automate the Cocoa window. See [AppleScript and Apple Events](#) and [macOS Command Line](#).

Platform Boundaries

Documentation that names `.exe`, `Win32`, `Direct2D`, `DirectWrite`, `Windows Credential Manager`, `%LOCALAPPDATA%`, or `C:\...` describes the legacy Windows shell or a Windows-only example. It must not be read as the behavior or install location of the Mac application.

Back to [Home](#) | [Implementation](#)

Glitz Asset Metadata

DocCrate is registered as a SecurityOperations asset.

Field	Value
Asset	SecurityOperations
Asset area ID	18590
Asset area name	docrate
Asset identity key	urn:asset:2249:securityoperations:18590:docrate
Description	documentation utility

Repository

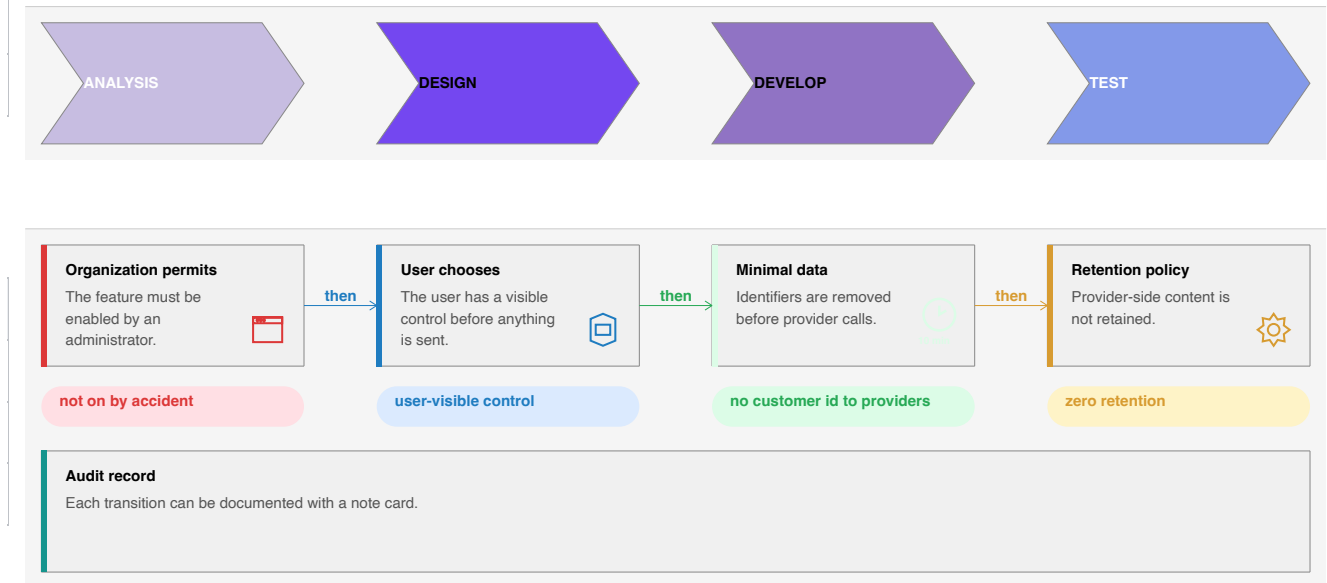
Field	Value
Repository	In-internal/docrate
Catalog owner	securityoperations
Code owners	@In-internal/securityoperations

Block Views

DocCrate supports a custom fenced block named `blockgraph`. It is not Mermaid or CommonMark; it is a compact DocCrate-only chart syntax for block views, control chains, review flows, and policy diagrams. *Very similar to blocks you might draw in powerpoint*

Use it when you want a row of cards with arrows between them, optional supporting notes, embedded shape icons, and small rounded label badges.

Use `card-shape=chevron` when the steps should read as a right-pointing strip:



Syntax

A block view is written in a fenced code block:

```
```blockgraph
// @id stable-id
// @name Human Name
step blue | Title | Detail text | option=value option="quoted value"
edge Label | green
note gold | Note title | Note detail
```
```

Lines are trimmed. Blank lines, `---`, and `//` comments are ignored. The `// @id` and `// @name` comments are optional metadata used by DocCrate's diagram inventory, snapshots, and navigation tools.

Cards

Cards use pipe-separated fields:

```
step <color> | <title> | <detail> [| options]
note <color> | <title> | <detail> [| options]
```

step cards form the main horizontal chain. **note** cards are the same block-card shape, but they are not part of the step-to-step edge chain. Use them for long supporting bands, comments, audit records, or callouts.

Grid annotations control card placement:

```
// @node note1 col=1 row=3 span=4 row-span=1
```

For a note card, **col** is the horizontal start column and **span** controls the horizontal end column ($\text{end} = \text{col} + \text{span} - 1$). **row-span=1** is normal height and **row-span=2** is double height. In the visual editor, drag a note to move its start row/column, drag resize handles to change **span** or normal/double height, and press Ctrl+N to add another note card.

Edges

Edges connect one step to the next in the order they appear:

```
edge <label> | <color>
```

If there are fewer edge lines than gaps between steps, DocCrate draws the missing arrows without labels. Extra edges are ignored once there are no more gaps to connect.

Options

The optional fourth card field accepts space-separated **key=value** tokens. Quote values that contain spaces.

| Option | Meaning |
|----------------------|--|
| card-shape=chevron | Render the card as a filled right-pointing chevron tile. Adjacent chevron steps suppress unlabeled default arrows. |
| icon=<shape> | Draw a named .shape icon inside the card, using the card accent stroke |
| icon-label="text" | Draw a small label below the icon |
| icon-pos=<position> | Icon position: top-left, top-right, bottom-left, bottom-right |
| icon-fill=<color> | Fill color for the icon, or none |
| icon-size=<number> | Icon size in DIPs, clamped by the renderer |
| icon-stroke=<color> | Legacy accepted option; rendered cards use the card accent stroke to match the visual editor |
| badge="text" | Draw a rounded label badge for the card |
| badge-pos=<position> | Badge position: below or above |
| | |

| | |
|--------------------------|------------------------------|
| badge-fill=
<color> | Badge fill color |
| badge-stroke=
<color> | Badge outline color, or none |
| badge-color=
<color> | Badge text color |

Short aliases are also accepted for icon options: `shape`, `label`, `pos`, `fill`, and `size`. Legacy `stroke` and `color` tokens are accepted but normalized by the visual renderer/editor.

Colors

Colors may be hex values or one of DocCrate's named colors:

| Name | Hex |
|-----------------|---------|
| blue | #1f7cc0 |
| green | #27a956 |
| gold, yellow | #d79b31 |
| magenta, purple | #c21ed6 |
| teal | #15948d |
| red | #dc3838 |
| gray, grey | #808080 |

Shapes

Icons use DocCrate's shape registry. Built-in shapes are loaded from the application resources, and document-specific shapes can be added under `.shapes/` in the docs root.

The `clock` shape is available as a built-in icon:

```
step gold | Temporary cache | Content expires automatically. | icon=clock icon-label="10 min"
icon-size=34
```

Command Line Help

The CLI can list supported chart types and print syntax help:

```
doc-crate --chart-types
doc-crate --chart-help blockgraph
doc-crate --chart-help mermaid-flowchart
```

Use `--validate <file> --json` after editing a block view. It runs the parser and layout pipeline without opening a viewer.

Back to [Home](#) | [User Guide](#)

Build the macOS App

This page covers the native macOS build that supplies the documentation bundle you are reading. Windows release scripts and `.exe` files belong to the legacy Windows shell and are not required to build the Cocoa application.

Requirements

| Tool | Purpose |
|---------------------------------------|--|
| A supported macOS development machine | Runs Xcode and the native AppKit target. |
| Xcode with Command Line Tools | Compiles Swift/AppKit, links the Rust static library, and builds the app bundle. |
| Rust stable toolchain | Builds the portable Rust core, bridge, CLI, and MCP helper. |
| Git | Source control and dependency checkout. |
| Network access on the first build | Downloads Rust crates unless Cargo's cache is already populated. |

Check the toolchain:

```
xcodebuild -version
swift --version
rustc --version
cargo --version
git --version
```

Repository Layout

| | |
|--|--|
| <code>docs/</code> | bundled in-app documentation |
| <code>macos/src/</code> | portable Rust core used by the Mac build |
| <code>macos/CocoaDocCrate/</code> | Xcode application and Swift sources |
| <code>macos/CocoaDocCrate/RustBridge/</code> | static Rust-to-C bridge |
| <code>selkie/</code> | DocCrate fork of the Selkie diagram engine |
| <code>crates/</code> | shared local Rust crates |

Debug Build

From the repository root:

```
xcodebuild \
  -project macos/CocoaDocCrate/CocoaDocCrate.xcodeproj \
  -scheme CocoaDocCrate \
  -configuration Debug \
  -derivedDataPath /tmp/docrate-cocoa-debug \
  CODE_SIGNING_ALLOWED=NO \
```



```
build
```

The app is written under:

```
/tmp/docrate-cocoa-debug/Build/Products/Debug/CocoaDocCrate.app
```

Launch it with:

```
open /tmp/docrate-cocoa-debug/Build/Products/Debug/CocoaDocCrate.app
```

The Xcode target runs build phases that compile the Rust bridge, copy the embedded `doc-crate` and `docrate-mcp` helpers, copy this `docs/` directory, and include approved optional library payloads when configured.

Rust Tests

Run the portable Mac core tests separately:

```
cargo test --manifest-path macos/Cargo.toml
```

Formatting checks:

```
cargo fmt --manifest-path macos/Cargo.toml --check
```

The Selkie fork is its own Rust package. When diagram-engine changes are in scope, run its relevant tests with:

```
cargo test --manifest-path selkie/Cargo.toml --no-default-features
```

Documentation Bundle

The Xcode project references the repository's top-level `docs/` directory as a folder resource. A clean build copies it to:

```
CocoaDocCrate.app/Contents/Resources/docs/
```

Edit source files under `docs/`, not files inside Derived Data or an existing `.app` bundle. Rebuild to refresh the bundled copy.

Release Build

A local unsigned compilation check can use `Release` in place of `Debug`, but a distributable build must follow the approved release process. That includes:

- the intended release bundle name and version;
- Developer ID signing and hardened runtime settings;

- notarisation and stapling;
- approved embedded helper binaries;
- the release signing public key where signed payload verification requires it;
- complete third-party notices;
- exclusion of passwords, PAQs, local databases, checkouts, and generated customer documents from the application bundle.

The SQLite content library, signed connection package, signature, CA certificate, and transient PAQ are operational components installed outside the application. See [Install on macOS](#).

Common Problems

| Symptom | Check |
|-----------------------------------|--|
| Xcode cannot find Rust | Ensure the Xcode build environment can resolve cargo; inspect the Rust bridge build phase. |
| Linker cannot find docrate_core | Build with the Xcode project so the Rust bridge phase runs before Swift linking. |
| Cargo dependency download fails | Restore network access or use a populated Cargo cache. |
| App opens with old documentation | Delete or change Derived Data, rebuild, and confirm the source docs/ folder was copied into the launched bundle. |
| Database TLS reports a missing CA | Install rds-ca.pem in ~/Library/Application Support/Cocoa DocCrate/. |
| macOS blocks a distributed build | Confirm Developer ID signing, notarisation, and Gatekeeper status; unsigned Debug builds are for development only. |

Use Restrictions

The branded fork is for the exclusive commercial use of LexisNexis and is not for redistribution or republication. See [Licences and Permitted Use](#) before packaging or sharing a build.

Back to [Home](#) | [Implementation](#)

cloud review doc

This page intentionally left blank *probably*



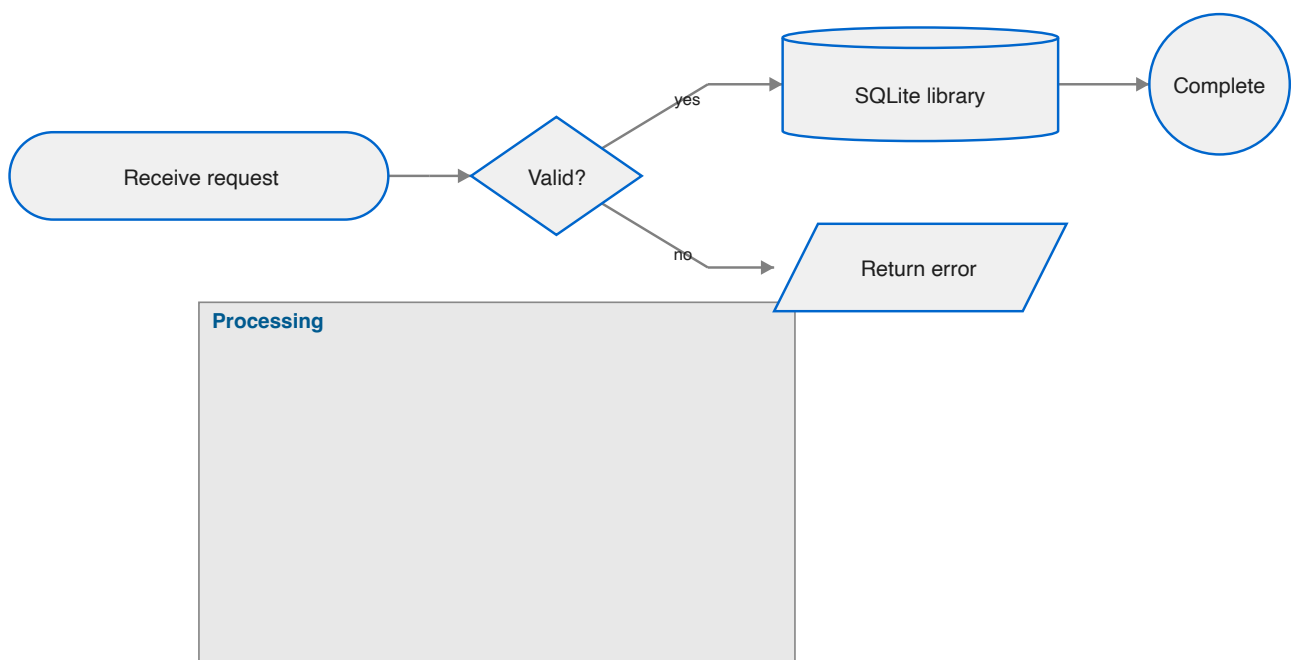
Diagram Editor Acceptance Suite

This document is a durable visual-editor fixture for the macOS port. It covers every diagram family exposed by Diagram Studio, plus several variants that exercise groups, notes, labels, relationships, shapes, and specialised items.

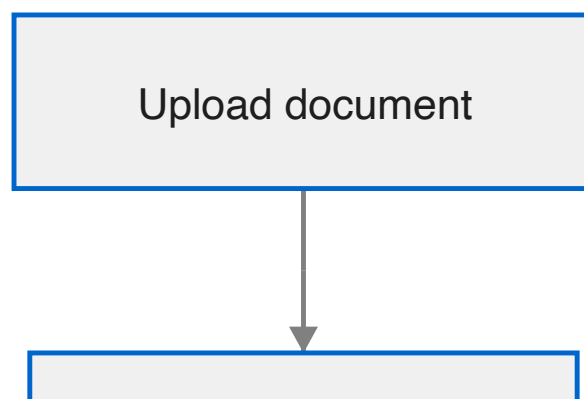
Use the diagram editor without saving exploratory changes. The acceptance run opens every diagram, selects representative objects and links, exercises the controls that are meaningful for that family, then cancels the changes so this file remains a stable baseline.

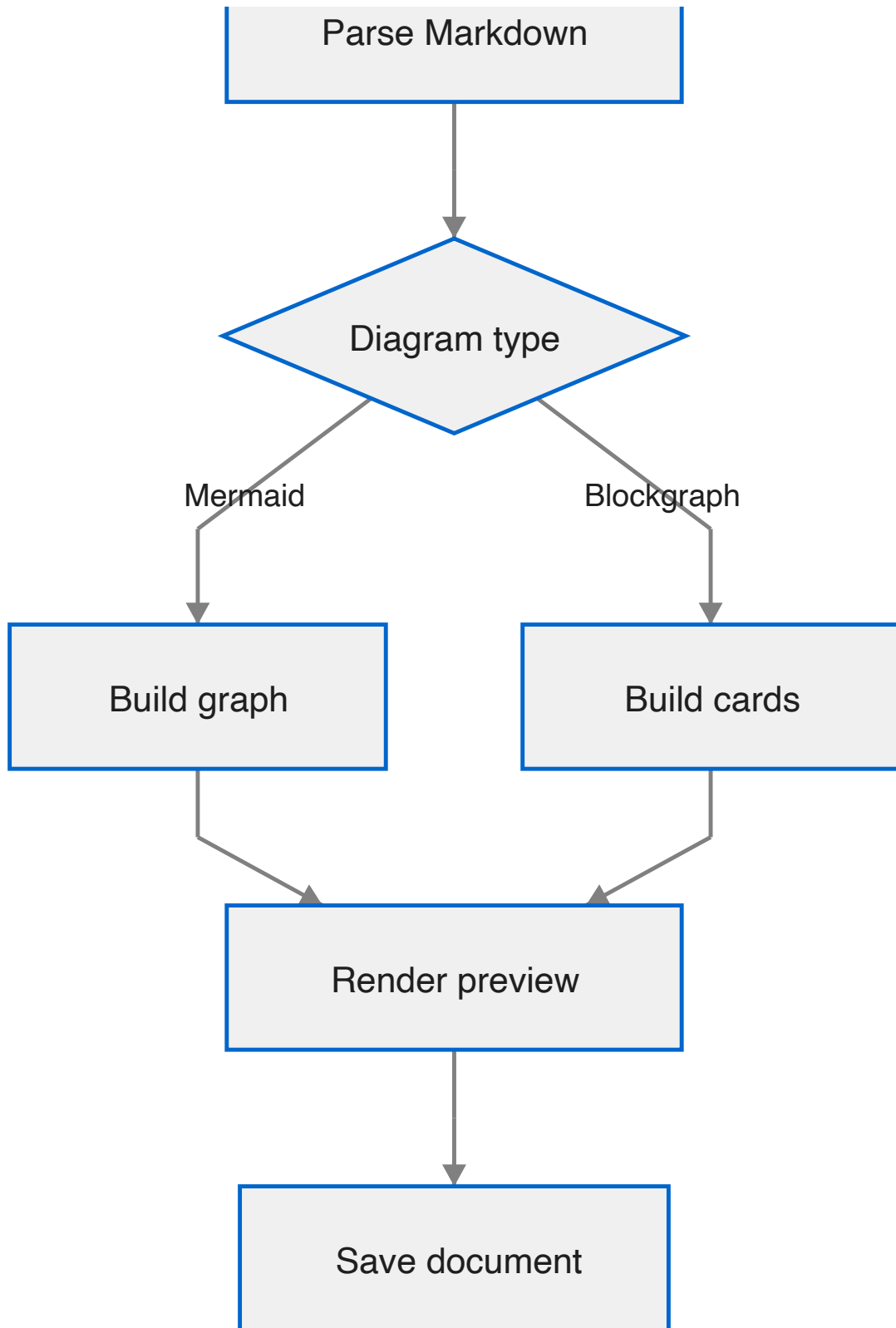
Flowcharts

Shapes, labels, and a group



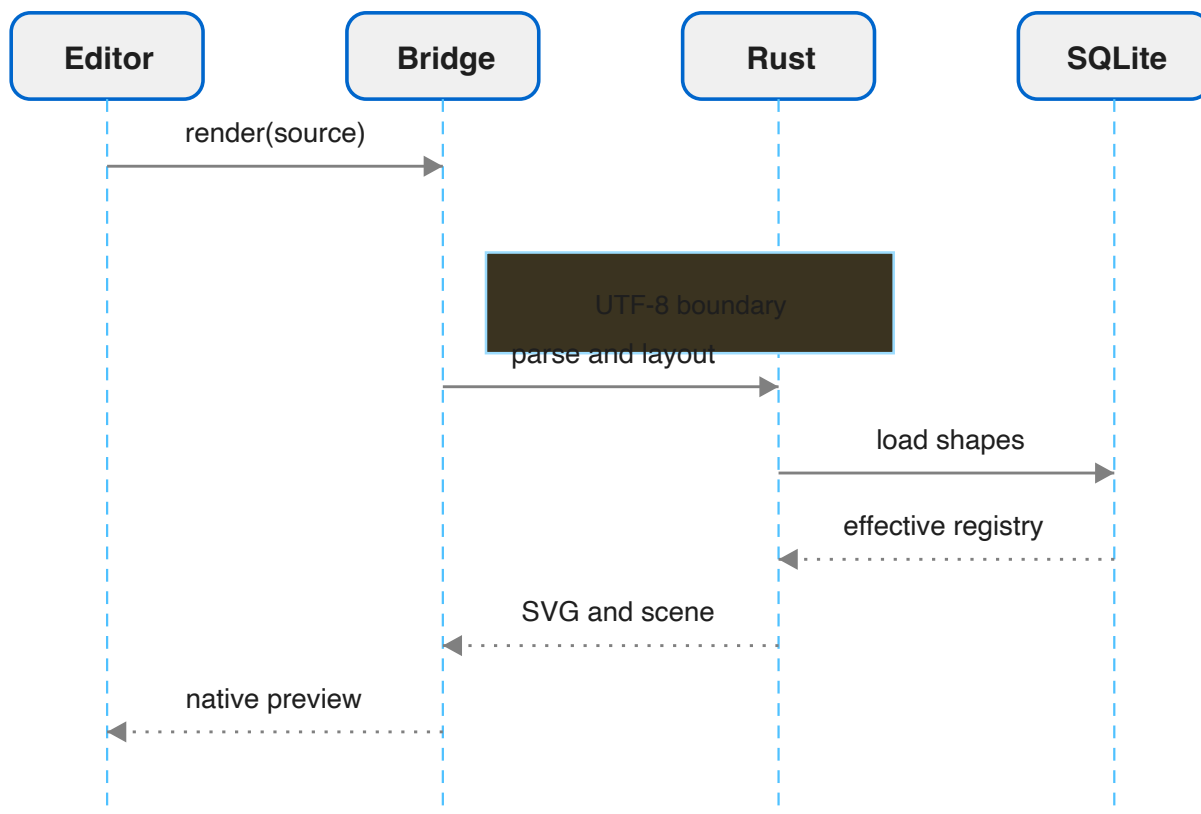
Branching service workflow



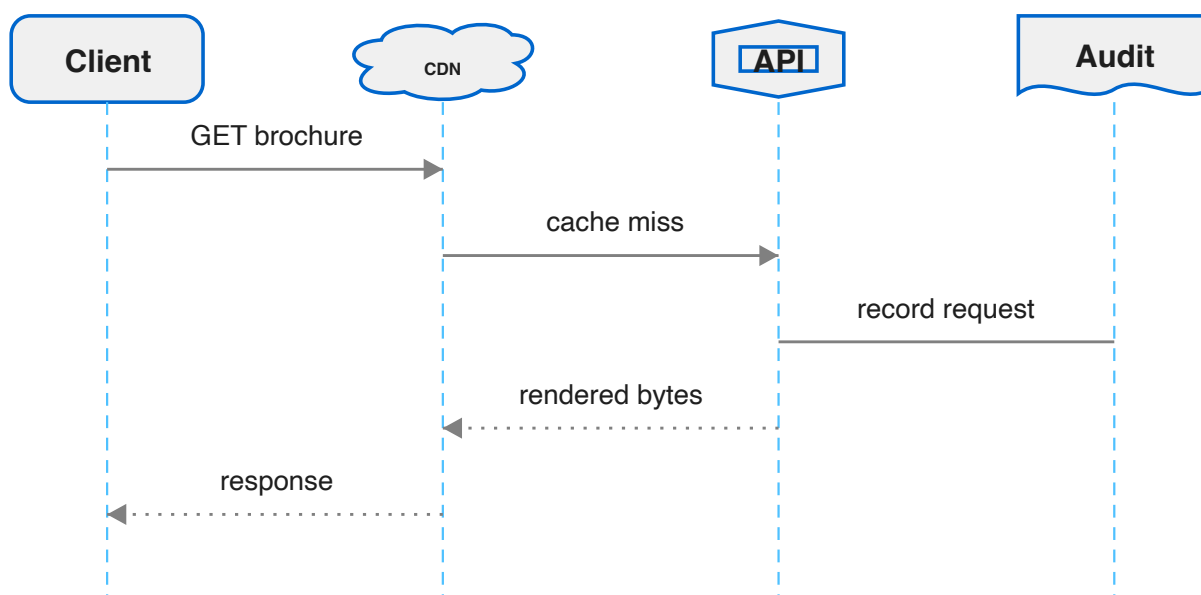


Sequence diagrams

Request, notes, and responses

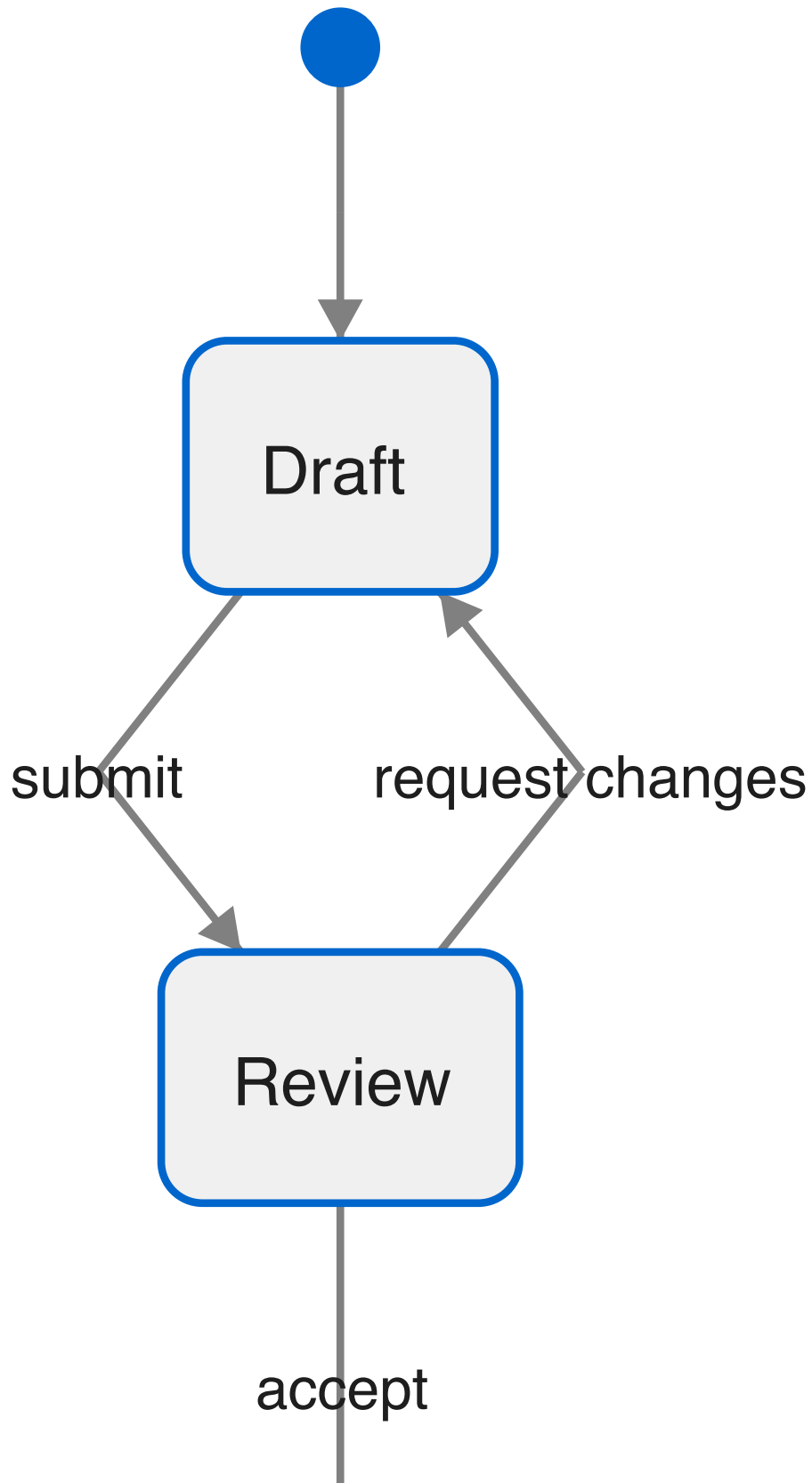


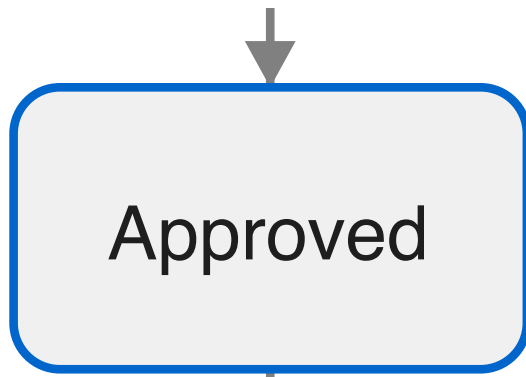
Custom-shaped participants



State diagrams

Document lifecycle

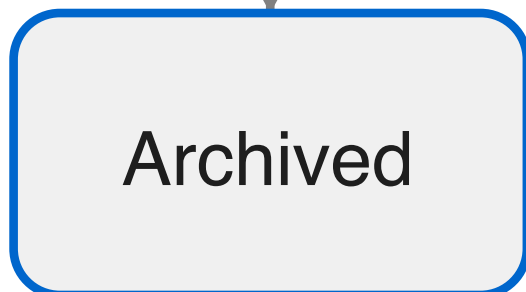




publish

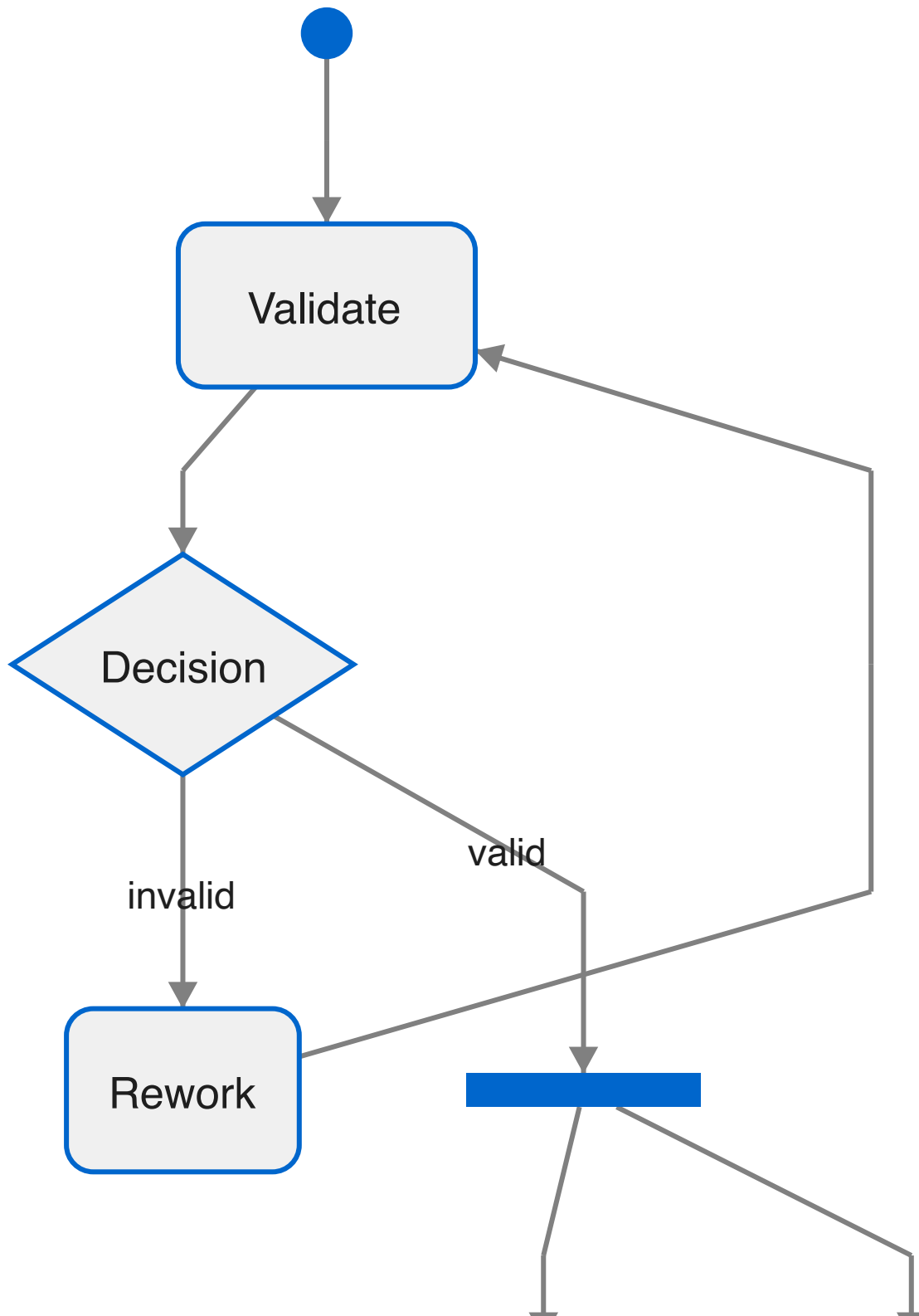


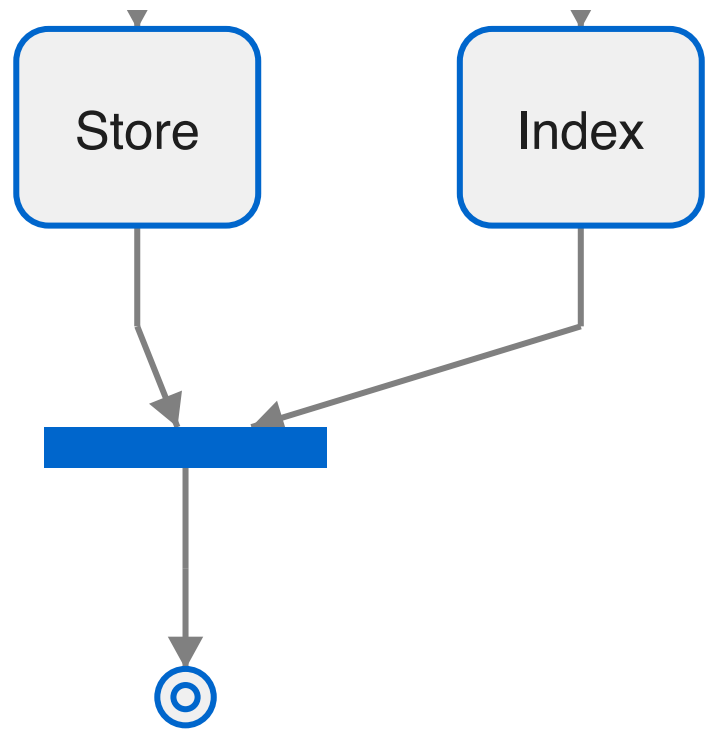
retire





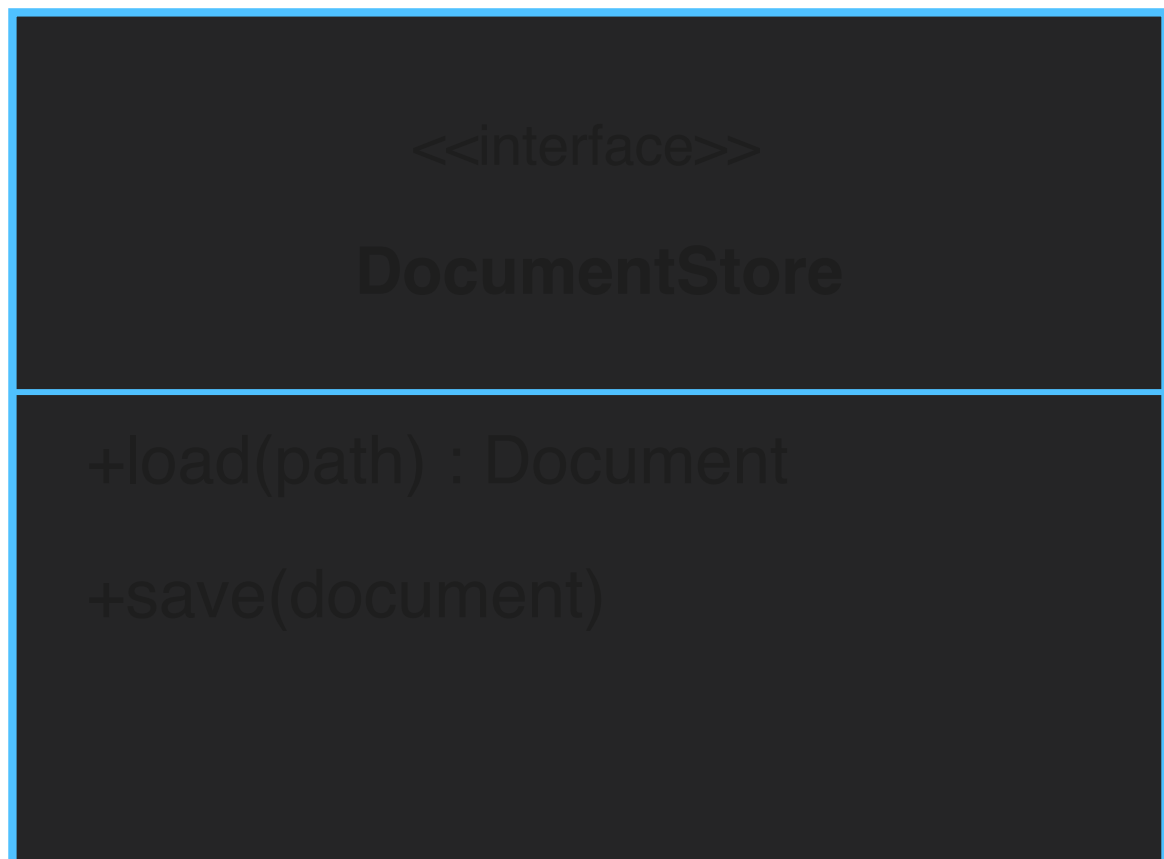
Choice, fork, and join

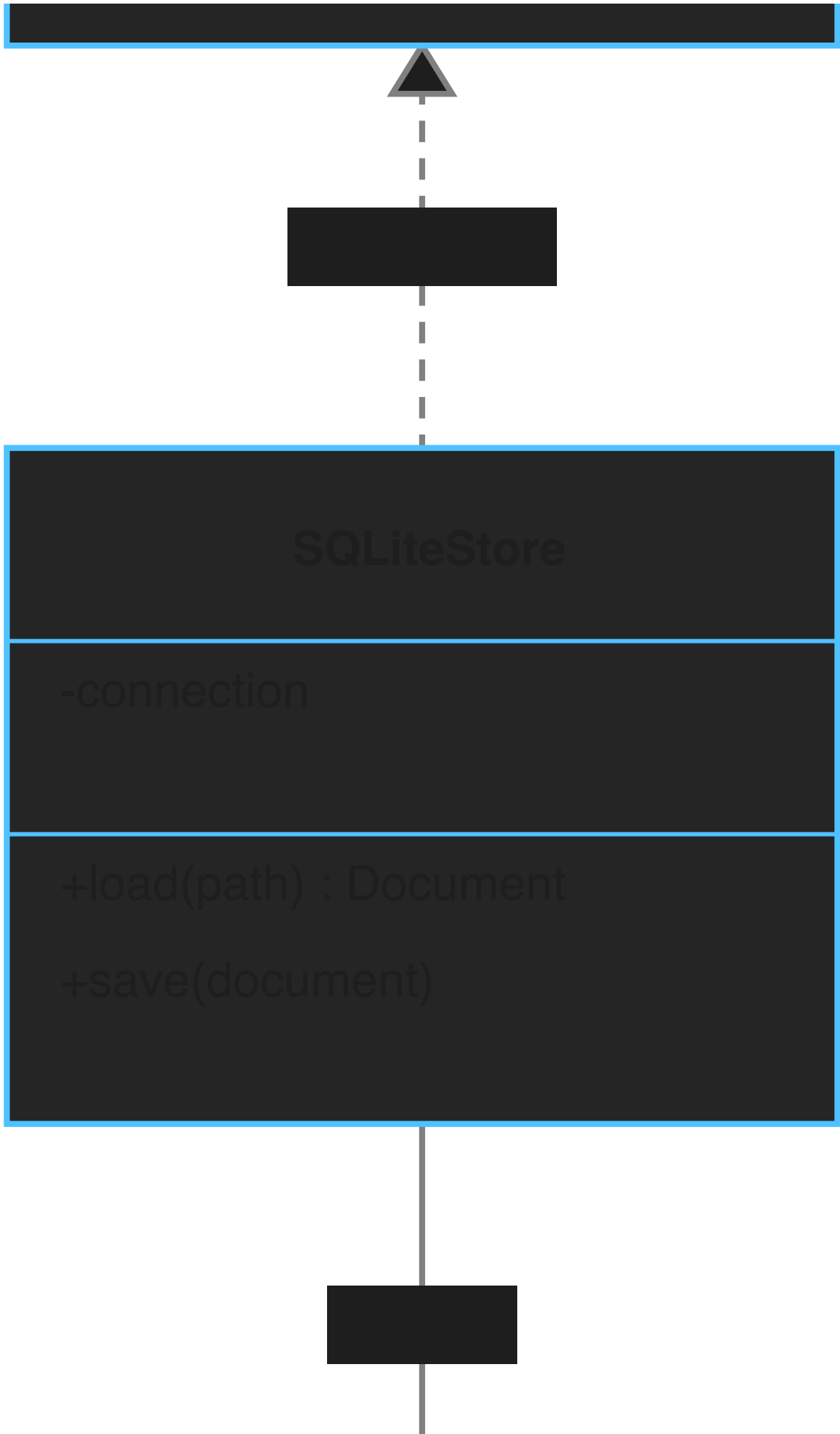




Class diagrams

Interfaces, members, and inheritance





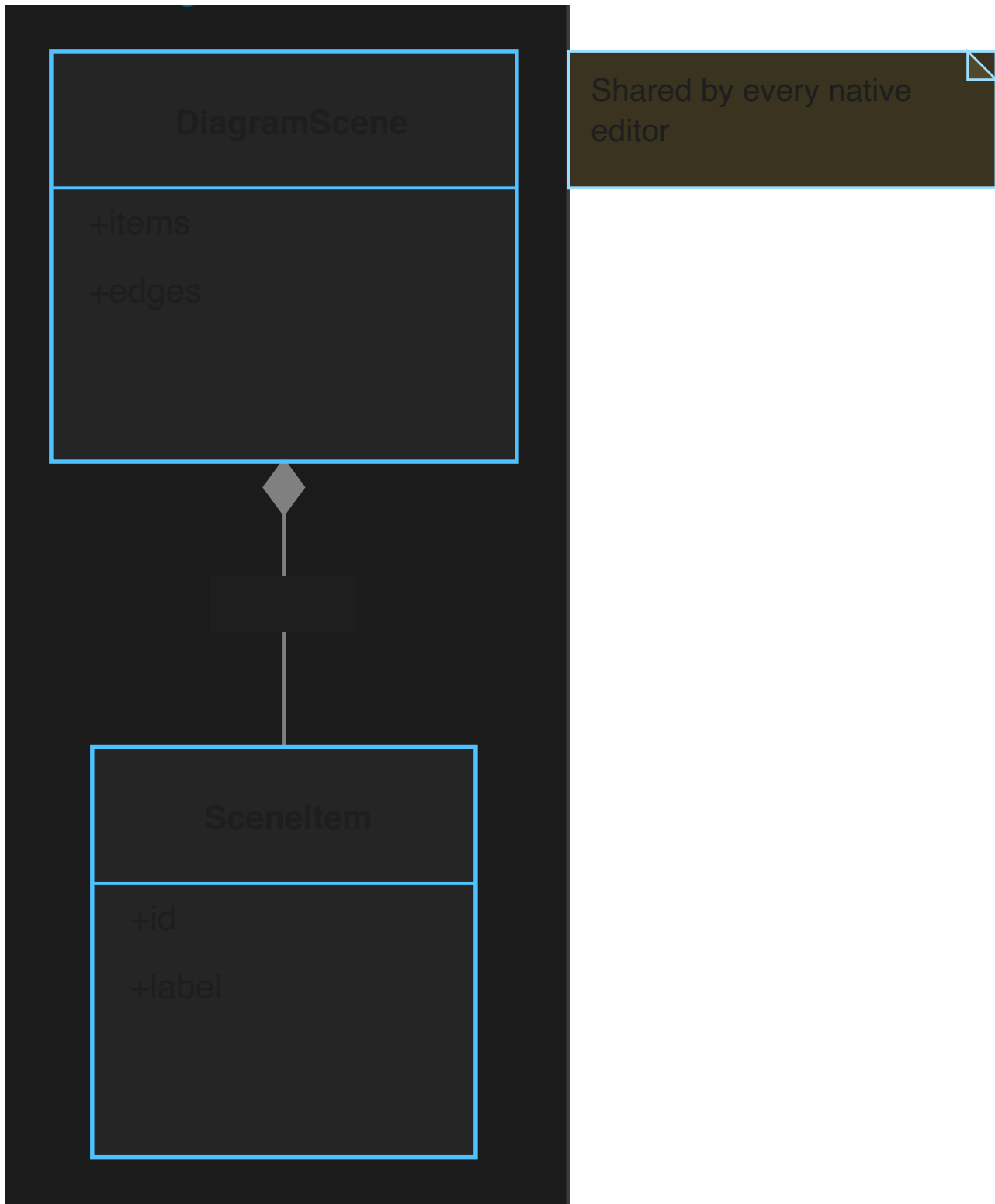


Document

+String title

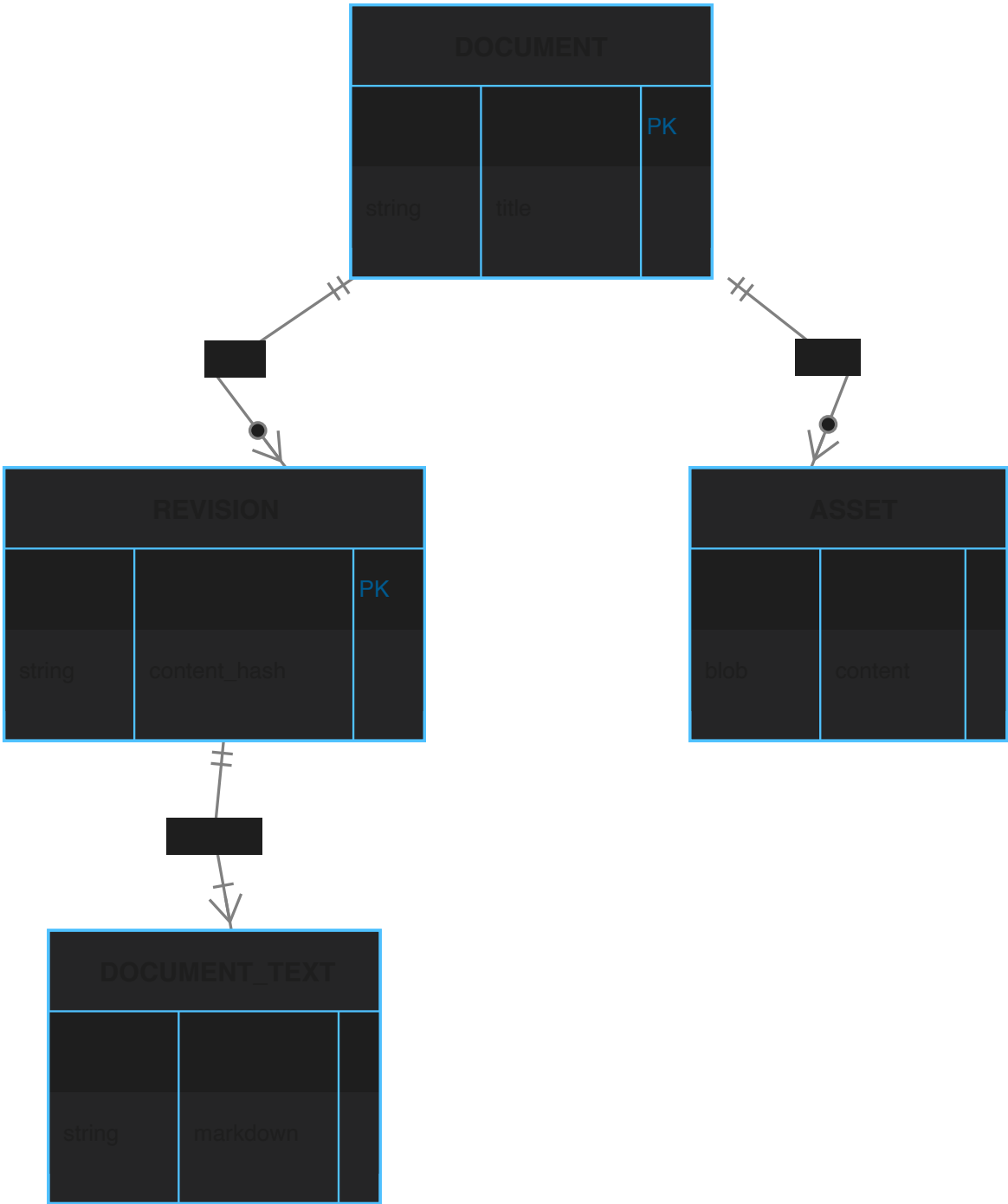
+String source

Namespace, composition, and note

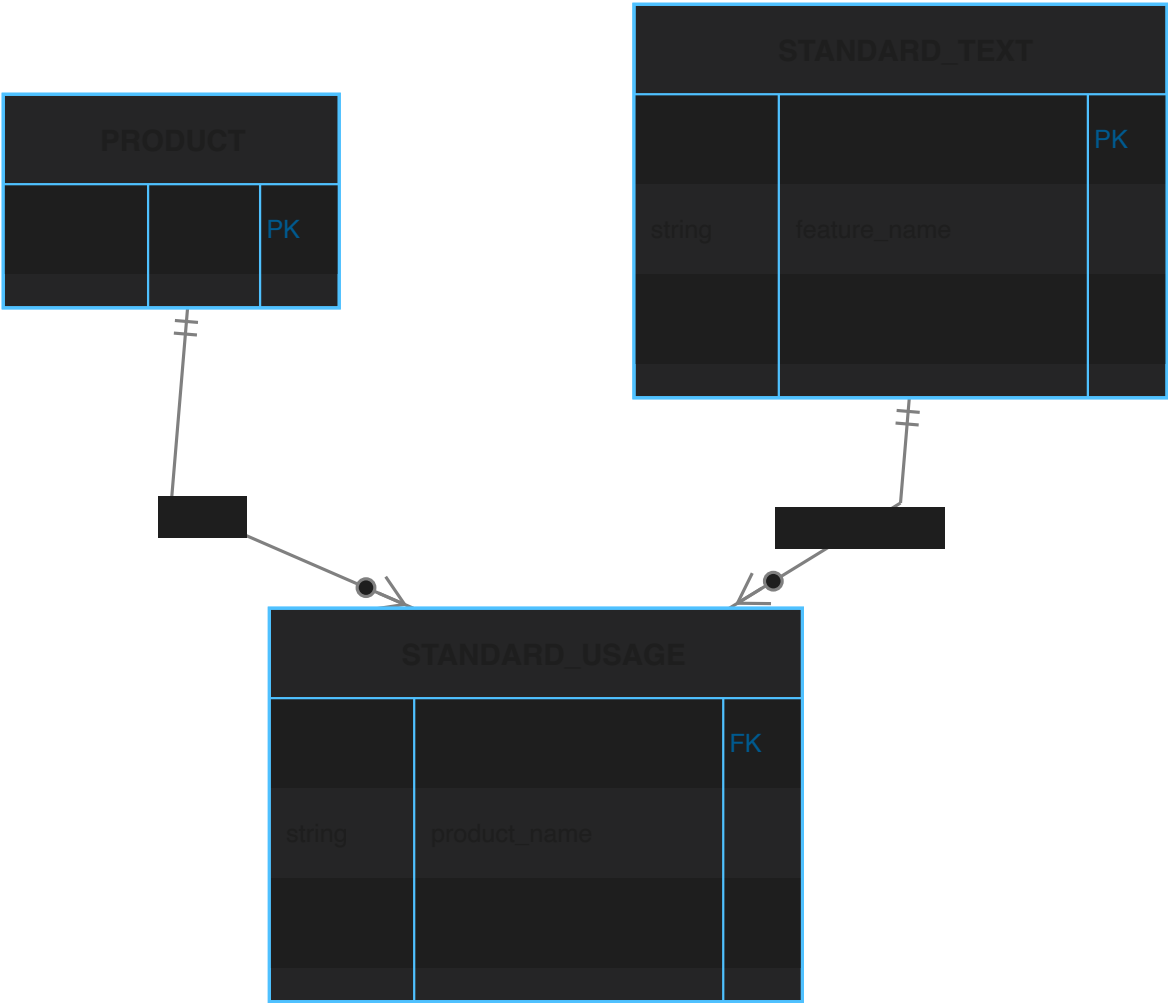


Entity-relationship diagrams

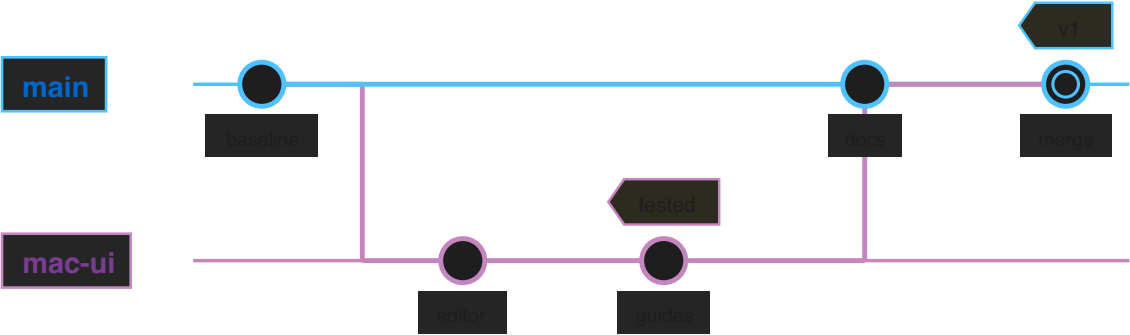
Library documents and revisions



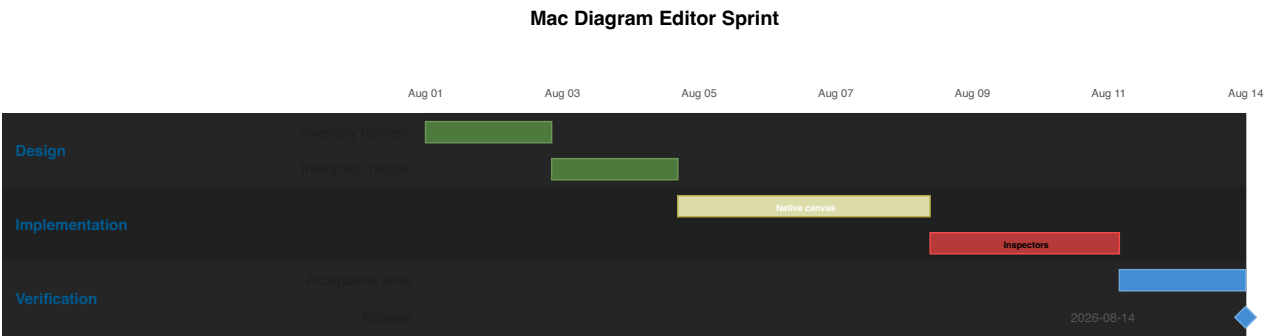
Standards and usage



Git graph

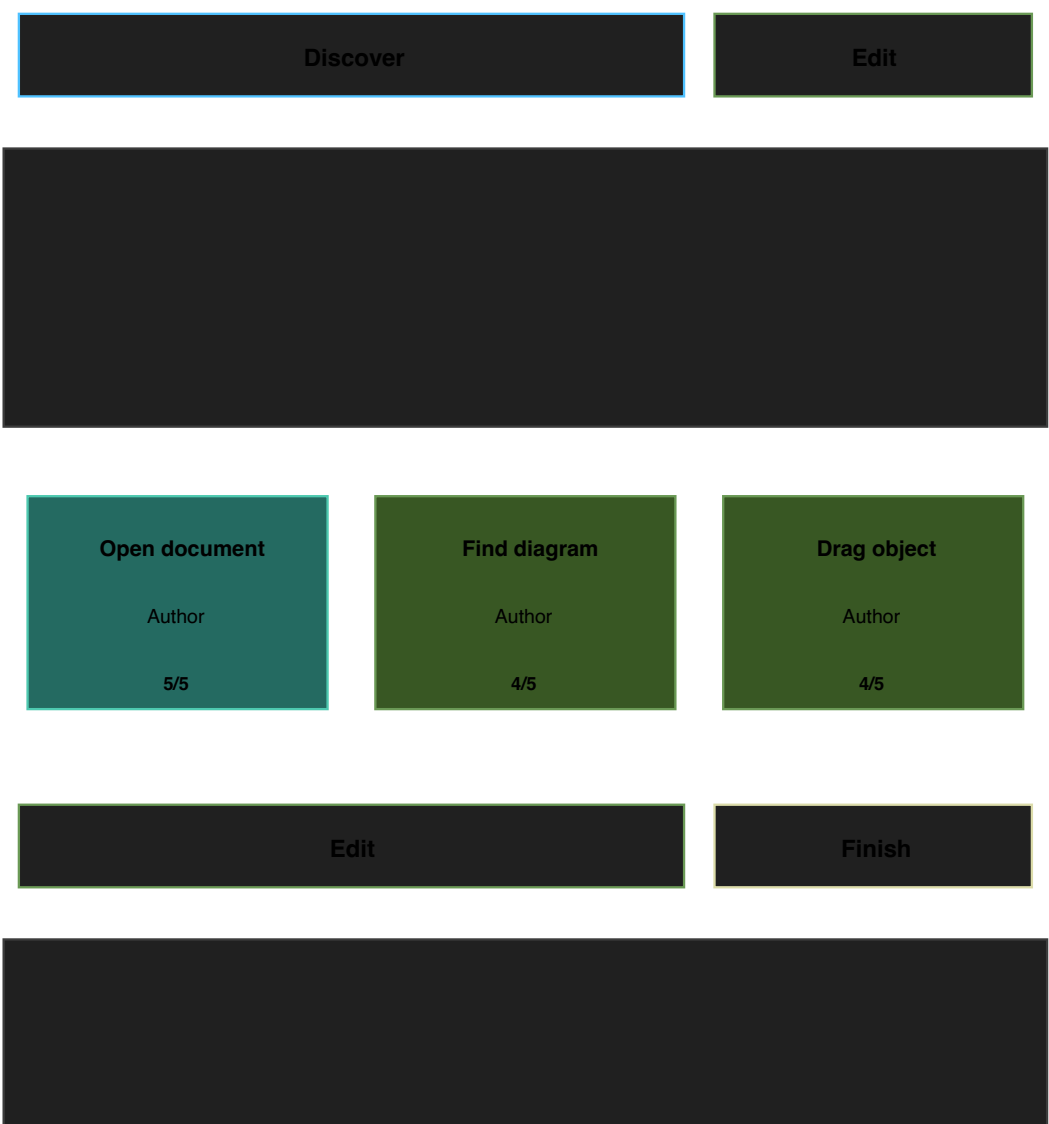


Gantt chart



User journey

Authoring a Diagram





Inspect attributes

Author

5/5

Edit label

Author

5/5

Validate preview

Author, Reviewer

4/5

Finish

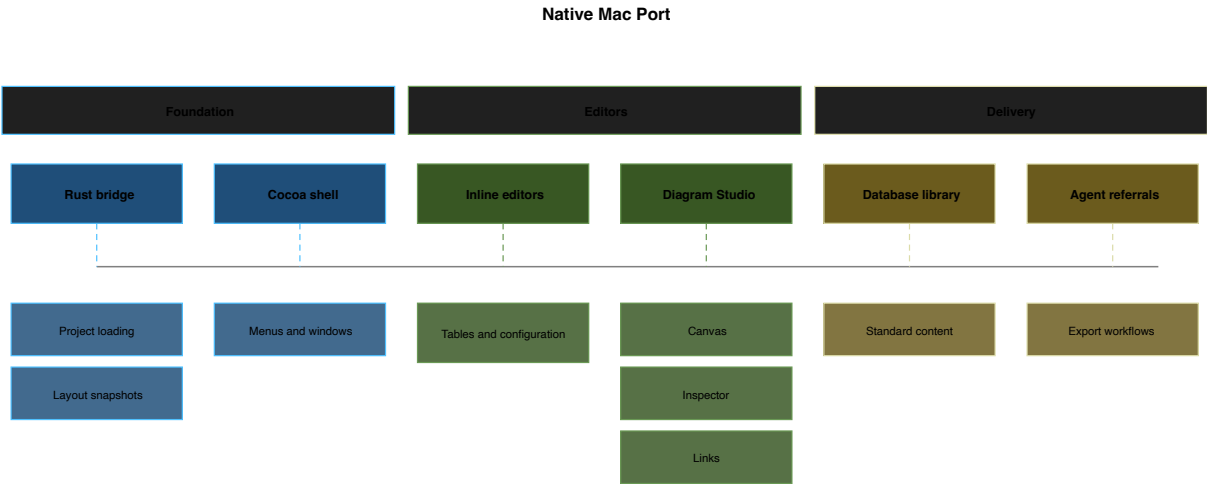


Save document

Author

5/5

Timeline

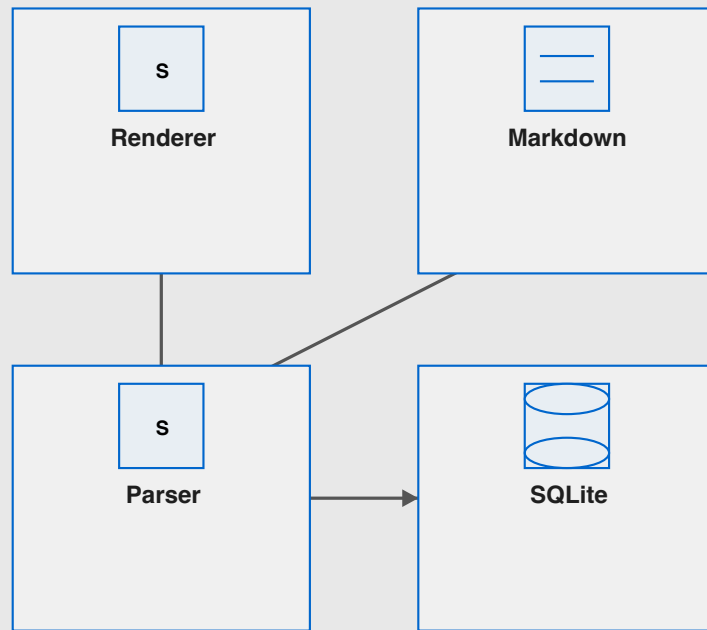


Architecture diagrams

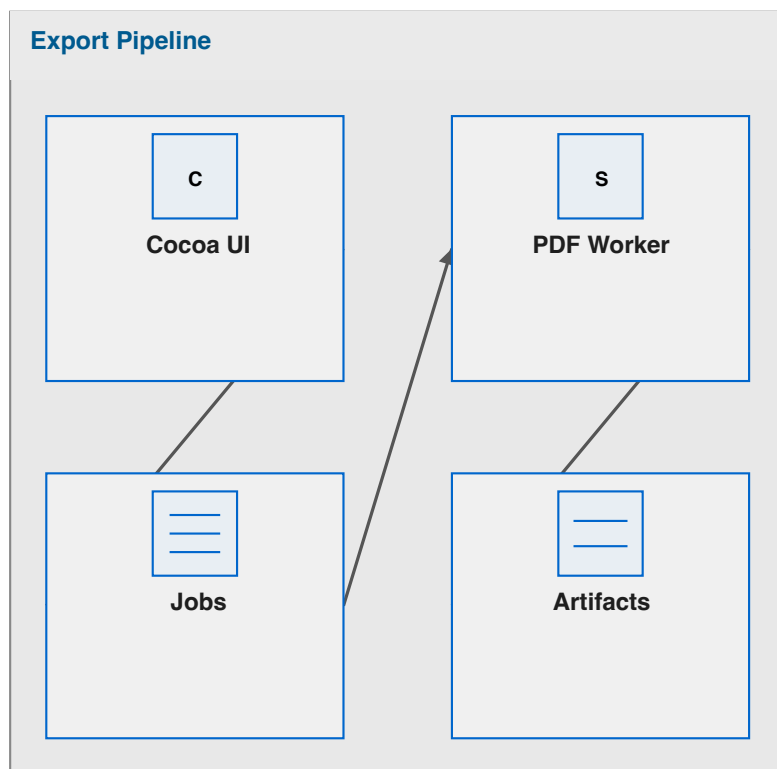
Grouped services and registered icons

DocCrate Runtime

Native Application

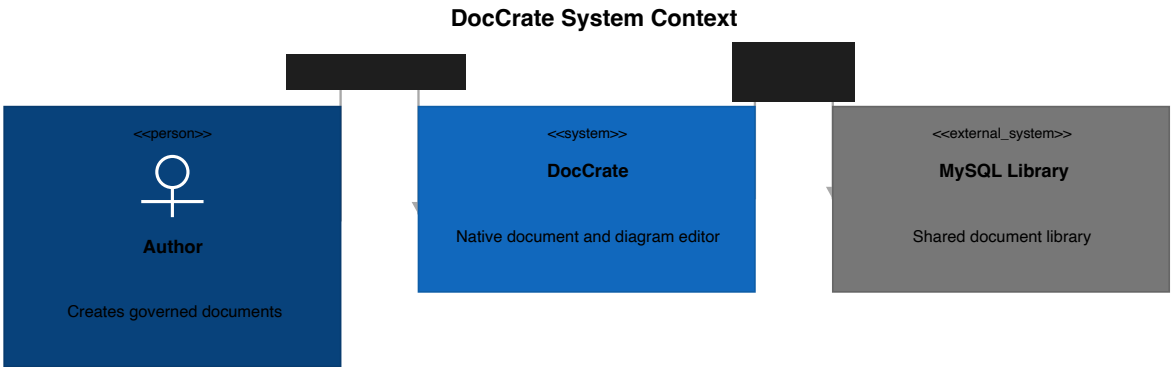


Queue-based export path

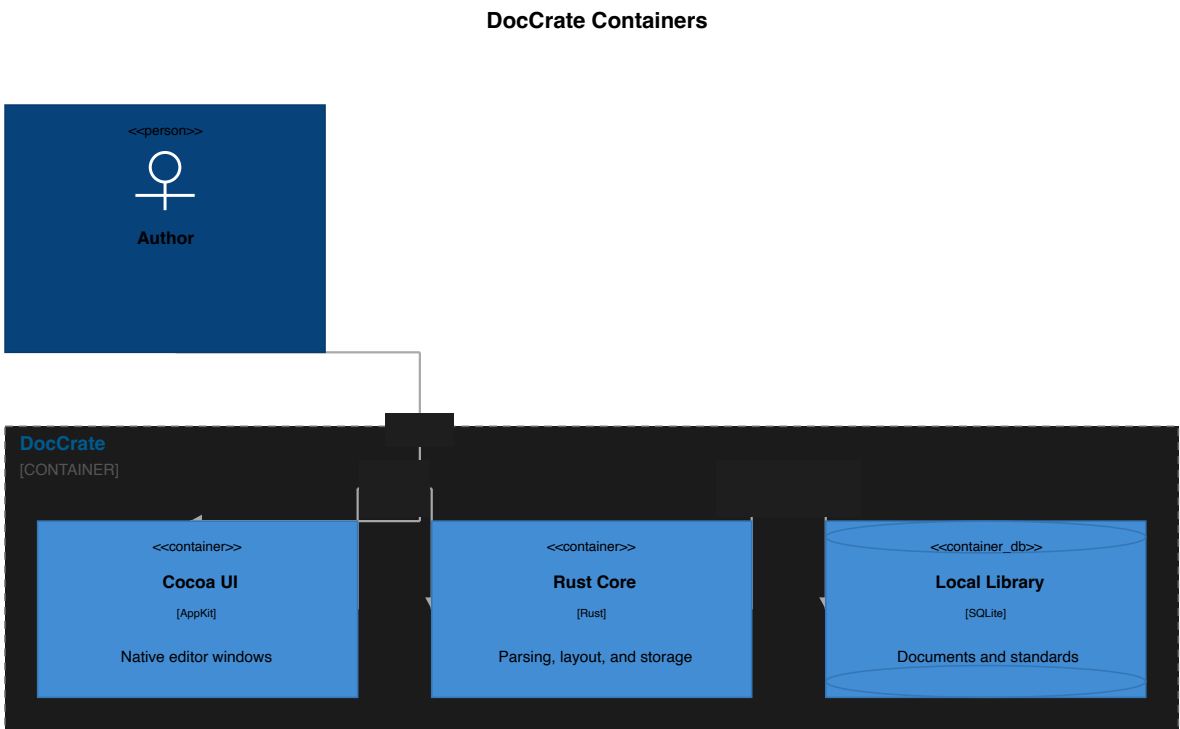


C4 diagrams

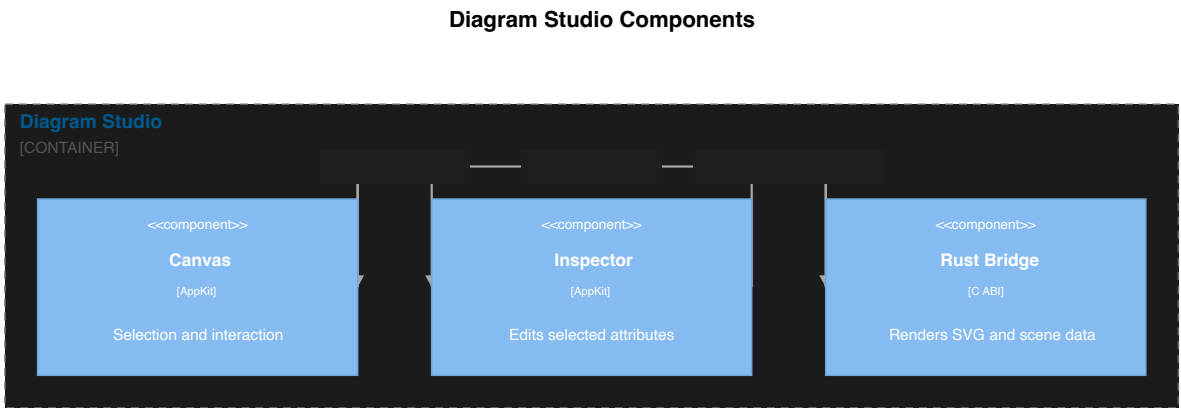
System context



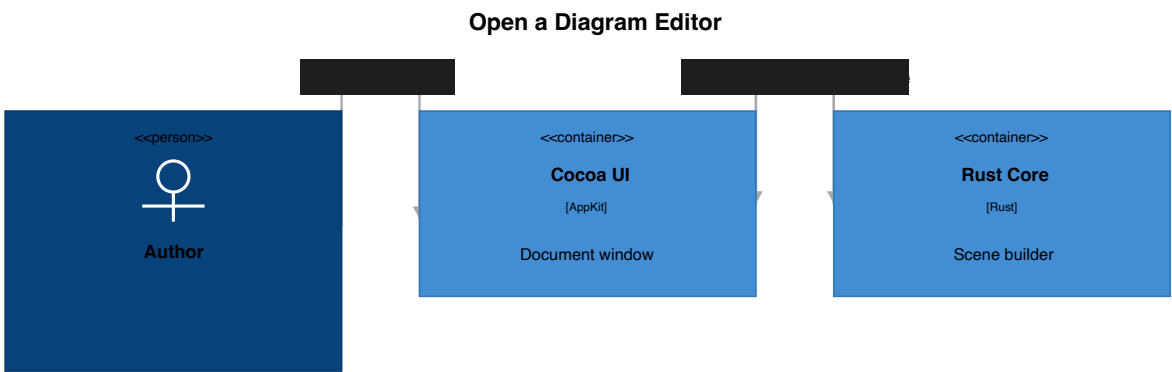
Container boundary



Component boundary

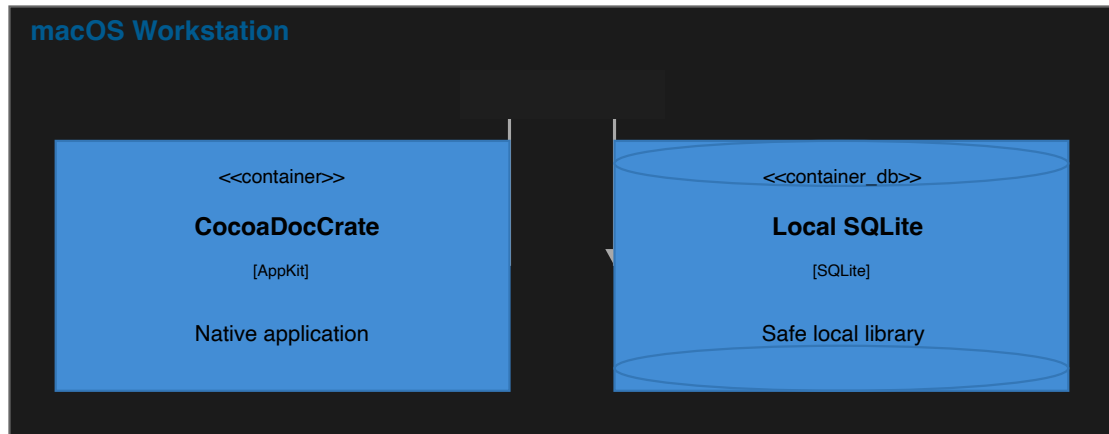


Dynamic interaction



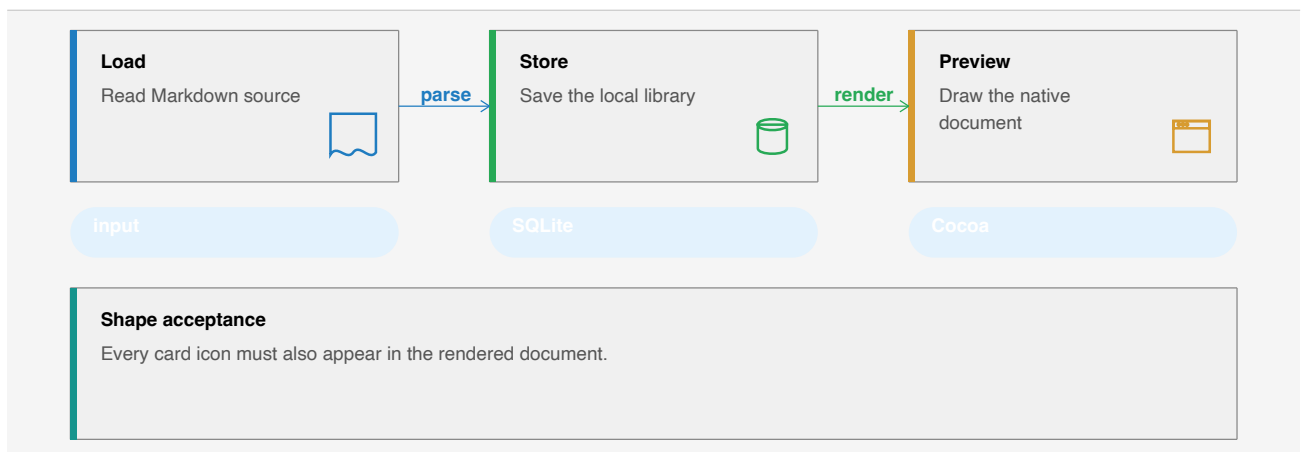
Deployment view

Local Mac Deployment

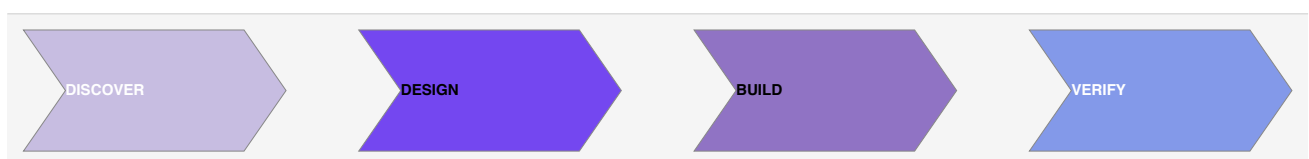


Blockgraphs

Cards, badges, registered shapes, and a note



Chevron process strip



macOS Feature Tour

DocCrate for Mac combines a native AppKit document application with a portable Rust document engine. This page highlights the features available from the Mac application; headless and agent functions are covered separately.

Native Mac Interface

- AppKit windows, menus, toolbar, sheets, clipboard, and accessibility.
- System, Light, and Dark appearance choices.
- Recursive project sidebar with back and forward navigation.
- Rendered Document and editable Source modes.
- Transient toolbar feedback while pagination is running.
- Native error-report dialogs with Copy to Clipboard for database failures.
- AppleScript and Apple Event automation of the live app.

The Mac shell does not use Win32, Direct2D, DirectWrite, or a Windows Credential Manager. Document semantics come from the portable Rust core.

Markdown

The Rust core parses CommonMark with tables and strikethrough enabled. The native canvas supports:

- headings from H1 through H6;
- paragraphs, emphasis, strong text, and inline code;
- fenced code blocks with syntax highlighting;
- blockquotes, ordered lists, and bullet lists;
- tables and toolbar-style image/link tables;
- local document links and external web links;
- local images;
- thematic rules;
- explicit `[[newpage]]` markers;
- source-line tracking between rendered and source views.

Source Editing and Find/Replace

Choose **View > Source** or press **Option-Command-2** for the complete Markdown source. The editor uses native macOS selection, undo, clipboard, spelling, and find behavior.

- **Command-F** finds text in the current document source.
- **Option-Command-F** opens Find & Replace.
- The toolbar Find & Replace button switches to Source and opens the replacement interface.
- Saving reparses the source, refreshes the rendered document, and starts a new page-output calculation.

Inline prose, tables, diagrams, and source spans can also be opened in focused editors from the rendered document.

Syntax Highlighting

Fenced code uses `syntect` with its pure-Rust regular-expression backend. Projects can add Sublime syntax definitions and themes under `.doccrate/` and select a highlight theme in `.doccrate/highlight.toml`.

```
pub fn example() -> &'static str {  
    "highlighted Rust source"  
}
```

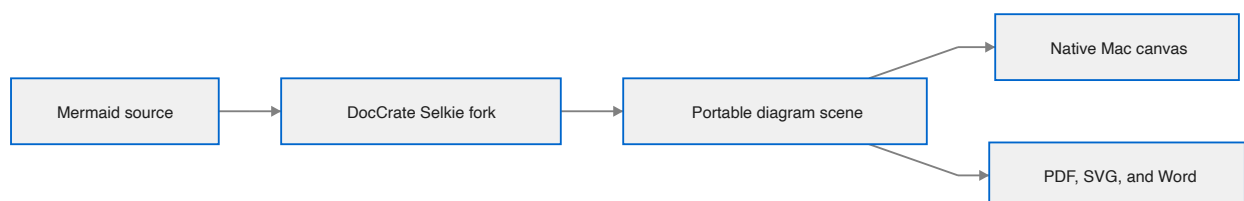
Diagrams

Mermaid-compatible diagrams are parsed natively through DocCrate's maintained fork of the MIT-licensed Selkie Rust project. No JavaScript or browser runtime is required for diagram parsing or layout.

Supported families include flowcharts, sequence, state, class, ER, Gantt, git graph, journey, timeline, architecture-beta, and C4, with additional families available as the fork develops.

DocCrate adds:

- stable diagram IDs;
- `%% @` annotations for manual position, route, group, and style overrides;
- native graphical editors for supported families;
- project-defined vector shapes under `.shapes/`;
- SVG clipboard and diagram-pack export;
- the separate DocCrate `blockgraph` card-and-edge format.



Project Configuration

`.doccrate/project-config.md` controls project metadata, reusable variables, themes, and export furniture. Standard-content tokens are expanded before parsing where the workflow permits it.

Project-Aware Spelling

The Rust spelling engine applies the brochure's English region, allow/deny lists, and governed replacements only to checkable prose. The Mac source editor shows those findings with native underlines and context actions without running Apple's checker across raw Markdown or silently applying corrections. See [Spelling on macOS](#).

Standard Content and Saved DocCrates

The Library menu provides native search and editing for governed standard text and diagrams. Saved DocCrates support database identity, revisions, managed checkouts, project history, and save-as-new workflows.

The local SQLite library is shared by the Cocoa app, embedded CLI, and MCP server. Optional managed MySQL sync requires the signed connection package, approved RDS CA, and a password held in the macOS Keychain.

Secure Credential Provisioning

The supported Mac provisioning flow uses:

1. a signed public `doccrate-db-connection.zip` and its `.sig`;
2. `rds-ca.pem` for the approved database trust chain;
3. a signed transient `.paq` file plus separately supplied setup code;
4. the macOS Keychain for the resulting database password.

The PAQ is not a permanent password file. It is removed from staging after a successful verification, Keychain install, and connection test.

Export

The portable core applies explicit breaks, keep-together ranges, page furniture, and semantic break rules. The Mac app reports loading, measuring, planning, and page generation in the toolbar.

Exports include:

- current page or whole-document PDF;
- current page or whole-document rendered Word;
- editable Word;
- SVG diagram packs;
- page layout and analysis through headless tooling.

See [Pagination on macOS](#) for the viewer setting and page-authoring workflow.

Agent and Automation Support

- AppleScript and Apple Events for the live Cocoa application.
- An embedded `doc-crate` helper for headless work.

- An embedded `doccrate-mcp` stdio server.
- TCL automation for repeatable project workflows.
- Agent referral prompts for current documents, selections, saved DocCrates, workfolders, standard content, diagrams, and pagination reviews.

Open-source Components and Use Restrictions

DocCrate began as open-source software. This branded fork is for the exclusive commercial use of LexisNexis and is not for redistribution or republication. Selkie and other third-party components retain their own licences. See [Licences and Permitted Use](#).

Back to [Home](#) | [User Guide](#)

macOS Implementation

The Mac version of DocCrate is a native AppKit application backed by an OS-portable Rust core. It does not run the Windows viewer under emulation and it does not use Direct2D or DirectWrite. Swift owns the Cocoa experience; Rust owns document, diagram, export, database, and automation semantics.

For the design rationale, begin with [Architecture](#).

Build Products

| Product | Technology | Purpose |
|---|--|---|
| DocCrate.app / CocoaDocCrate.app in development | Swift and AppKit linked to a Rust static library | Native Mac desktop application. |
| doc-crate | Rust executable embedded under Contents/Helpers | Headless validation, inspection, database, and export client. |
| doccrate-mcp | Rust executable embedded under Contents/Helpers | Stdio MCP server for agent tooling. |
| libdocrate_core.a | Rust static library | Portable core exposed to Swift through the C bridge. |

The release may use DocCrate.app as its distributed product name while the Xcode development target and bundle are named CocoaDocCrate.

Source Layout

| | |
|--------------------------|--|
| macos/ | |
| Cargo.toml | portable Mac-side Rust crate |
| src/ | core modules and embedded helpers |
| CocoaDocCrate/ | |
| CocoaDocCrate.xcodeproj/ | Xcode application project |
| CocoaDocCrate/ | Swift/AppKit source |
| RustBridge/ | Rust static library and public C header |
| release/ | packaging and installation material |
| selkie/ | DocCrate fork of Selkie |
| crates/ | shared local Rust crates |
| docs/ | documentation copied into the app bundle |

Rust Core Modules

| Module | Responsibility |
|---------------------|--|
| macos/src/docs.rs | Recursive project scan, document load, links, and file metadata. |
| macos/src/parser.rs | CommonMark block and inline model with source positions. |

| | |
|---------------------------------|---|
| macos/src/layout.rs | Platform-neutral draw commands, hit regions, headings, and document geometry. |
| macos/src/pagination.rs | Semantic page planning, explicit breaks, and keep-together policy. |
| macos/src/mermaid/ | Selkie integration, diagram-family lowering, manual overrides, and custom shapes. |
| macos/src/blockgraph.rs | DocCrate blockgraph parsing and layout. |
| macos/src/pdf_export.rs | PDF/SVG layout, font embedding, compression, and protection policy. |
| macos/src/word_export.rs | Rendered and editable Word generation. |
| macos/src/project_config.rs | Project variables and document/export configuration. |
| macos/src/standard_content.rs | Governed standard-content data and token expansion. |
| macos/src/doccrate_db.rs | Saved DocCrates, revisions, managed checkouts, and library search. |
| macos/src/db_credentials.rs | Credential retrieval and signed provisioning support. |
| macos/src/connection_failure.rs | DNS/TLS/database error classification and report text. |
| macos/src/cache_paths.rs | Platform-aware storage locations. |
| macos/src/find_replace.rs | Headless source find/replace operations. |
| macos/src/doccrate_tcl.rs | TCL automation verbs. |
| macos/src/bin/doccrate_mcp.rs | MCP tools and resources. |

The core crate has no dependency on AppKit. OS-specific storage and credential operations are adapters around portable services.

Swift and AppKit Modules

| File | Responsibility |
|--------------------------------------|---|
| AppDelegate.swift | Application lifecycle, menus, settings, appearance, and scripting registration. |
| DocumentWindowController.swift | Main workflow orchestration, toolbar, project state, exports, database actions, and errors. |
| DocumentViewController.swift | Document/source modes, native source editor, find/replace, source tracking, and selection. |
| DocumentCanvasView.swift | Native canvas for Rust-provided layout and diagram scenes. |
| ProjectNavigatorViewController.swift | Sidebar and recursive document navigation. |
| RustBridge.swift | Swift ownership wrappers and conversions for the C ABI. |
| MacPageOutputModel.swift | Page planning/export handoff and pagination progress. |
| MacApplicationStorage.swift | Application Support, cache, connection package, CA, and staged credential locations. |
| MacAppleEventAutomation.swift | AppleScript and Apple Event command handling. |

| | |
|--|---|
| InlineEditorWindowController.swift | Rich/source editing for a selected Markdown span. |
| BlockgraphEditorWindowController.swift | Native graphical blockgraph editor. |
| ShapeEditorWindowController.swift | Project custom-shape editor. |
| StandardItemEditors.swift | Standard text and diagram search/edit windows. |

Bridge Ownership

`macos/CocoaDocCrate/RustBridge` builds a static Rust library and publishes `include/docrate_core.h`. Returned strings and model handles have explicit free functions; Swift wraps those pointers so ownership is not inferred or shared accidentally.

Long-running work such as database access and PDF page generation runs away from the main thread. Results return to AppKit on the main queue before any view or window is updated.

Document and Editing Flow

1. The Cocoa shell asks Rust to open and scan a project.
2. Rust returns the recursive document tree.
3. Opening a document returns source plus a layout snapshot.
4. AppKit draws the snapshot and tracks the corresponding source line.
5. Source, inline, table, and diagram edits update Markdown source.
6. The normal save path writes the source, reparses it, refreshes layout, and starts page-output planning.

The toolbar's Find & Replace control switches to the native source editor and uses `NSTextFinder` across the current document source. Replacement notifications enter the same dirty/save pipeline as typed edits.

Diagram Pipeline

The vendored `selkie-rs` dependency is a DocCrate-maintained fork of the MIT-licensed Selkie project. The Mac build links it with default features disabled, using only its Rust library parser and layout support.

```
Mermaid fence
-> Selkie fork parser/layout
-> DocCrate Mermaid IR
-> annotations and custom shapes
-> platform-neutral diagram scene
-> AppKit canvas or export backend
```

There is no runtime Node.js, Puppeteer, Chromium, or Mermaid JavaScript bundle. DocCrate annotations beginning `%% @` preserve Mermaid-compatible source while adding stable identities, manual placement, routes, grouping, and styling.

Page Output

After a document opens or changes, Rust constructs page furniture and semantic pagination input. `MacPDFExporter` measures and plans pages, reporting loading, measurement, planning, and page-generation progress. The main toolbar displays that activity transiently. A request ID prevents an old completion from dismissing the indicator for a newer document.

The viewer preview and exports share page-policy inputs, including configured headers/footers, `[[newpage]]`, and keep-together ranges.

Database and Security

- The SQLite library normally lives in Application Support.
- Public MySQL routing metadata comes from a signed connection ZIP.
- The database password is stored in the macOS Keychain.
- A signed PAQ is verified, installed into Keychain, connection-tested, and then removed from staging after success.
- MySQL connections require TLS and use the approved `rds-ca.pem` trust file.
- Resolver parsing accepts normal macOS scoped resolver output without treating an interface scope as part of an IP address.
- Database errors are shown in a readable report dialog with Copy to Clipboard.

See [Connection Package and Credential Protection](#).

Storage

| Data | Default |
|-----------------------|---|
| Shared app data | ~/Library/Application Support/Cocoa DocCrate/ |
| SQLite library | ~/Library/Application Support/Cocoa DocCrate/security_text.sqlite |
| Managed checkouts | ~/Library/Application Support/Cocoa DocCrate/Managed Checkouts/ |
| Generated/cache files | ~/Library/Caches/Cocoa DocCrate/ |
| Secret password | macOS Keychain, not a file |

Automation

AppleScript operates the live Mac application. The embedded CLI and MCP server operate the portable/headless surface. The Debug-only local interface socket is a test harness and is not the installed automation contract.

Legacy Windows Code

The repository retains a Windows application and some Windows-oriented design history. Files such as root `src/render.rs`, Win32 message handling, Direct2D/DirectWrite drawing, `.exe` launchers, Windows Credential Manager, and named viewer pipes describe that shell. They are not used to draw or automate the macOS desktop application.

Licensing

DocCrate began as open-source software. The current LexisNexis-branded fork has additional restrictions and is not for redistribution or republication. See [Licences and Permitted Use](#).

Back to [Home](#) | [Build This App](#)

Licences and Permitted Use

DocCrate origin

DocCrate was originally created as open-source software by Alban Read and was released under the MIT License. That original open-source history and the rights granted by its applicable source release are acknowledged here.

This branded fork

This LexisNexis-branded fork contains additional code, integrations, configuration, documentation, themes, assets, and other intellectual property developed or supplied for LexisNexis and RELX.

This branded fork is provided for the exclusive commercial use of LexisNexis. It is not for public redistribution, republication, sublicensing, or external publication. Do not copy the application, its branded assets, its documentation bundle, connection material, or internal configuration into a public repository, public package, article, demonstration, or third-party distribution.

Authorised internal use, modification, testing, backup, and deployment must follow the applicable LexisNexis policies and commercial arrangements. This notice does not replace those agreements and does not alter the licences of third-party components.

Third-party and open-source components

Third-party components retain their own copyright notices and licence terms. In particular:

| Component | Use in DocCrate | Licence |
|---|---|---|
| Selkie, forked from Benjamin Tucker's project | Native Mermaid-compatible parsing and diagram layout | MIT; copyright Benjamin Tucker |
| Mermaid syntax compatibility | Authoring syntax targeted by the Selkie fork | Refer to the Mermaid project's own notices; Mermaid is not embedded as a JavaScript runtime in the Mac app. |
| pulldown-cmark | CommonMark parsing | MIT |
| docx-rs local fork | Word document generation | MIT; original copyright bokuweb |
| Rust crates resolved by Cargo | Parsing, graphics, databases, TLS, export, and support services | Their respective MIT, Apache-2.0, BSD, ISC, Unicode, Zlib, and other declared terms |

The in-product [Third-Party Licences and Acknowledgements](#) page lists the complete locked macOS dependency inventory, bundled font notice, prominent copyrights, licence expressions, and canonical licence texts. The source repository also retains `selkie/LICENSE`, `crates/docx-rs/LICENSE`, and the dependency-generated `licenses.md` record.

Documentation and generated material

The restriction on republication applies to this bundled documentation and its LexisNexis-specific examples, themes, logos, screenshots, diagrams, and export assets. Documents created by authorised users may have separate ownership and publication rules; users must follow the policy applicable to that content.

Questions

Before distributing any part of this branded fork outside LexisNexis, obtain written approval through the appropriate LexisNexis legal, security, and software-governance channels.

Back to [Home](#) | [Architecture](#)

Custom Shapes

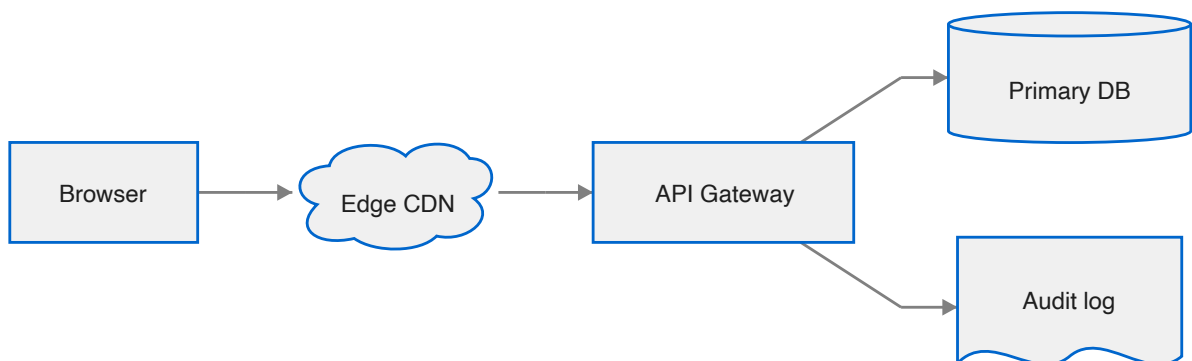
DocCrate ships with the classic mermaid shape catalog (rectangle, diamond, cylinder, ...) baked into the renderer. **Extension shapes** are declared in plain-text `.shape` files and referenced from mermaid using the v10 metadata syntax:

```
A@{ shape: cloud, label: "AWS Region" }
```

Bundled extension shapes include `cloud`, `document`, and the architecture glyph set: `api`, `browser`, `cache`, `database`, `db`, `disk`, `file`, `function`, `gateway`, `internet`, `lock`, `mobile`, `queue`, `server`, `service`, `shield`, `user`, and `worker`. Drop your own `.shape` files into `docs/.shapes/` (sibling to your markdown) to import or cache them. For database-opened DocCrates, saved shapes live in the local database and the `.shapes` files are the managed checkout mirror. A shape whose name matches a built-in will **override** the bundled version.

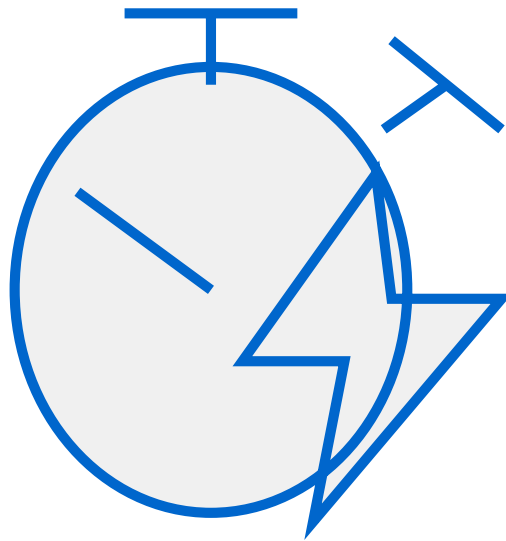
The bundled presentation icon set also includes `ease-of-use` (cog with check), `efficiency` (stopwatch with bolt), and `confidence` (protected documents). Use them as normal Mermaid metadata shapes, with the label in the diagram node rather than baked into the icon.

Bundled shapes in a real diagram

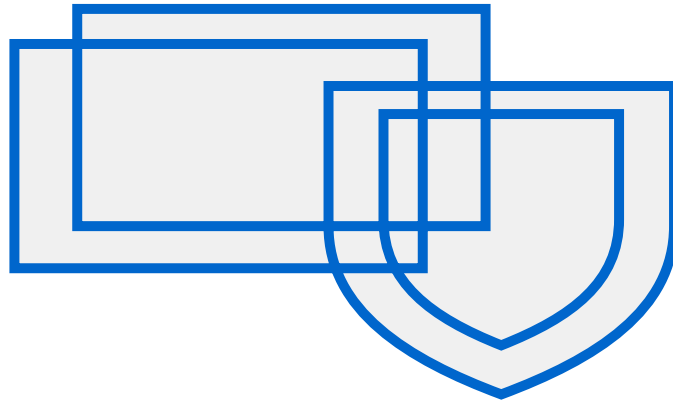




Ease of Use



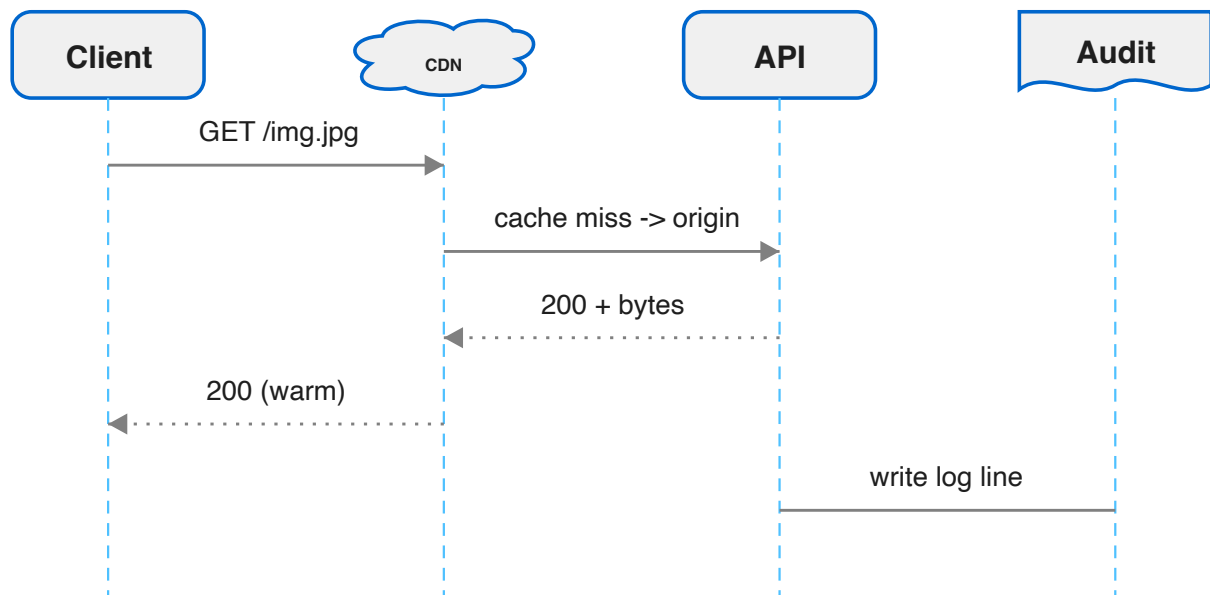
Efficiency



Confidence

Custom shapes in sequence diagrams

The same registry is wired through to sequence diagrams. Put the metadata on the `participant` declaration:



The lifeline drops from the bottom-centre of the actor box exactly as for a regular participant.

Architecture icons use the same registry

The icon token in `architecture-beta` service declarations resolves through the shape registry too:

```

service api(api)[Docs API]
service db(database)[Metadata DB]
service queue(queue)[Job Queue]
  
```

That means a doc set can replace any architecture glyph globally by saving a matching database shape such as `server`.

Editing shapes visually

Use `Edit > Shape Editor...` to open the project shape editor. The right panel lets you select an existing built-in or project shape, create a new shape, edit the shape metadata, and add/remove `.shape` commands. The canvas shows the normalized `0..1` coordinate space at high zoom; drag handles to move path points, Bezier controls, circle radius handles, the label anchor, and text-area corners. `Ctrl+S` saves to the local database for the current DocCrate and refreshes `docs/.shapes/<name>.shape` in the managed checkout; saving a built-in shape creates a project override with the same name.

The `.shape` DSL

Coordinates are normalised — `0..1` along both axes. The renderer scales to the actual node width and height. A shape file declares one shape:


```

shape my-thing
  aspect      1.6          # hard ratio (w / h)
  label       0.5 0.5      # label centre
  text-area   0.1 0.2   0.9 0.8  # optional tighter label box
  stroke-mult 1.0          # optional stroke-width multiplier

  moveto      0.00 0.00
  lineto      1.00 0.00
  curveto     1.05 0.50  0.50 1.10  0.00 0.50
  close
end

```

| Directive | Form | Meaning |
|-----------|---------------------|------------------------------------|
| moveto | x y | start a sub-path |
| lineto | x y | straight segment |
| curveto | c1x c1y c2x c2y x y | cubic Bézier |
| quadto | cx cy x y | quadratic Bézier |
| circle | cx cy r | additive ellipse sub-path |
| polygon | x1,y1 x2,y2 ... | closed polyline (≥ 3 points) |
| close | | close the current sub-path |

Lines starting with `#` are comments; blank lines are ignored.

Aspect ratio is hard

When a `.shape` declares `aspect 1.6`, DocCrate resizes the node *before* selkie lays out the graph so edge routing sees the final dimensions. The ratio is enforced by **growing** the smaller dimension — your text label is never squished below the size that the estimator computed for it.

Mermaid State Fixture - this is a test page for the document generation.

This file is a pagination test fixture. It intentionally contains ordinary paragraphs before a state-diagram section so the PDF paginator has to keep the section heading with the diagram that follows it.

The text is synthetic and should stay free of product, customer, credential, or environment details.

Warm-up Section

DocCrate renders Markdown into a stream of draw commands before pagination. This fixture gives the paginator enough content to exercise a page boundary near a diagram.

The paragraphs below are deliberately plain. Their job is to occupy vertical space in a predictable way while staying easy to read if someone opens the fixture in the viewer.

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Integer non nibh imperdiet, blandit massa at, facilisis lorem. Donec sed lectus vitae neque aliquet dictum. Praesent efficitur tortor at sapien pulvinar, vitae iaculis quam dignissim.

Suspendisse potenti. Aenean laoreet ipsum et justo consequat, id malesuada magna tristique. Curabitur gravida sem ac justo dignissim, eget faucibus nulla congue. Maecenas ultrices enim non felis bibendum, vitae volutpat justo consectetur.

Vestibulum ante ipsum primis in faucibus orci luctus et ultrices posuere cubilia curae; Aliquam erat volutpat. Nunc congue, nibh in commodo vestibulum, lacus urna placerat erat, at eleifend velit est in nisl.

Phasellus non tincidunt nunc. Integer gravida tellus eget justo porttitor consectetur. Vivamus eget luctus mauris. Sed non ipsum laoreet, facilisis justo at, viverra mi.

Ut lacinia fermentum lacus, vel volutpat justo facilisis vitae. Aliquam feugiat sem in mauris porta, nec malesuada purus rutrum. Cras in neque id magna imperdiet cursus.

Donec vel ex in tortor facilisis blandit. Proin semper lectus non sem fringilla, at posuere sem accumsan. Duis luctus sem ac lectus dictum, sed vulputate lectus blandit.

Mauris mattis arcu non nunc sagittis, a lacinia lorem iaculis. Etiam molestie metus non risus tincidunt, vel fermentum lorem pellentesque. Nunc id turpis in lacus rhoncus tincidunt.

Curabitur consequat ligula quis velit ultricies, vel facilisis enim luctus. Suspendisse vitae nibh ac augue efficitur porttitor. Etiam consequat augue at sapien bibendum, nec sagittis quam interdum.

Integer condimentum enim ut lectus gravida, non faucibus massa dignissim. Pellentesque id odio sit amet lacus consequat convallis. Integer ac arcu a mauris malesuada bibendum.

Sed id felis interdum, convallis neque ac, fermentum est. Duis egestas quam at turpis pretium, id porttitor lacus tempor. Donec vel neque vel enim sagittis vulputate.

Nam in tellus id arcu congue volutpat. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Vestibulum ante ipsum primis in faucibus.

Aliquam ac magna aliquet, efficitur erat sit amet, tristique justo. Vestibulum sit amet orci faucibus, pharetra massa vel, feugiat mauris. Curabitur gravida ligula sed dolor sollicitudin, vitae blandit eros

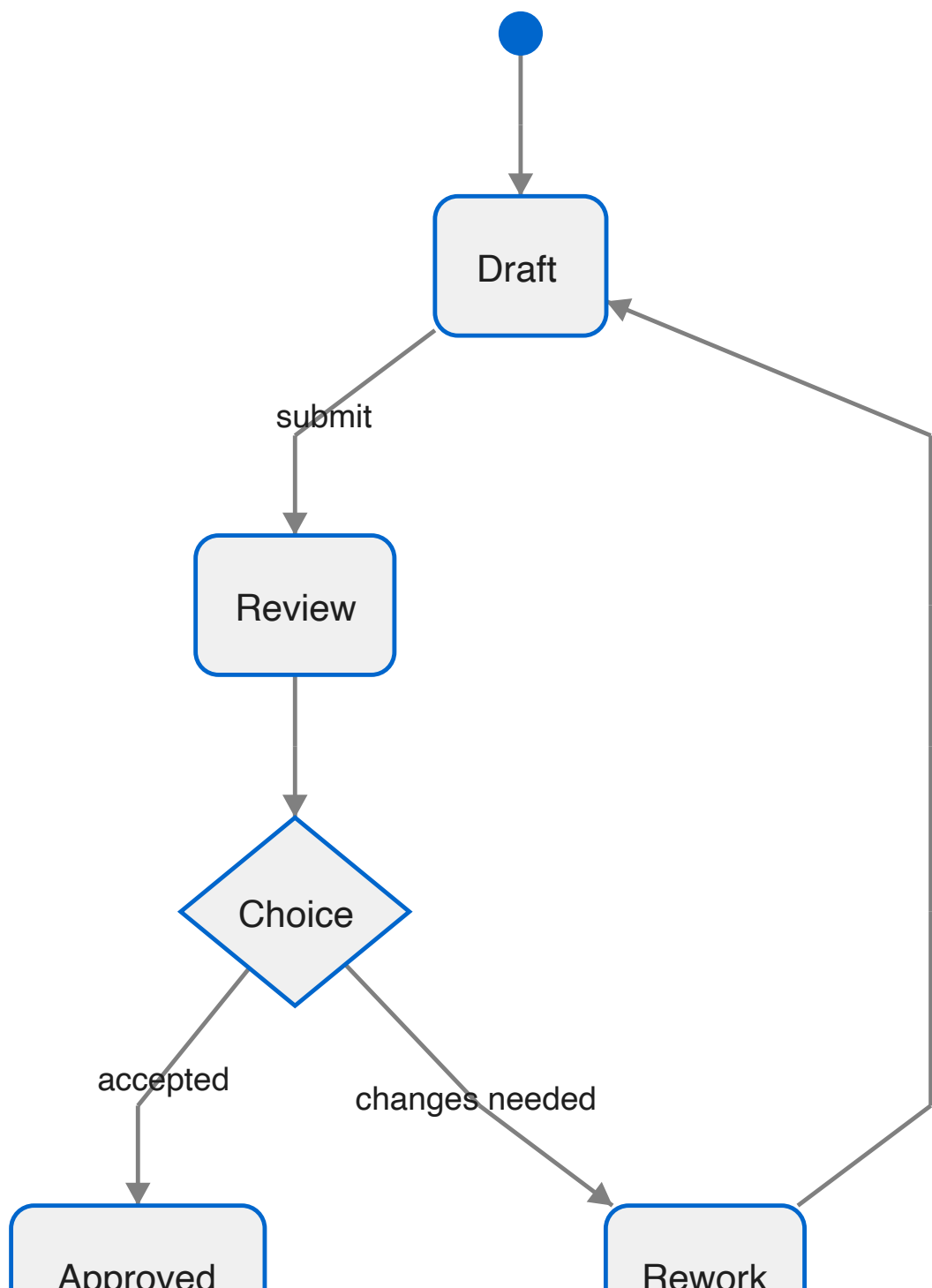
luctus.

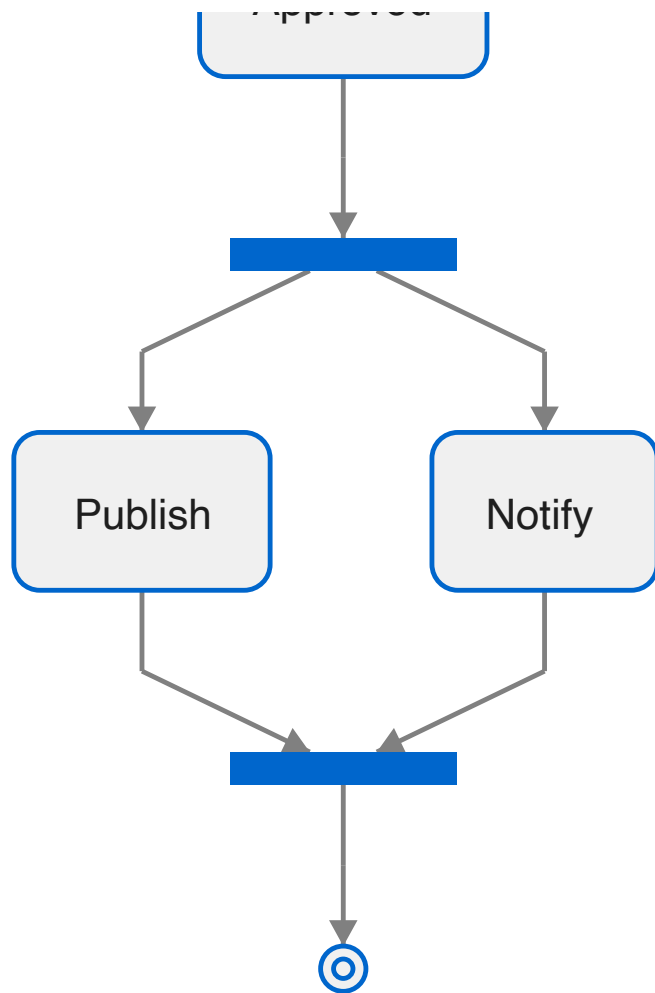
Praesent at nibh volutpat, bibendum massa vitae, suscipit metus. Morbi luctus nisl sed libero ultricies, eget placerat justo mollis. Nulla facilisi.

Donec consequat augue quis urna cursus, nec pulvinar elit sollicitudin. Integer quis enim sit amet nibh gravida eleifend. Sed id libero sed arcu cursus imperdiet.

Vivamus bibendum sem ut quam aliquam, nec posuere justo consequat. Vestibulum consequat lorem non enim tempus, a finibus risus porta. Praesent sit amet metus non massa placerat feugiat.

Choice + fork / join





macOS Menus and Keyboard Shortcuts

This is the quick reference for the native Mac application. The Command symbol is written as `Command`, Option as `Option`, and Control as `Control`.

Application Menu

| Command | Shortcut | Action |
|---------------|------------------|--|
| Settings | Command-, | Open appearance and application settings. |
| Hide DocCrate | Command-H | Hide the application. |
| Hide Others | Option-Command-H | Hide other applications. |
| Quit DocCrate | Command-Q | Quit, prompting for unsaved source where required. |

File

| Command | Shortcut | Action |
|-------------------------|-----------|--|
| Open Project | Command-O | Search and open a saved DocCrate. |
| Open Folder | | Choose a local Markdown project folder. |
| Open Recent Folder | | Reopen the most recently authorised external folder. |
| Close | Command-W | Close the window. |
| Save | Command-S | Save the current document source. |
| Save Project to Library | | Store the current project as a saved DocCrate. |

Edit

| Command | Shortcut | Action |
|----------------------|-----------------------------|--|
| Undo / Redo | Command-Z / Shift-Command-Z | Undo or redo in the active editor. |
| Cut / Copy / Paste | Command-X/C/V | Standard native clipboard operations. |
| Select All | Command-A | Select all in the active editor or text field. |
| Find | Command-F | Switch to Source and find text in the current document source. |
| Find and Replace | Option-Command-F | Switch to Source and open the replacement interface. |
| Spell Check Document | Command-; | Run the selected language profile over the document. |
| | | |

| | | |
|----------------------------|--|---|
| Check Spelling as You Type | | Toggle native continuous spelling in the source editor. |
| Shape Editor | | Open the project custom-shape editor. |

The toolbar Find & Replace button performs the same action as `Option-Command-F`. It replaces the older heading-only toolbar search.

View

| Command | Shortcut | Action |
|-----------------------|-------------------|---|
| Document | Option-Command-1 | Show the rendered document. |
| Source | Option-Command-2 | Show the whole-document Markdown source. |
| Appearance | | Follow System, force Light, or force Dark appearance. |
| Project Configuration | Option-Command-, | Open .doccrate/project-config.md. |
| Show/Hide Sidebar | Control-Command-S | Toggle the project navigator. |
| Enter Full Screen | Control-Command-F | Toggle native macOS full screen. |

Toolbar back and forward buttons navigate document history. Clicking a sidebar item opens that Markdown document; local links navigate within the project and external links open in the default browser.

Export

| Command | Shortcut | Action |
|------------------------------------|-----------------|--|
| Current Page to PDF | Command-E | Export the current Markdown document as PDF. |
| Document to PDF | | Export the whole project as PDF. |
| Current Page to Word | Shift-Command-E | Export the current Markdown document as rendered Word. |
| Document to Word | | Export the whole project as rendered Word. |
| Editable Word Document | | Export native editable Word structures. |
| Generate Diagram Pack | | Export supported diagrams as SVG files. |
| Refer Pagination & Layout to Agent | | Copy the governed pagination-review workflow. |

Successful GUI exports reveal the output in Finder. Generated/cache defaults are under `~/Library/Caches/Cocoa DocCrate/` unless an explicit destination is selected.

Library

| | |
|--|--|
| | |
|--|--|

| Command | Action |
|--------------------------------|---|
| Find and Edit Standard Text | Search, inspect, and edit governed standard text. |
| Find and Edit Standard Diagram | Search, inspect, and edit governed diagrams. |
| Sync Library with MySQL | Synchronise the local SQLite library using the approved secure connection. |
| Install Signed Credential PAQ | Select a .paq or .txt PAQ, verify its signature and setup code, store the password in Keychain, test the connection, and remove the staged PAQ after success. |

Connection ZIP, signature, CA, and PAQ placement are described in [Install on macOS](#).

Agent

| Command | Action |
|-------------------------------------|---|
| Refer Current Document | Copy a prompt scoped to the open document. |
| Refer Current Selection | Copy a prompt scoped to selected rendered or source text. |
| Refer Current DocCrate | Copy a managed-checkout edit-and-save workflow. |
| Create New from Current DocCrate | Copy a workflow that creates a separate saved DocCrate. |
| Review Pagination with Agent | Copy a visual and semantic pagination-review workflow. |
| Find Standard Text/Diagram to Refer | Open the appropriate library before referring an item. |
| Refer Workfolder | Choose source material and copy a new-document drafting workflow. |

Source Editor

The source editor follows normal macOS text conventions. Important DocCrate behavior:

- saving reparses and rerenders the document;
- switching back to Document preserves the shared source position where possible;
- find and replace searches the entire current document source;
- syntax highlighting and line numbers refresh after edits;
- `Esc` may return from focused editing where the active editor supports it.

Graphical Editors

Editors provide native toolbar, inspector, selection, clipboard, save, and cancel behavior. Available actions vary by diagram family. Common actions are move or resize items, create links, edit labels and styles, change group membership, delete a selected item, reset the view, and save changes back to the fenced source.

Window

| Command | Shortcut | Action |
|--------------------|-----------|------------------------------------|
| Minimize | Command-M | Minimize the active window. |
| Zoom | | Use the native window zoom action. |
| Bring All to Front | | Bring application windows forward. |

Use Restrictions

The LexisNexis-branded fork and this documentation are not for republication or external redistribution. See [Licences and Permitted Use](#).

Back to [Home](#).

TCL Scripting on macOS

DocCrate embeds TCL for repeatable local workflows. The Mac helper is `doc-crate` without an `.exe` suffix and is embedded in the application bundle.

```
doc-crate --tcl -e 'puts [doccrate/status]' --result
doc-crate --tcl ./scripts/publish.tcl --json
cat ./scripts/publish.tcl | doc-crate --tcl --json
```

Use `--result` to print the TCL result and `--json` for an automation-friendly envelope. The exact installed verb surface is authoritative through `doc-crate --help tcl`.

Example

```
set doc docs/index.md
puts [doccrate/status]
doccrate/validate $doc
doccrate/export-pdf $doc output/index.pdf
```

Common Verbs

| Verb | Purpose |
|------------------------------------|---|
| doccrate/status | Return the current automation/project status. |
| doccrate/project-config | Inspect or initialise project configuration. |
| doccrate/write-doc | Write Markdown content to an authorised project file. |
| doccrate/append-doc | Append Markdown content. |
| doccrate/find-replace | Perform reviewed source replacement. |
| doccrate/validate | Validate Markdown and project references. |
| doccrate/lint | Run lint checks. |
| doccrate/list-diagrams | Enumerate diagram identities and source positions. |
| doccrate/export-pdf | Export PDF. |
| doccrate/export-docx | Export rendered Word. |
| doccrate/export-editable-docx | Export editable Word. |
| doccrate/export-svg | Export SVG output. |
| doccrate/standard-content-search | Search governed text. |
| doccrate/standard-content-show | Read a governed record. |
| doccrate/standard-content-expand | Expand a standard-content reference. |
| doccrate/standard-content-validate | Validate tokens in a document. |

| | |
|-------------|---|
| docrate/exe | Return the current doc-crate helper path. |
|-------------|---|

Paths and Quoting

TCL uses forward-slash paths naturally on macOS. Braces avoid unwanted substitution:

```
set root {/Users/example/Documents/my docs}  
docrate/validate [file join $root index.md]
```

Do not put passwords, PAQ content, tokens, or connection strings in scripts. Database commands obtain their password from the macOS Keychain.

Safety

- Scope scripts to the project or managed checkout explicitly provided.
- Validate before export or database save.
- Do not write generated files into managed source checkouts.
- Use an explicit output directory for release workflows.
- Treat changes to governed standard content as managed database writes.

The live Cocoa application is automated with AppleScript rather than the old Windows viewer-control channel. See [AppleScript](#).

Back to [Home](#) | [macOS Command Line](#)

DocCrate for macOS User Guide

DocCrate is a native Mac application for reading, editing, governing, and exporting Markdown document projects. The desktop interface uses Swift and AppKit; an OS-portable Rust core supplies parsing, diagrams, pagination, database services, and exports.

Install and Start

Follow [Installing DocCrate on macOS](#) for the app, standard-content library, connection package, CA certificate, and signed PAQ.

When DocCrate opens without another project, it displays this bundled help. Choose **File > Open Folder...** to author a local folder, or **File > Open Project...** to search saved database DocCrates.

The Main Window

The project navigator is on the left and the current document is on the right. Use the toolbar to:

- browse saved DocCrates;
- show or hide the sidebar;
- move backward and forward through document history;
- switch between Document and Source;
- open Find & Replace for the current document source.

While page output is being measured and planned, a temporary toolbar spinner shows the current pagination phase and page count. It disappears when the current pagination request completes.

The status bar reports application, library, and database state. Database checks do not repeatedly ask for Keychain access in the background.

Projects and Documents

A local project is a folder containing Markdown files. DocCrate scans `.md` files recursively, sorts `index.md` and `readme.md` first in each folder, and shows subfolders as nested sidebar sections. Project control files live under `.doccrate/`.

A saved DocCrate has a database identity and a managed filesystem checkout. That checkout lets the viewer, editors, CLI, MCP server, and authorised agents work with ordinary files while revisions remain governed by the local library and optional MySQL sync.

| File command | Purpose |
|-------------------------|---|
| Open Project | Search and open a saved DocCrate. |
| Open Folder | Authorise and open a local Markdown folder. |
| Open Recent Folder | Reopen the most recently authorised folder. |
| Save | Save the current source document. |
| Save Project to Library | Create or update a saved DocCrate from the project. |

Managed checkouts normally live under:

```
~/Library/Application Support/Cocoa DocCrate/Managed Checkouts/
```

Document and Source Modes

Document mode shows the natively rendered Markdown. Source mode shows the complete Markdown file in a native text editor with line numbers, syntax highlighting, selection, undo, clipboard, and spelling support.

| Action | Shortcut |
|------------------------------|------------------|
| Show Document | Option-Command-1 |
| Show Source | Option-Command-2 |
| Save | Command-S |
| Find source text | Command-F |
| Find and replace source text | Option-Command-F |

The toolbar Find & Replace control switches to Source and opens the native replacement bar. It searches the current document's complete source, including text inside its sections. Replacements use the same dirty/save flow as typing.

Switching modes preserves the corresponding source position where possible. Saving reparses the document, refreshes its rendered snapshot, and recalculates pagination.

Editing from the Rendered Document

Rendered objects retain their Markdown source positions. Depending on the object, a context or edit action can open:

- focused rich/source prose editing;
- a table grid editor;
- diagram source at the relevant fence;
- a graphical diagram editor;
- a referenced standard-text record;
- the whole document source at the relevant line.

Edits ultimately write Markdown or an explicitly selected governed library record. This avoids maintaining a second proprietary document format.

Markdown Support

DocCrate supports headings, paragraphs, strong and emphasized text, inline code, fenced code, links, images, blockquotes, ordered and bullet lists, tables, thematic rules, and strikethrough. `[[newpage]]` adds an explicit page break.

Fenced code is highlighted with `syntect`. Project-specific Sublime syntaxes, TextMate themes, and highlight selection can live under `.doccrate/`.

Diagrams

Mermaid-compatible fences are parsed through DocCrate's maintained fork of the MIT-licensed Selkie Rust engine. The Mac application does not need Node.js, Chromium, or Mermaid JavaScript at runtime.

The pipeline is:

```
Mermaid source -> Selkie fork -> DocCrate scene -> AppKit canvas or export
```

DocCrate supports stable diagram IDs, native editing for supported families, custom `.shape` vectors, and `%% @` annotations for manual layout and styling. It also supports the DocCrate-specific `blockgraph` format.

Use **Export > Generate Diagram Pack** to write supported diagrams as SVG.

Project Configuration

Choose **View > Project Configuration...** or press `Option-Command-`, to open:

```
.doccrate/project-config.md
```

If the selected project does not have this file, DocCrate creates it and opens the new Variables table in the same editor. The generated configuration starts with the cover, trailer, page header, footer, and spelling checks switched off; automatic paginated PDF preview remains on. Set `paginate_for_pdf` to `off` for online-only projects that do not need page previews. Explicit PDF export remains available regardless of this setting.

Project configuration contains reusable variables, metadata, and export furniture such as document headers and page footers. Variables and governed standard-content references are expanded by the Rust core before rendering and export where the selected workflow permits expansion.

Standard Content

The Library menu opens native search/edit windows for governed standard text and diagrams. A standard-text token looks like:

```
{{standard:42:"privacy notice"}}
```

The number is the governed identity; the quoted text is an author-readable label. Tokens remain visible in source and are expanded for viewing, validation, and export. Unknown references remain visible and are reported.

The local SQLite library normally lives at:

```
~/Library/Application Support/Cocoa DocCrate/security_text.sqlite
```

Use **Library > Sync Library with MySQL** only when the approved connection components and credential have been installed.

Signed Database Provisioning

Normal Mac provisioning uses:

```
~/Library/Application Support/Cocoa DocCrate/doccrate-db-connection.zip
~/Library/Application Support/Cocoa DocCrate/doccrate-db-connection.zip.sig
~/Library/Application Support/Cocoa DocCrate/rds-ca.pem
```

Choose **Library > Install Signed Credential PAQ...** Select the supplied `.paq` or `.txt` PAQ and enter the separately delivered setup code. DocCrate verifies the embedded release signature, writes the password to the macOS Keychain, tests a mandatory-TLS MySQL connection, and removes the staged PAQ after success.

The connection ZIP contains public routing metadata, not the password. Do not put passwords in project Markdown, `.doccrate` files, shell history, or sync reports.

If DNS, TLS, login, database access, or a test query fails, DocCrate displays a readable report dialog with **Copy to Clipboard**. The report keeps the PAQ in place when installation has not completed so the problem can be corrected and retried. It does not include the database password.

See [Connection Package and Credential Protection](#).

Export

The Export menu supports:

| Command | Output |
|------------------------|--|
| Current Page to PDF | PDF for the current Markdown document. |
| Document to PDF | One paginated PDF for the project. |
| Current Page to Word | Rendered Word for the current document. |
| Document to Word | Rendered Word for the project. |
| Editable Word Document | Native editable Word structures where supported. |
| Generate Diagram Pack | SVG files for document diagrams. |

Successful GUI exports are revealed in Finder. Generated/cache defaults are under:

```
~/Library/Caches/Cocoa DocCrate/Generated/
```

PDF editing protection asks conforming readers to prevent modification; it is not a substitute for storage encryption, transport security, information rights management, or customer-data controls.

See [Pagination on macOS](#) for automatic preview, `paginate_for_pdf`, page breaks, keep-together ranges, and review tools.

Spelling and Language

Choose a spelling profile from the Language menu, then use **Edit > Spell Check Document...** (`Command-;`). The source editor can also use native continuous spelling underlines driven by DocCrate's project-aware Rust checker. Profiles distinguish supported English dialect and suffix policies; automatic correction is disabled so brochure terminology is never silently rewritten. See [Spelling on macOS](#).

Governed standard text follows the same project language. Existing `{{standard:n:"label"}}` references try the brochure's regional variant and fall back to approved US wording with a warning. The standard-text finder marks fallback results and writes an explicit locale for non-US selections, for example `### Vault Expiration`

Users at a customer organization may create Vaults for long-running projects.

- For that reason, Vaults do not auto-expire.
- A customer can delete a Vault at any time.

An organization's Vaults will be deleted within 60 days of a customer closing their account. `.

Appearance and Accessibility

Choose System, Light, or Dark from **View > Appearance**. AppKit provides native keyboard focus, menu behavior, sheets, clipboard, window management, and accessibility labels for primary controls.

Agent Referrals

The Agent menu creates clipboard prompts; it does not silently launch or grant an agent access. Referrals can target the current document, selected text, current saved DocCrate, a new document based on a saved DocCrate, pagination, standard content, a diagram, or an authorised workfolder.

Saved-DocCrate referrals use the embedded Mac CLI names:

```
doc-crate --db-open <root-key> --json
doc-crate --db-save <checkout-path> --json
doc-crate --db-save-as <checkout-path> "New title" --json
```

Agents must work only in the returned managed checkout and use the required save command when the referral specifies one.

Automation

| Surface | Use |
|------------------------------|---|
| AppleScript and Apple Events | Control the live native Mac application. |
| doc-crate | Headless validation, inspection, database, and export work. |
| doccrate-mcp | Agent-facing MCP tools and discovery resources. |

| | |
|-----|--|
| TCL | Repeatable local multi-step workflows. |
|-----|--|

The helpers are embedded under `/Applications/DocCrate.app/Contents/Helpers/`. See [macOS Command Line](#), [MCP](#), and [AppleScript](#).

Configuration Paths

| Path | Purpose |
|--|--|
| <code>.doccrate/project-config.md</code> | Variables, metadata, and export furniture. |
| <code>.doccrate/highlight.toml</code> | Syntax-highlight theme selection. |
| <code>.doccrate/syntaxes/</code> | Additional Sublime syntax definitions. |
| <code>.doccrate/themes/</code> | Additional highlighting themes. |
| <code>.shapes/</code> | Custom diagram shape definitions. |

Licences and Permitted Use

DocCrate originated as open-source software. This LexisNexis-branded fork is for the exclusive commercial use of LexisNexis and is not for public redistribution or republication. Third-party components, including Selkie, retain their own licence terms. See [Licences and Permitted Use](#).

Back to [Home](#) | [macOS Shortcuts](#)

Third-Party Licences and Acknowledgements

This page provides the third-party acknowledgement for the macOS product. It lists the components resolved by the locked `aarch64-apple-darwin` Cargo build, plus bundled fonts and important vendored components. Each component remains subject to its own licence; no LexisNexis product-use restriction replaces or narrows those third-party terms.

The inventory was generated from `macos/Cargo.toml` and `Cargo.lock` using Cargo metadata. Versions can change when dependencies are deliberately updated, so this page must be regenerated and reviewed as part of every release that changes the lockfile.

Prominent acknowledgements

Selkie

DocCrate includes a modified fork of [Selkie](#), a native Rust implementation of a Mermaid-compatible parser and renderer.

```
MIT License
Copyright (c) 2026 Benjamin Tucker
```

The Selkie fork is used as a library for Mermaid-compatible parsing and diagram layout. It retains the MIT terms reproduced below.

docx-rs

DocCrate includes a local fork of `docx-rs` for Word document generation.

```
MIT License
Copyright (c) 2020 bokuweb
```

Lato fonts

DocCrate bundles the Lato Regular, Bold, Italic, and Bold Italic font files. Lato was designed by Łukasz Dziedzić and is distributed under the [SIL Open Font License 1.1](#). The font may be used, studied, modified, and redistributed subject to that licence, including its Reserved Font Name and standalone-sale conditions.

Sequoia OpenPGP and buffered-reader

`sequoia-openpgp 2.4.1` and `buffered-reader 1.4.1` are distributed under the GNU Library General Public License, version 2 or any later version. Their source licence files are authoritative. The complete [LGPL-2.0 text](#) and corresponding source availability obligations must accompany distributions as required by that licence.

webpki

webpki 0.22.4 uses ISC-style terms, copyright 2015 Brian Smith. Its third-party/chromium files retain the notices identified by webpki's bundled licence file.

Complete locked Rust dependency inventory

Packages are grouped by the exact licence expression reported in Cargo metadata. OR means the package offers a licence choice; AND means all named terms apply.

LGPL-2.0-or-later

buffered-reader 1.4.1, sequoia-openpgp 2.4.1

MIT

bincode 1.3.3, bitvec 1.1.1, bytes 1.12.1, color_quant 1.1.0, core_maths 0.1.1, data-encoding 2.11.1, docx-rs 0.4.20, ecb 0.1.2, ecb 0.2.0, fancy-regex 0.16.2, float-cmp 0.9.0, fontdb 0.23.0, funty 2.0.0, generic-array 0.14.7, imagesize 0.14.0, libm 0.2.16, libsqlite3-sys 0.30.1, lopdf 0.43.0, lru 0.16.4, memsec 0.7.0, mio 1.2.2, new_debug_unreachable 1.0.6, nom 8.0.0, pem 3.0.6, phf_shared 0.13.1, pico-args 0.5.0, plist 1.9.0, precomputed-hash 0.1.1, pulldown-cmark 0.12.2, pulldown-cmark-escape 0.11.0, quick-xml 0.36.2, quick-xml 0.39.4, radium 0.7.0, rgb 0.8.53, rusqlite 0.32.1, rustybuzz 0.20.1, saturating 0.1.0, selkie-rs 0.3.0, sha1collisiondetection 0.3.4, simd-adler32 0.3.9, slab 0.4.12, spin 0.9.9, strict-num 0.1.1, synstructure 0.13.2, syntect 5.3.0, tap 1.0.1, tokio 1.53.1, tokio-macros 2.7.2, tracing 0.1.44, tracing-core 0.1.36, twox-hash 2.1.3, wyz 0.5.1, xmlwriter 0.1.0, zip 0.6.6, zmij 1.0.21

MIT OR Apache-2.0

aead 0.5.2, aes 0.8.4, aes 0.9.2, ahash 0.8.12, allocator-api2 0.2.21, anyhow 1.0.104, argon2 0.5.3, arrayvec 0.7.7, async-trait 0.1.91, base64 0.22.1, bitflags 2.13.0, blake2 0.10.6, block-buffer 0.10.4, block-buffer 0.12.1, block-padding 0.3.3, block-padding 0.4.2, blowfish 0.9.1, btoi 0.5.0, camellia 0.1.0, cast5 0.11.1, cbc 0.1.2, cbc 0.2.1, cc 1.2.65, cfb-mode 0.8.2, cfg-if 1.0.4, chacha20 0.10.1, chrono 0.4.42, cipher 0.4.4, cipher 0.5.2, cmac 0.7.2, core-foundation 0.10.1, core-foundation 0.9.4, core-foundation-sys 0.8.7, cpubits 0.1.1, cpufeatures 0.2.17, cpufeatures 0.3.0, crc32fast 1.5.0, critical-section 1.2.0, crossbeam-channel 0.5.16, crossbeam-epoch 0.9.20, crossbeam-queue 0.3.13, crossbeam-utils 0.8.21, crypto-common 0.1.7, crypto-common 0.2.2, ctr 0.9.2, data-url 0.3.2, dbl 0.3.2, deranged 0.5.8, des 0.8.1, digest 0.10.7, digest 0.11.3, displaydoc 0.2.7, dyn-clone 1.0.20, either 1.17.0, ena 0.14.4, euclid 0.22.14, fdeflate 0.3.7, find-msvc-tools 0.1.9, fixedbitset 0.5.7, flate2 1.1.9, form_urlencoded 1.2.2, futures-channel 0.3.32, futures-core 0.3.32, futures-io 0.3.33, futures-macro 0.3.32, futures-task 0.3.32, futures-util 0.3.32, getopts 0.2.24, getrandom 0.2.17, getrandom 0.3.4,

getrandom 0.4.3, gif 0.14.2, hashbrown 0.14.5, hashbrown 0.15.5, hashbrown 0.17.1, hashlink 0.9.1, hickory-net 0.26.1, hickory-proto 0.26.1, hickory-resolver 0.26.1, hkdf 0.12.4, hmac 0.12.1, hybrid-array 0.3.1, hybrid-array 0.4.13, iana-time-zone 0.1.65, idea 0.5.1, idna 1.1.0, image 0.25.9, image-webp 0.2.4, inout 0.1.4, inout 0.2.2, ipnet 2.12.1, itertools 0.14.0, itoa 1.0.18, lazy_static 1.5.0, libc 0.2.186, lock_api 0.4.14, log 0.4.33, md-5 0.10.6, md-5 0.11.0, mmap2 0.9.11, mysql_common 0.37.3, num-bigint 0.4.8, num-conv 0.2.2, num-integer 0.1.46, num-iter 0.1.46, num-traits 0.2.19, once_cell 1.21.4, opaque-debug 0.3.1, parking_lot 0.12.5, parking_lot_core 0.9.12, password-hash 0.5.0, percent-encoding 2.3.2, pest 2.8.6, pest_derive 2.8.6, pest_generator 2.8.6, pest_meta 2.8.6, petgraph 0.8.3, pkg-config 0.3.33, png 0.18.1, polycool 0.4.0, powerfmt 0.2.0, ppv-lite86 0.2.21, prefix-trie 0.8.4, proc-macro2 1.0.106, quote 1.0.46, rand 0.10.2, rand 0.8.7, rand 0.9.5, rand_chacha 0.3.1, rand_core 0.10.1, rand_core 0.6.4, rand_core 0.9.5, regex 1.12.4, regex-automata 0.4.14, regex-syntax 0.8.11, resolv-conf 0.7.6, ripemd 0.1.3, roxmltree 0.20.0, roxmltree 0.21.1, rsa 0.9.10, rustc_version 0.4.1, rustls-pki-types 1.15.1, scopeguard 1.2.0, security-framework 3.7.0, security-framework-sys 2.17.0, semver 1.0.28, serde 1.0.228, serde_core 1.0.228, serde_derive 1.0.228, serde_json 1.0.149, sha1 0.10.7, sha2 0.10.9, sha2 0.11.0, sha3 0.10.9, sha3 0.11.0, shake 0.1.0, shlex 2.0.1, smallvec 1.15.2, socket2 0.6.5, sponge-cursor 0.1.0, stable_deref_trait 1.2.1, string_cache 0.9.0, syn 2.0.118, syn 3.0.3, system-configuration 0.7.0, system-configuration-sys 0.6.0, term 1.2.1, thiserror 1.0.69, thiserror 2.0.18, thiserror-impl 1.0.69, thiserror-impl 2.0.18, time 0.3.52, time-core 0.1.9, time-macros 0.2.31, ttf-parser 0.21.1, ttf-parser 0.25.1, twofish 0.7.1, typenum 1.20.1, ucd-trie 0.1.7, unicas 2.9.0, unicode-bidi 0.3.18, unicode-normalization 0.1.25, unicode-script 0.5.8, unicode-width 0.2.2, unicode-xid 0.2.6, universal-hash 0.5.1, url 2.5.8, weezl 0.1.12, weezl 0.2.1

Apache-2.0 OR MIT

aes-gcm 0.10.3, autocfg 1.5.1, base16ct 0.2.0, base64ct 1.8.3, bit-set 0.8.0, bit-set 0.9.1, bit-vec 0.8.0, bit-vec 0.9.1, cmov 0.5.4, const-oid 0.10.2, const-oid 0.9.6, crypto-bigint 0.5.5, ctutils 0.4.2, der 0.7.10, derive_utils 0.15.1, dsa 0.6.3, eax 0.5.0, ecdsa 0.16.9, ed25519 2.2.3, elliptic-curve 0.13.8, equivalent 1.0.2, ghash 0.5.1, idna_adapter 1.2.2, indexmap 2.14.0, io-enum 1.2.1, keccak 0.1.6, keccak 0.2.0, kem 0.3.0, kurbo 0.13.1, lalrpop 0.23.1, lalrpop-util 0.23.1, ml-dsa 0.1.1, ml-kem 0.3.2, module-lattice 0.2.3, ocb3 0.1.0, p256 0.13.2, p384 0.13.1, p521 0.13.3, pem-rfc7468 0.7.0, pin-project-lite 0.2.17, pkcs1 0.7.5, pkcs8 0.10.2, polyval 0.6.2, portable-atomic 1.14.0, primeorder 0.13.6, resvg 0.47.0, rfc6979 0.4.0, sec1 0.7.3, serdect 0.2.0, signature 2.2.0, signature 3.0.0, simplecss 0.2.2, slh-dsa 0.0.3, spki 0.7.3, svgtypes 0.16.1, usvg 0.47.0, utf8_iter 1.0.4, uuid 1.23.4, zeroize 1.9.0, zeroize_derive 1.5.0

MIT/Apache-2.0 and spelling variants

ascii-canvas 4.0.0, bufstream 0.1.4, fallible-iterator 0.3.0, fallible-streaming-iterator 0.1.9, ff 0.13.1, fnv 1.0.7, group 0.13.0, linked-hash-map 0.5.6, mysql 28.0.0, num-bigint-dig 0.8.6, quick-error 2.0.1, rangemap 1.7.1, siphasher 1.0.3,

stringprep 0.1.5, tagptr 0.2.0, unicode-bidi-mirroring 0.4.0, unicode-ccc 0.4.0, unicode-properties 0.1.4, unicode-vo 0.1.0, vcpkg 0.2.15, version_check 0.9.5, yaml-rust 0.4.5

BSD licences

- BSD-2-Clause: arrayref 0.3.9.
- BSD-2-Clause OR Apache-2.0 OR MIT: zerocopy 0.7.35, zerocopy 0.8.52, zerocopy-derive 0.7.35.
- BSD-3-Clause: curve25519-dalek 4.1.3, cx448 0.1.1, ed25519-dalek 2.2.0, subtle 2.6.1, tiny-skia 0.12.0, tiny-skia-path 0.12.0, x25519-dalek 2.0.1.
- BSD-3-Clause OR Apache-2.0: moxcms 0.7.11, pxfm 0.1.29.
- (Apache-2.0 OR MIT) AND BSD-3-Clause: encoding_rs 0.8.35.

ISC licences

- ISC: rustls-webpki 0.103.13, untrusted 0.9.0.
- Apache-2.0 AND ISC: ring 0.17.14.
- Apache-2.0 OR ISC OR MIT: rustls 0.23.43, rustls-pemfile 2.2.0.
- ISC-style licence file: webpki 0.22.4.

Unicode-3.0

icu_collections 2.2.0, icu_locale_core 2.2.0, icu_normalizer 2.2.0, icu_normalizer_data 2.2.0, icu_properties 2.2.0, icu_properties_data 2.2.0, icu_provider 2.2.0, litemap 0.8.2, potential_utf 0.1.5, tinystr 0.8.3, writeable 0.6.3, yoke 0.8.3, yoke-derive 0.8.2, zerofrom 0.1.8, zerofrom-derive 0.1.7, zerotrie 0.2.4, zerovec 0.11.6, zerovec-derive 0.11.3.

unicode-ident 1.0.24 is (MIT OR Apache-2.0) AND Unicode-3.0.

Zlib-family expressions

- Zlib: foldhash 0.1.5, slotmap 1.1.1.
- Zlib OR Apache-2.0 OR MIT: bytemuck 1.25.0, tinyvec 1.11.0.
- MIT OR Apache-2.0 OR Zlib: fontdue 0.9.3, tinyvec_macros 0.1.1, zune-core 0.5.1, zune-jpeg 0.5.15.
- MIT OR Zlib OR Apache-2.0: miniz_oxide 0.8.9.

Unlicense and MIT choices

- Unlicense OR MIT: aho-corasick 1.1.4, byteorder 1.5.0, byteorder-lite 0.1.0, memchr 2.8.2.
- Unlicense/MIT: csv 1.4.0, csv-core 0.1.13, same-file 1.0.6, walkdir 2.5.0.

Other permissive expressions

- `0BSD OR MIT OR Apache-2.0: adler2 2.0.1.`
- `Apache-2.0 OR BSL-1.0: ryu 1.0.23.`
- `BSL-1.0: xxhash-rust 0.8.18.`
- `CDLA-Permissive-2.0: webpki-roots 1.0.9.`
- `(Apache-2.0 OR MIT) AND Unicode-3.0: noted above for unicode-ident.`
- `(Apache-2.0 OR MIT) AND Apache-2.0: moka 0.12.15.`

Licence texts

The canonical full texts apply. The following links identify the corresponding standard terms; source distributions must also retain the licence files and notices supplied with each component.

| Identifier | Canonical text |
|---------------------|---|
| 0BSD | Zero-Clause BSD |
| Apache-2.0 | Apache License 2.0 |
| BSD-2-Clause | BSD 2-Clause |
| BSD-3-Clause | BSD 3-Clause |
| BSL-1.0 | Boost Software License 1.0 |
| CDLA-Permissive-2.0 | Community Data License Agreement – Permissive 2.0 |
| ISC | ISC License |
| LGPL-2.0-or-later | GNU Library GPL 2.0 |
| MIT | MIT License |
| OFL-1.1 | SIL Open Font License 1.1 |
| Unicode-3.0 | Unicode License v3 |
| Unlicense | The Unlicense |
| Zlib | zlib License |

MIT text used by Selkie and docx-rs

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

Release obligation

This page is the in-product acknowledgement, not a substitute for preserving the original licence files. Release packaging must retain all legally required copyright notices, licence texts, source offers, attribution files, and component-specific notices. Legal or open-source governance should review any dependency, font, or asset change before release.

Back to [Licences and Permitted Use](#) | [Home](#)

Configuration

DocCrate supports per-docs-root runtime configuration under `.doccrate/`. Viewer themes live in `.doccrate/themes.json`, syntax highlighting choices live in `.doccrate/highlight.toml`, and custom syntaxes/themes/shapes can be dropped into subfolders there.

Viewer Themes

Create `<docs>/.doccrate/themes.json` to add named viewer themes:

```
{
  "active": "midnight",
  "themes": {
    "midnight": {
      "base": "dark",
      "colors": {
        "bg": "#101418",
        "text": "#d7e2ea",
        "link": "#6bb6ff",
        "diagram_card_bg": "#111a24",
        "mermaid_group_title": "#ffd166"
      }
    }
  }
}
```

Launch with `--theme <name>` to override `active`, or switch an open viewer with `--set-theme <name>` / the MCP `doccrate_set_theme` tool. Built-ins are `dark`, `light`, and `lexis-red`. See [Theming](#) for every supported key and the optional `raw` colour-remap map.

Compile-Time Constants

Fonts, spacing, and default tokens still live in `src/theme.rs` as `pub const` values. Change those and recompile when you need to alter layout metrics or add new token names.

Theme Constants

Colours

| Constant | Default | Purpose |
|-------------|---------|-------------------------|
| BG | #1E1E1E | Main background |
| SIDEBAR_BG | #252526 | Sidebar background |
| SIDEBAR_SEL | #37373D | Selected item highlight |
| TEXT | #D4D4D4 | Body text |
| TEXT_BRIGHT | #FFFFFF | Selected / heading text |

| | | |
|---------|---------|-----------------------|
| LINK | #4FC1FF | Hyperlink colour |
| CODE_BG | #0D0D0D | Code block background |
| CODE_FG | #CE9178 | Code block text |

Typography

| Constant | Default | Purpose |
|------------|-----------------|------------------------|
| BODY_FONT | "Lato" | Prose font family |
| CODE_FONT | "Cascadia Code" | Code block font |
| BODY_SIZE | 15.0 | Body text size (DIPs) |
| LINE_EXTRA | 1.4 | Line-height multiplier |

Layout

| Constant | Default | Purpose |
|-----------|---------|------------------------------|
| SIDEBAR_W | 220.0 | Default sidebar width (DIPs) |
| H_PAD | 32.0 | Horizontal content padding |
| V_PAD | 20.0 | Vertical content padding |
| PARA_GAP | 10.0 | Space between blocks |

Changing the Docs Folder

Pass the path as the first argument:

```
doc-crate ~/Documents/my-project/docs
```

Or set a wrapper script / shortcut that always passes your preferred path.

See also: [Theming](#) | [Project Config](#) | [Home](#)

Installing DocCrate on macOS

Install the application

1. Open the supplied `DocCrate.dmg`.
2. Drag `DocCrate.app` to `/Applications`.
3. Open it once from `/Applications`.

The app is Developer ID signed and notarized. Its command-line clients are embedded in the application bundle:

```
/Applications/DocCrate.app/Contents/Helpers/doc-crate
/Applications/DocCrate.app/Contents/Helpers/doccrate-mcp
```

Use those full paths in shell scripts and MCP client configuration, or add `/Applications/DocCrate.app/Contents/Helpers` to your shell `PATH`.

Install the document library

The SQLite standard-content library is deliberately delivered separately from the notarized application. Obtain the approved library from the organisation's content-delivery channel and copy it to:

```
~/Library/Application Support/Cocoa DocCrate/security_text.sqlite
```

Create the `Cocoa DocCrate` directory if it does not yet exist. The Cocoa app, CLI, and MCP server share this default. In the app, use **DocCrate > Settings > Library Locations** to select a different administrator-managed SQLite file.

Install the signed connection package

Copy both delivered files, retaining their names, to:

```
~/Library/Application Support/Cocoa DocCrate/
```

```
doccrate-db-connection.zip
doccrate-db-connection.zip.sig
```

The `.sig` is mandatory. The ZIP holds public MySQL routing metadata only, not the database password. If supplied, also place `rds-ca.pem` in this directory.

An authorised administrator then chooses **Library > Install Signed Credential Package...**, selects the supplied PAQ, and enters the separately delivered PIN. The password is written only to the native Keychain and is never stored in the SQLite database, connection ZIP, or project Markdown.

Manual Keychain recovery

Use this only when the signed PAQ flow cannot be used. The password is not in this application or documentation and must be obtained separately by an authorised administrator.

Create or update a generic-password item in the user's login Keychain with:

```
Service: com.doccrate.database.credentials
Account: docrate db connection
```

Put the separately supplied database password in the item's password field. DocCrate also recognises the historic account name `doccrate db connection`, but new entries should use `docrate db connection`.

MCP configuration

Use the embedded server and CLI together:

```
{
  "mcpServers": {
    "doccrate": {
      "command": "/Applications/DocCrate.app/Contents/Helpers/doccrate-mcp",
      "env": {
        "DOCRATE_EXE": "/Applications/DocCrate.app/Contents/Helpers/doc-crate"
      }
    }
  }
}
```

For native GUI automation, see [AppleScript and Apple Events](#). See [macOS Command Line](#) and [MCP Server](#) for the supported headless automation surface.

Pagination on macOS

DocCrate for Mac can turn the current Markdown project into a page-aware PDF preview while you work. This is useful for brochures and other documents whose page breaks, headers, footers, tables, and diagrams must be reviewed before export. Online-only documentation can disable the automatic preview and keep the normal continuous document view.

Continuous View and Paginated Preview

The document first opens in the native continuous viewer. When automatic PDF pagination is enabled, DocCrate then builds a paginated preview in the background and replaces the continuous rendering when the pages are ready.

The toolbar shows the active phase:

- loading page-output source;
- measuring content;
- planning the page count;
- generating each page.

The indicator disappears when the current request finishes. Opening another document or saving new source cancels the relevance of an older request, so an out-of-date result cannot replace the current document.

Project Setting

Open **View > Project Configuration...** and edit `paginate_for_pdf` in the Variables table:

```
name	value	description
paginate_for_pdf	on	Build the automatic paginated PDF preview
page_size	a4	PDF page size: a4 or letter
page_orientation	portrait	PDF page orientation: portrait or landscape
```

The setting behaves as follows:

| Value | Viewer behaviour |
|-------------------------|---|
| on | Build the paginated PDF preview after opening or changing a document. |
| off | Keep the continuous document view and skip background PDF pagination. |
| Missing or unrecognised | Treat as on, which is the project default. |

New project configurations default to `on`. The documentation bundled with DocCrate sets it to `off` because online and in-app help does not need page layout. Saving a changed value refreshes the current document immediately.

This setting controls only automatic viewer pagination. It does not remove the PDF export commands and does not prevent an explicit PDF from being generated.

`page_size` accepts `a4` or `letter`; `us` and `us-letter` are also understood as Letter aliases. If the setting is missing, a Mac whose current region is the United States uses US Letter and other regions use A4. `page_orientation` accepts `portrait` or `landscape` and defaults to `portrait`. Project values drive the automatic preview and are the initial choices in the export dialog.

What Controls Page Layout

The Mac page-output path uses the portable Rust document model together with native macOS measurement and PDF generation. Pagination considers:

- A4 or US Letter page size and portrait or landscape orientation;
- configured document cover, trailer, running header, and footer;
- headings and their first meaningful content block;
- tables, diagrams, images, lists, code, and paragraphs;
- manual page breaks;
- authored keep-together ranges;
- printable height after page furniture and margins are reserved.

PDF export and automatic preview use the same page-policy inputs. The rendered Mac window is not captured as an image to make the PDF.

Manual Page Breaks

Place `[[newpage]]` on its own line in Markdown source:

```
End of the current section.
```

```
[[newpage]]
```

```
# Appendix
```

The older spelling `[[pagebreak]]` is also accepted. A manual break is a hard authoring boundary and is not removed by whitespace compaction.

Keep Content Together

To keep a heading, paragraph, table, diagram, or other range on one page when it fits:

1. Right-click the first rendered block or source line.
2. Choose **Keep Together....**
3. Extend the selection to the last block in the range.

DocCrate records the range in Markdown using invisible comments:

```
<!-- doccrate:keep-together:start -->
```

```
## Authentication
```

```
This explanation and its diagram should remain together.
```

```
<!-- doccrate:keep-together:end -->
```

The marked range starts on the next page when it fits there but not in the remaining space. A range taller than a printable page must still split at a legal boundary; DocCrate never clips content merely to honour the annotation. Right-click inside a marked range and choose **Remove Keep Together** to remove it.

Exporting a PDF

Use the Export menu:

| Command | Result |
|---------------------|---|
| Current Page to PDF | Export the current Markdown document. |
| Document to PDF | Export the complete project as one paginated PDF. |

Explicit export works whether `paginate_for_pdf` is `on` or `off`. Successful exports are revealed in Finder. If no explicit destination is chosen, generated output is stored under:

```
~/Library/Caches/Cocoa DocCrate/Generated/
```

Reviewing Pagination

For a visual review of the whole document, use **Agent > Review Pagination with Agent...** The referral describes the governed analysis, contact-sheet, page inspection, source-edit, validation, and export workflow.

The embedded command-line helper can perform the same inspections:

```
/Applications/DocCrate.app/Contents/Helpers/doc-crate \  
--analyze-pdf ~/Documents/my-docs --json  
  
/Applications/DocCrate.app/Contents/Helpers/doc-crate \  
--export-pdf-layout ~/Documents/my-docs ~/Desktop/my-docs-layout.png
```

The layout PNG is a numbered contact sheet. Use it to find sparse pages, orphaned headings, unexpected blank pages, split tables, or diagrams that need closer inspection before final export.

Troubleshooting

| Symptom | Check |
|---|---|
| No pagination indicator appears | Confirm <code>paginate_for_pdf</code> is on. |
| Online docs pause while loading | Set <code>paginate_for_pdf</code> to off. |
| A heading is left at the bottom of a page | Add a focused <code>keep-together</code> range around the heading and its lead content. |

| | |
|---|--|
| A new chapter must start on a page | Add [[newpage]] immediately before it. |
| Preview does not reflect a config edit | Save the Project Configuration editor; the current page-output request then refreshes. |
| PDF export is still available when preview is off | This is intentional: the option suppresses automatic preview only. |

See also: [Project Configuration](#) | [macOS Command Line](#) | [Agent Referrals](#)

Agent Discovery

DocCrate is MCP-first. Agents should discover DocCrate through the MCP stdio server, then use the CLI or TCL surfaces only for fallback or batch scripting.

What Exists

DocCrate exposes three machine-readable MCP resources:

| Resource | Use |
|---|---|
| <code>doccrate://discovery/agent-card</code> | A2A-style capability card for tools that understand agent cards. |
| <code>doccrate://discovery/capabilities</code> | Compact inventory of authoring, diagram, export, standard-content, project-config, and TCL surfaces. |
| <code>doccrate://discovery/project-roots</code> | Runtime paths for the MCP server, doc-crate, current working directory, bundled docs, <code>doccrateCache</code> , and <code>doccrateGenerated</code> . |
| <code>doccrate://help/cache</code> | Exact resolved database DocCrate checkout cache path and checkout metadata guidance. |

The release folder also includes static discovery files:

| File | Use |
|--|--|
| <code>agent-card.json</code> | Human and agent-readable capability card. |
| <code>doccrate-agent.json</code> | Same card under a DocCrate-specific name. |
| <code>.well-known/agent-card.json</code> | Well-known style location for tools that look for agent cards. |
| <code>AGENTS.md</code> | Codex-style release-folder rules and database checkout workflow. |
| <code>CLAUDE.md</code> | Claude-oriented release-folder quick guide. |
| <code>MCP-QUICKSTART.md</code> | Copy/paste MCP configuration plus headless CLI fallback loops. |

These files are discovery metadata. They do not make DocCrate an A2A task server. The actual integration surface is still MCP.

Recommended Agent Bootstrap

1. Read `doccrate://discovery/agent-card`.
2. Read `doccrate://discovery/capabilities`.
3. Read `doccrate://help/agent`.
4. Run `docrate_doctor`.
5. Use `docrate_new_root` for a new documentation project, or `docrate_import_projects` for an existing project/collection that belongs in Saved DocCrates.
6. For database-backed brochures, read `doccrate://help/cache` or `doccrate://discovery/project-roots` and use the reported `doccrateCache` folder for

checkout source and `docrateGenerated` for generated PDFs, Word documents, SVGs, layout PNGs, snapshots, and generated reports. In the macOS release these are normally `~/Library/Application Support/Cocoa DocCrate/Managed Checkouts` and `~/Library/Caches/Cocoa DocCrate/Generated`.

Agents should not author customer documents inside DocCrate's bundled `docs` folder. That folder is product help and examples.

For jobs started from the desktop app, see [Agent Referrals](#). Those commands copy a ready-to-paste prompt with the right workfolder or saved DocCrate root key and the required validation/check-in workflow.

Why This Shape

The agent-discovery ecosystem is still fragmented. A2A-style agent cards are useful for describing capabilities, but DocCrate's useful interface is already the MCP server:

- MCP lists tools and resources directly.
- MCP resources can expose help, capability JSON, and runtime paths.
- DocCrate tools already support authoring, standard-content lookup, diagram validation, viewer capture, and document export.

For now, the right implementation is a clear MCP discovery surface plus static agent-card files in the release folder.

Security Notes

- DocCrate works offline and does not require network access for normal use.
- Agents should operate only on user-selected document roots or explicit paths.
- Standard-content references should remain as standard tokens, with optional quoted labels for readability, unless the user asks for editable copied prose.
- PDF export is compressed and protected from modification in conforming PDF readers by default.

Agent Referrals

Agent referrals are application-generated prompts that copy to the clipboard. They are designed for handing a specific DocCrate job to an AI agent without making the agent guess the project, database root, source folder, or required validation loop.

Referrals do not run an agent by themselves. After DocCrate copies the referral, paste it into the agent task, then let the agent follow the included workflow.

Referral Types

| Referral | Where to start it | Use when | What the copied prompt contains |
|--------------------------------|--|--|---|
| Brochure workfolder | Project > Refer AI to Workfolder... | A source-material folder should become a new DocCrate brochure project. | The selected folder path, the generated AGENT-BROCHURE-INSTRUCTIONS.md path, and a requirement to work only inside that folder. |
| Current saved DocCrate | Agent > Refer Current DocCrate... | A clean database-backed DocCrate needs content, diagram, wording, or source review. | The saved DocCrate title, database root key, doc-crate --db-open <root-key> --json, the user's custom instructions, and the required -db-save <checkout_path> check-in command. |
| Create new from saved DocCrate | Ask Agent > Create New From This... | A new brochure should be based on the current saved brochure without changing the source brochure. | The source title and root key, the checkout command, config-update guidance, the user's requested changes, PDF validation/export expectations, and the required --db-save-as <checkout_path> "<new title>" command. |
| Pagination and layout | Export > Refer Pagination & Layout to Agent... | A saved DocCrate needs PDF pagination, page-flow, diagram-fit, overflow, or final-export review. | The database root key plus the full analyze, layout contact-sheet, page-preview, Markdown edit, validate, export, and database check-in workflow. |
| Current document or section | Ask Agent > Refer Current Document to Agent..., a sidebar document right-click, or a rendered-document right-click | An agent should review a local document, one text section, table, standard-text reference, or diagram. | The document path, clicked source line when applicable, target type, nearby source context, current diagram source when applicable, and the user's custom instructions. |
| Standard text item | Library > Manage Standard Text or Library > Find Standard Text, then Refer to Agent | A governed standard-content item needs review or revision outside the app. | The selected {{standard:n}} reference when available, the item Markdown, and the user's custom instructions. |
| Standard diagram item | Open a standard diagram in the visual editor, then Refer to | A governed standard diagram needs wording, | The standard-content reference, the current diagram-bearing Markdown, and the user's custom instructions. |

| | | | |
|--|-------|---------------------------|--|
| | Agent | layout, or source review. | |
|--|-------|---------------------------|--|

Ask `Agent` gathers every referral route in one top-level menu. The standard-text and standard-diagram entries open their existing managed editors, where select the item and use `Refer to Agent`. In a rendered document, right-click the target section, table, standard-text reference, or diagram and choose the contextual referral action. The copied prompt names the exact document and source line so the agent can make the intended update.

How To Use A Referral

1. Save any local changes first. Database-backed referrals require a clean, current managed checkout so the agent starts from the latest saved revision.
2. Choose the relevant referral command or `Refer to Agent` button.
3. Enter concise instructions when DocCrate prompts for them. For pagination referrals, DocCrate supplies the workflow automatically.
4. Paste the clipboard text into the agent task.
5. For database-backed DocCrates, the agent must open the root key with `doc-crate --db-open <root-key> --json`, use only the returned `checkout_path`, validate and export from that checkout, then save the result with `doc-crate --db-save <checkout_path> --json`. For create-new referrals, the agent saves a separate brochure with `doc-crate --db-save-as <checkout_path> "<new title>" --json` instead of `--db-save`.

Database Checkout Rules

Saved-DocCrate referrals intentionally pass a database root key rather than the current local folder. This keeps the database authoritative and gives the agent a fresh managed checkout.

Agents should:

- Read `.doccrate\checkout.json` after opening a saved DocCrate.
- Edit only files inside the returned `checkout_path`.
- Keep generated PDF, Word, SVG, layout PNG, snapshot PNG, and reports in the generated cache folder or another explicit output path, not mixed into source Markdown unless requested.
- Run DocCrate validation before saving back.
- Report the new revision after `--db-save` succeeds.

If DocCrate refuses to create a saved-DocCrate referral, save the current DocCrate first or reopen it from `Saved DocCrates...` so the checkout is no longer dirty or stale.

Create-new referrals deliberately keep the source database root unchanged. The agent should update `.doccrate\project-config.md` for the new brochure's product, region, business unit, header, footer, cover, trailer, spelling, and related settings, then use `--db-save-as` so the completed work appears as a separate saved DocCrate.

Brochure Workfolder Rules

Refer AI to Workfolder... is for new brochure drafting from a folder of source materials.

DocCrate scaffolds the folder as a project if needed and writes AGENT-BROCHURE-INSTRUCTIONS.md.

The referred agent should:

- Treat the selected folder as both the source-material folder and the DocCrate project root.
- Review all relevant source PDFs, documents, images, diagrams, and existing Markdown before drafting.
- Keep the brochure in index.md.
- Put product, region, cover, trailer, header, footer, and spelling settings in .doccrate\project-config.md.
- Use approved standard-content references as tokens after searching and inspecting the relevant standard items.
- Validate Markdown and standard-content references, analyze PDF layout, inspect the layout visually, fix actionable issues, and export the final PDF.

See also: [Agent Discovery](#), [MCP Server](#), and [Database-Backed DocCrates](#).

AppleScript and Apple Events

The installed macOS app exposes its native GUI and document services through Apple Events. Scripts should address the stable bundle identifier rather than a display name:

```
tell application id "com.doccrate.cocoa"
    activate
    set state to interface status
end tell
```

Open **Script Editor > File > Open Dictionary... > DocCrate** to browse the full DocCrate GUI Suite. AppleScript controls the actual Cocoa application; CLI and MCP remain better choices for unattended batch work and agent tool calls.

First-use permission

macOS asks for consent when another application sends Apple Events to DocCrate. Approve that prompt for Script Editor, Shortcuts, or the authorised automation host. The script never receives a database password: the credential installer only opens the native PAQ/PIN user interface.

Common examples

Open a project, search headings, and move both rendered and source views to a heading:

```
tell application id "com.doccrate.cocoa"
    activate
    open doccrate project POSIX file "/Users/me/Documents/my-docs"
    set matchingHeadings to search doccrate project "security"
    set destination to open doccrate heading "Security" in document "index.md"
end tell
```

Read and safely replace the current document source:

```
tell application id "com.doccrate.cocoa"
    set originalMarkdown to current document source
    set replacementMarkdown to originalMarkdown & return & "## Review notes" & return
    replace current document source replacementMarkdown saving true
    docrate commit document
end tell
```

Set appearance and capture the front DocCrate window without granting Screen Recording permission:

```
tell application id "com.doccrate.cocoa"
    set application appearance to "dark"
    capture front doccrate window POSIX file "/tmp/docrate.png"
end tell
```

Available automation groups

| Area | Representative commands |
|---------------------------|---|
| Window and appearance | interface status, doccrate window names, show application settings, set application appearance, capture front doccrate window |
| Documents | open doccrate project, list doccrate documents, search doccrate project, open doccrate heading, open doccrate document, show source, show rendered document |
| Source editing | current document source, replace current document source, append current document source, doccrate commit document |
| SQLite library | show saved doccrates, search saved doccrates, open saved doccrate, doccrate store in library, managed doccrate status |
| Standard content | show standard text library, show standard diagram library, search standard text, search standard diagrams, doccrate update standard text |
| Editors | show shape library, edit doccrate table, replace doccrate table, edit doccrate diagram, replace doccrate diagram |
| Export and pagination | export current page, export doccrate document, doccrate export status, keep together, remove keep together, pdf preview state |
| Referrals and credentials | refer current document, refer pagination, refer workfolder, show credential installer |

Most methods return a Boolean, a path, JSON text, or a typed `interface status` record. Library writes use the same validation and managed-checkout checks as equivalent GUI actions.

Discovery and forward compatibility

Use `supported doccrate commands` to enumerate stable command names, or send a supported name through `do gui command` with an optional list of text arguments. Prefer the named commands above where one exists because they provide typed AppleScript parameters and clearer errors.

```
tell application id "com.doccrate.cocoa"
  set commandNames to supported doccrate commands
  set resultJSON to do gui command "document-outline"
end tell
```

Never pass a password, PAQ token, or PIN as an Apple Event argument. Use `show credential installer` to present the app's authorised native flow.

Blockgraph Editor

Platform note: The current macOS editor is the native AppKit `BlockgraphEditorWindowController` backed by the portable Rust blockgraph model. The Windows audit below is retained only as design provenance and does not describe Mac menus, shortcuts, or storage paths.

Purpose

The macOS Blockgraph editor is a native graphical editor for DocCrate's `blockgraph` fence. It edits the existing Blockgraph language and its existing `// @node` layout annotations; it does not introduce a second Mac-only diagram format.

The editor is intended to feel like a small, focused drawing application:

- the canvas remains visible and stable while selection changes;
- the inspector follows the current selection;
- cards come from a draggable stencil palette when nothing is selected;
- text is edited directly on the card or in the inspector;
- cards move and resize by direct manipulation;
- connectors are selectable and have their own inspector;
- every persistent change participates in undo and redo;
- Save writes valid Blockgraph source back to the document.

Existing language and renderer

The source of truth is `src/blockgraph.rs` (mirrored by `macos/src/blockgraph.rs`) and the native layout in `src/layout.rs`. The Windows editor in `src/render.rs` is the behavioural reference for editing and source round-tripping.

Model

A Blockgraph has three ordered collections:

| Element | Source | Meaning |
|-----------|---|--|
| Step card | <code>step <colour> <title> <detail> [options]</code> | A card in the main process chain |
| Edge | <code>edge <label> <colour> [options]</code> | Styling for the gap between step <i>n</i> and step <i>n</i> +1 |
| Note card | <code>note <colour> <title> <detail> [options]</code> | An unconnected supporting card or band |

Step and note identity is positional: `step1`, `step2`, ... and `note1`, `note2`, Edges are also positional and do not contain arbitrary source and target IDs. The editor must therefore not pretend that the language supports arbitrary graph links. It may create or edit the connector for an adjacent pair of steps. Notes are not link targets.

Card options already supported by the parser are:

- `card-shape=standard|chevron`;
- `icon`, `icon-label`, `icon-pos`, `icon-fill`, `icon-size`, and the legacy icon stroke aliases;
- `badge`, `badge-pos`, `badge-fill`, `badge-stroke`, and `badge-color`.

Edge options already supported are line width, solid/dashed/dotted line style, and start/end markers (`none`, `triangle`, `circle`, or `cross`).

Layout is stored with the existing annotation:

```
// @node step2 col=2 row=1 span=2 row-span=1
```

The native renderer uses 260 by 120 point cells, 72 point horizontal gaps, and 84 point vertical gaps. Movement snaps to `col` and `row`; resizing changes `span` and `row-span`. The editor must use the same grid and coordinate system as the rendered preview.

Windows editor audit

The Windows implementation provides a dedicated Blockgraph edit model rather than treating Blockgraph as a generic Mermaid graph. It currently supports:

- loading step and note cards with their title, detail, accent, shape, icon, badge, and grid placement;
- a Blockgraph grid overlay;
- moving cards by cell and resizing their column/row span;
- direct title editing and inspector editing of title, detail, and badge text;
- fill-colour swatches, badge text colours, and a scrollable icon picker;
- adding blocks and notes;
- selecting, styling, and resetting explicit connectors to their automatic adjacent-step form;
- deleting original and newly added cards while removing affected edge source;
- source-preserving rewriting of card and edge lines;
- emitting `// @node ... col=... row=... span=... row-span=...` only when needed;
- preserving unrecognised source and options where the edited field does not own them.

The Mac implementation should match those capabilities and improve the normal desktop interactions: native controls, drag-and-drop stencils, smooth live movement, selection handles, keyboard commands, undo/redo, and an inspector that scrolls independently of the canvas.

Production-data review

The local SQLite copy contains 31 documents with real `blockgraph` fences. They include brochure diagrams with:

- four-step process and data-protection chains;
- long titles and multi-sentence detail text;
- named and hexadecimal accent colours;
- labelled connectors with custom colours;
- badges with fill, border, and text colour options;
- multiple Blockgraphs in one document;

- CRLF source as well as LF source.

Acceptance testing must use at least one of these production-style four-card brochure chains in addition to the repository demo. Testing only `step blue | A | B` is not adequate.

Interaction research

The interaction model follows established drawing-editor conventions:

- Visio's Shapes pane supplies shapes by dragging them onto the page, while connection handles create connectors between shapes. Its selection model uses eight resize handles and direct text entry on a selected shape.
- OmniGraffle places the inspector beside the canvas, changes inspector panes according to the selected object, and uses a stencil browser for objects placed on the canvas.
- AppKit's `NSCollectionView` drag API writes a represented item to the dragging pasteboard, and the canvas accepts the pasteboard type as a dragging destination.
- AppKit's `UndoManager` records inverse model changes and automatically builds redo while an undo operation is performed. Persistent drawing changes need to use the same undo manager so the edit chain remains coherent.
- `NSScrollView` supplies independent scrolling and magnification. The canvas viewport and the inspector are separate scroll contexts.

Primary references:

- [Apple: NSCollectionView](#)
- [Apple: UndoManager](#)
- [Apple: NSScrollView magnification](#)
- [Apple: NSColorWell](#)
- [Microsoft: Add and connect shapes in Visio](#)
- [Microsoft: Select, move, resize, and rotate shapes](#)
- [Omni Group: Working with the inspectors](#)

Mac design

Window

The Blockgraph editor opens as its own resizable document utility window. It has four persistent regions:

1. a compact toolbar for undo, redo, and source view;
2. a canvas viewport that occupies the remaining central space;
3. a fixed-width, independently scrolling inspector on the right;
4. a compact status bar for validation state, zoom, fit, cancel, and save.

Changing selection updates only the interaction overlay and inspector. It must not re-fit, re-centre, or blank the canvas. Re-rendering after a model change keeps the current zoom and pan.

Inspector tabs and palettes

The right-hand pane has two tabs:

- **Inspector** is contextual: it shows canvas controls with no selection, card controls for a selected card, and connector controls for a selected connector.
- **Shapes** is a separate, scrollable library of every bundled and project/database-defined vector shape. Each entry previews the real normalized path rather than a generic placeholder.

In the Shapes tab, dragging a shape to empty canvas creates a standard card with that shape as its icon. Dropping it on an existing card or note replaces only that card's icon.

No selection: canvas and card palette

When nothing is selected, the inspector shows:

- the native Blockgraph grid dimensions and interaction guidance;
- a scrollable Blockgraph stencil palette containing Standard Card, Chevron Card, and Note Card;
- short keyboard guidance.

Each stencil has a graphical preview and can be dragged onto the canvas. A double-click also inserts it at the viewport centre for keyboard/accessibility use. Dropped cards snap to the nearest Blockgraph grid cell. A dropped step is inserted into the ordered step chain; a dropped note remains unconnected.

Card selection

Selecting a card shows:

- title and multi-line detail editors;
- card kind and Standard/Chevron shape controls where valid;
- accent colour;
- grid column, row, column span, and row span;
- icon shape, label, position, fill, and size;
- badge text, position, fill, border, and text colour;
- Delete Card.

The same title can be edited directly by double-clicking the card text. Dragging the body moves the card smoothly and commits a grid-snapped move on release. Visible handles resize the span. Arrow keys move by one cell; Delete removes the card; Return starts text editing.

Connector selection

Clicking a connector selects it and shows:

- the adjacent step endpoints;
- label;
- line colour;
- line width;
- solid, dashed, or dotted line style represented graphically;

- start and end marker previews;
- Reset to Automatic Connector.

A connector can be created by selecting one step and Shift-clicking its immediate neighbour, or by dragging a connection port between the two. This respects the existing positional Blockgraph grammar: notes and non-adjacent steps are not valid link targets. The connector remains attached as cards move.

Source view

Source view is an explicit secondary editing mode, not the main inspector. It edits the complete fence body, validates through the Rust parser, and returns to the visual view without losing viewport state. This provides access to future or uncommon syntax without exposing code as the graphical editor's normal UI.

Persistence

The editor keeps an ordered, source-preserving representation. Mutations own only their established fields:

- card changes rewrite the corresponding `step` or `note` line;
- connector changes rewrite the corresponding `edge` line;
- moves and resizes replace or insert the corresponding `// @node` annotation;
- deletion removes the card, its annotation, and edge lines whose positional gap no longer exists; deleting an interior step retains an automatic placeholder when needed so a later explicit edge cannot shift onto the wrong pair of cards;
- unrelated comments, `// @id`, `// @name`, blank lines, and unknown preserved tokens remain in their original order;
- values containing `|`, CR, or LF are normalised to spaces because the current grammar does not escape pipe-separated fields.

The Rust parser and renderer remain the authority for validation and preview. The Mac scene returned by Rust must expose the same geometry as the native Blockgraph layout plus the existing `card/icon/badge/edge` properties needed by the inspector.

Implementation sprints

Sprint 1 — shared model boundary

- Make the Rust Blockgraph scene use native layout grid dimensions.
- Expose complete card, icon, badge, edge, and real vector-shape metadata.
- Add parser/scene tests for production-style cards and exact geometry.

Exit: a real brochure Blockgraph produces selectable scene objects whose rectangles align with the rendered SVG.

Sprint 2 — dedicated Mac editor shell

- Route Blockgraph fences to a dedicated controller.
- Build a stable split layout with canvas, scrolling inspector, and toolbar.
- Add graphical standard/chevron/note stencils with real drag-and-drop.
- Preserve zoom and pan across inspector and preview changes.

Exit: all three stencil types can be dragged onto the grid without moving the viewport unexpectedly.

Sprint 3 — card editing

- Implement source-preserving card parsing and serialization.
- Add title/detail, shape, colour, grid, icon, and badge controls.
- Add direct title editing, drag movement, resize handles, keyboard movement, delete, and undo/redo.

Exit: every supported card property round-trips through Rust parse/render and Save emits only established syntax.

Sprint 4 — connector editing

- Select connectors by hit testing.
- Implement label, colour, width, style, markers, Shift-click/port creation, and reset to the implicit adjacent-step connector.
- Keep routes live while cards are moved.

Exit: connector edits survive preview refresh, save, close, and reopen.

Sprint 5 — validation and real-data QA

- Add focused Rust tests for scene metadata and source round-trips.
- Build the Cocoa target and run the Rust bridge test suite.
- Load a production-style database Blockgraph and exercise select, move, resize, text, colour, badge, edge, add, delete, undo, save, and reopen.
- Verify that invalid source is reported without replacing the last valid canvas.

Exit: the editor passes the full interaction checklist on the copied SQLite data without MySQL access or changes to a live source library.

Acceptance checklist

- The canvas fills the available window region and never jumps on selection.
- The inspector starts at the top, scrolls independently, and never falls below the window.
- Stencils are graphical, scrollable, and genuinely draggable.
- The Shapes tab previews bundled and project shape geometry; dropping an entry on a card changes its icon.
- Selection handles are small, correctly centred, and have usable hit targets.
- Movement is live and smooth; release snaps to the Blockgraph grid.

- Resize changes span and row span using existing annotations.
- Double-clicking card text edits it in place.
- Card and connector inspectors expose every property supported by the current Blockgraph model.
- Delete, undo, and redo work for every model-changing operation.
- Save preserves metadata/comments and produces source accepted by the Rust parser.
- A production-style four-card brochure Blockgraph reopens with the same text, colours, badges, edges, and geometry.

Implemented and verified

The Cocoa target now routes `blockgraph` fences to the dedicated editor. The Rust scene boundary uses the native Blockgraph grid and exposes cards, notes, icons, badges, explicit and automatic connectors, markers, and registered shape paths. The Shapes tab is built from the active project's bundled and project/database shape definitions, so it cannot show a previous project's shapes. Metadata-first sources such as `// @id ...` are classified as Blockgraph before preview and scene generation.

The completed interaction checks cover:

- true `NSCollectionView` drag-and-drop from the three-card palette;
- graphical colour selection without replacing or re-fitting the canvas;
- live card movement, grid-snapped persistence, resize, and direct title edit;
- card, icon, badge, and connector inspectors;
- Shapes-tab icon replacement and creation from a dragged real vector shape;
- connector creation from Shift-click or ports, line styling, markers, reset, and correct positional behaviour after deleting a middle step;
- undo and redo for add, edit, reset, move, resize, and delete;
- invalid source rejection while retaining the last valid canvas;
- Save, close, and reopen against both `docs/blockgraph-demo.md` and the copied SQLite brochure containing the Data Protection and Retention Chain.

The bridge test suite includes an explicit unlabelled-chevron connector case, and the Cocoa build and native Blockgraph parser/layout tests pass alongside the interactive checks.

Command-Line Reference for macOS

The Mac application embeds the supported headless helper at:

```
/Applications/DocCrate.app/Contents/Helpers/doc-crate
```

Add `/Applications/DocCrate.app/Contents/Helpers` to `PATH` to use the shorter examples on this page. There is no `.exe` suffix on macOS.

The exact installed command surface is authoritative:

```
doc-crate --help
doc-crate --help export-pdf
doc-crate --help database
doc-crate --help cache
```

Project and Authoring Commands

```
doc-crate --new-root ~/Documents/my-docs --name "My Docs" --json
doc-crate --project-config ~/Documents/my-docs --init --json
doc-crate --validate ~/Documents/my-docs/index.md --json
doc-crate --inspect ~/Documents/my-docs/index.md --json
doc-crate --lint ~/Documents/my-docs --json
doc-crate --find-replace ~/Documents/my-docs/index.md --find "old" --replace "new" --global -
-json
doc-crate --page-break-before ~/Documents/my-docs/index.md "## Appendix" --json
```

Commands that modify source should be used only inside the project or managed checkout explicitly placed in scope.

Diagrams

```
doc-crate --list-diagrams ~/Documents/my-docs/index.md --json
doc-crate --find-diagram "access"
doc-crate --chart-types
doc-crate --chart-help mermaid-flowchart
```

The diagram engine is DocCrate's fork of Selkie, used as a native Rust library. See [Architecture](#).

Export

```
doc-crate --export-pdf ~/Documents/my-docs ~/Desktop/my-docs.pdf
doc-crate --export-docx ~/Documents/my-docs ~/Desktop/my-docs.docx
doc-crate --export-editable-docx ~/Documents/my-docs ~/Desktop/my-docs-editable.docx
doc-crate --export-svg ~/Documents/my-docs ~/Desktop/my-docs.svg
doc-crate --generate-diagram-pack ~/Documents/my-docs --out ~/Desktop/my-docs-diagrams
doc-crate --analyze-pdf ~/Documents/my-docs --json
```

```
doc-crate --export-pdf-layout ~/Documents/my-docs ~/Desktop/my-docs-layout.png
```

Common export options include page size, portrait/landscape orientation, PDF compression, and PDF modification protection. Use the relevant `--help` topic for the build's exact flags. See [Pagination on macOS](#) for automatic preview and page-authoring guidance.

Saved DocCrates

```
doc-crate --db-list --query "security brochure" --json
doc-crate --db-open <root-key> --json
doc-crate --db-status <root-key-or-checkout> --json
doc-crate --db-versions <root-key-or-checkout> --json
doc-crate --db-restore-version <root-key-or-checkout> <revision-id> --json
doc-crate --db-save <checkout-path> --json
doc-crate --db-save-as <checkout-path> "New title" --json
doc-crate --db-import ~/Documents/existing-doccrate --json
```

Database-opened projects are materialised under the Mac managed-checkout location reported by `doc-crate --help cache`. Agents must use the returned checkout path and the required save command.

Standard Content and Sync

```
doc-crate --standard-content-search --query privacy --json
doc-crate --standard-content-show 42 --json
doc-crate --standard-content-expand '{{standard:42:"privacy notice"}}' --json
doc-crate --standard-content-validate ~/Documents/my-docs/index.md --json
doc-crate --standard-content-update 42 --content-file privacy.md --expect-hash <hash> --json
doc-crate --standard-content-sync --json
```

Sync uses the signed public connection package, macOS Keychain password, and approved RDS CA. Never pass the database password on the command line.

TCL

```
doc-crate --tcl -e 'puts [doccrate/status]' --result
doc-crate --tcl ./scripts/publish.tcl --json
```

Cocoa GUI Control

The Mac headless helper does not use the Windows live-viewer named-pipe flags. Use AppleScript/Apple Events for the live AppKit application, or the documented `--mac-ui` bridge where appropriate.

Shared Locations

| Purpose | Default |
|----------------------|---|
| Application data | ~/Library/Application Support/Cocoa DocCrate/ |
| Managed checkouts | ~/Library/Application Support/Cocoa DocCrate/Managed Checkouts/ |
| Generated/cache data | ~/Library/Caches/Cocoa DocCrate/ |
| Embedded helpers | /Applications/DocCrate.app/Contents/Helpers/ |

For more detail see [macOS Command-Line Client](#), [AppleScript](#), and [MCP](#).

Back to [Home](#)

Connection Package and Credential Protection on macOS

DocCrate keeps public MySQL routing metadata separate from the database password. The normal macOS deployment has three distinct components:

| Component | Location | Contains |
|---------------------------|---|--|
| SQLite library | ~/Library/Application Support/Cocoa DocCrate/security_text.sqlite | Local standard text, diagrams, saved DocCrates, and sync metadata. |
| Signed connection package | ~/Library/Application Support/Cocoa DocCrate/doccrate-db-connection.zip and adjacent .sig | Public host, database, user, TLS CA name, port, and table-prefix metadata. |
| Native credential | macOS Keychain | The database password only. |

The notarized application does not contain the SQLite library, connection ZIP, PAQ, release private key, or database password.

Install the public connection package

Copy the delivered files without renaming them:

```
~/Library/Application Support/Cocoa DocCrate/doccrate-db-connection.zip
~/Library/Application Support/Cocoa DocCrate/doccrate-db-connection.zip.sig
```

The detached signature is mandatory. DocCrate verifies it with the embedded release public key before it reads the package. The ZIP may be inspected by an authorised release administrator, but it must not contain any password-like fields, private keys, PAQs, or credential exports.

If the package specifies `rds-ca.pem`, copy that CA file into the same Application Support directory. Administrators can select a non-default package or SQLite library from **DocCrate > Settings > Library Locations**.

The CLI and MCP server use the same Application Support paths. For a temporary or managed override, set `DOCCRATE_DB_CONFIG_PACKAGE` to the ZIP path and keep its `.sig` alongside it.

Normal password provisioning: signed PAQ

The authorised release operator creates and signs a short-lived PAQ using the private release key. They deliver the PAQ file and its setup PIN through separate approved channels. The password is never included with the application or this documentation.

The recipient uses the graphical flow:

1. Choose **Library > Install Signed Credential Package....**
2. Select the delivered PAQ file.
3. Enter the separately received PIN.

4. DocCrate verifies the PAQ, writes the password to Keychain, performs a TLS MySQL connection test, and removes the transient PAQ only after success.

The corresponding CLI flow is:

```
doc-crate --credentials-install-paq --signed-paq /path/to/password.paq \  
--package "$HOME/Library/Application Support/Cocoa DocCrate/doccrate-db-connection.zip"
```

The command asks for the PAQ setup code interactively. Do not pass a password, PAQ token, or setup code in an environment variable, script, process argument, or project file.

Manual Keychain recovery

Use this only if the signed PAQ flow is unavailable and an authorised administrator has separately provided the database password. Create or update a generic-password item in the current user's login Keychain:

```
Service: com.doccrate.database.credentials  
Account: docrate db connection
```

Set the item's password field to the separately supplied database password. DocCrate accepts this raw password recovery entry. The historic compatibility account `doccrate db connection` is also read, but new manual entries must use `docrate db connection`.

Never add the password to the connection ZIP, detached signature, SQLite file, AppleScript, shell script, or environment.

Security boundary

The ZIP signature establishes the origin and integrity of the public routing metadata. The PAQ signature and setup PIN protect password provisioning. macOS Keychain protects the recovered password for that user. DocCrate requires TLS and certificate validation for MySQL connections; use a least-privilege database account and rotate credentials through the normal operator process.

For complete installed-app setup, see [Installing DocCrate on macOS](#).

Database-Backed DocCrates

DocCrate stores database-backed documentation projects as snapshots of source content, not as generated export artefacts. A saved DocCrate contains the Markdown navigation set, project control files needed to render and edit those documents, and project shape definitions.

User Model

The user-facing model is a library:

- **Save Current DocCrate** stores the currently open docs root in the local SQLite database.
- **Save DocCrate As** stores the currently open docs root as a separate named saved entry, useful when the same working copy needs a customer-specific or review-specific library identity.
- **Import DocCrate Folder** accepts one project or a collection folder, discovers configured project roots recursively, and stores each as a normal saved entry without changing the source folders.
- **Saved DocCrates** opens a searchable browser listing saved roots. Holding `Shift` while choosing the menu item opens the generated Markdown library page instead. Browser search tokenizes the query and requires every term to match somewhere in the saved root metadata or document corpus. Each result shows an extended excerpt from the start of its first document.
- Clicking an open link materializes the saved root into a managed local cache and switches the viewer to that cache.
- Managed cache folders include `.doccrate/checkout.json` with the database root key, source hash, revision id, and source manifest. Reopening a saved root refuses to replace an existing managed checkout when its source files have local edits.
- Recent projects prefer saved database roots. If the local database has no recent roots, the existing filesystem recent list is used.
- Shape lookup loads database shapes for the current root first. Filesystem `.shapes/*.shape` entries are treated as cache/import material and are saved back into `doccrate_shapes` when the managed checkout is saved.

This keeps the normal editing, rendering, link resolution, export, and diagram editor flows unchanged. The database is the durable library. The materialized folder is the working copy.

Local Schema

The local database uses the same managed SQLite database as standard content. These tables are created on demand:

| Table | Purpose |
|--------------------|--|
| doccrate_roots | One saved DocCrate root, keyed by a stable root_key. |
| doccrate_documents | Markdown documents under the root, with relative path, title, display order, hash, and UTC update time. |
| doccrate_assets | Project control/support files such as <code>.doccrate/project-config.md</code> , highlighting config, themes, syntaxes, and export assets. |
| doccrate_shapes | Project-level <code>.shape</code> definitions. |

| | |
|--------------------------|---|
| doccrate_recent_roots | Recent database roots, independent of the legacy filesystem recent file. |
| doccrate_document_search | FTS index for saved document search pages and future interactive search UI. |
| doccrate_sync_items | Shared sync metadata, extended with root, document, asset, shape, and recent-root item kinds. |

Every syncable row has:

- a UTC `updated_utc` timestamp;
- a SHA-256 `content_hash`;
- an item entry in `doccrate_sync_items`;
- an `origin` marker of `sqlite`, `mysql`, `sync`, or `bootstrap`.

Storage Rules

The snapshot stores source content only:

- included: Markdown documents discovered by the normal sidebar scan;
- included: `.doccrate/project-config.md`, `.doccrate/highlight.toml`, `.doccrate/themes.json`, `.doccrate/syntaxes/*`, `.doccrate/themes/*`, and `.doccrate/export-assets/*`;
- included: `.shapes/*.shape`;
- excluded: `.git`, `target`, runtime data directories, generated sync reports, generated database/search pages, TCL run logs, and export outputs such as PDF, DOCX, SVG, and PNG files;
- excluded: managed checkout metadata such as `.doccrate/db-root-key`, `.doccrate/db-source-path`, `.doccrate/checkout.json`, and `.doccrate/checkout-lock.json`.

Imported export assets retain their source formats, including SVG. Import keys are derived from canonical source paths rather than copied checkout markers, so database-backed folders can be imported safely without taking over their prior database identity.

The Saved DocCrates browser reads `product/product_name`, `region`, and `business_unit` from each saved `.doccrate/project-config.md`. These values are shown directly below the saved brochure title and participate in browser search, so users can find brochures by their business context even when those terms do not appear in the document title or body.

Specific filters are available in the same search box:

```
product:"Lexis+ AI" region:EMEA bu:"Risk Solutions" security
```

`product:`, `region:`, and `bu:` generate predicates against their dedicated saved-root metadata columns. Values containing spaces should be quoted. Any unprefixed terms remain broad AND filters across title, path, all three metadata fields, and saved document text.

By default, import also compares project-variable-resolved source with managed standard content. The document and eligible standards are expanded once with the same project variables; line-ending-normalized exact matches are then mapped back to literal document spans and replaced by labelled `{{standard:n:"label"}}` tokens. Internal whitespace and Markdown structure remain significant, and ambiguous, fenced, or unresolved content is not replaced. Import saves the literal source as a

restorable baseline revision before making the consolidated snapshot active. The import report records every reference and replacement count. `--no-standard-substitutions` creates only a verbatim source snapshot.

Database-opened roots are materialized into the macOS managed-checkout folder:

```
~/Library/Application Support/Cocoa DocCrate/Managed Checkouts/<name>-<root-key>/
```

For other installs, DocCrate reports the exact cache folder in `doc-crate --help cache`, `MCP docrate_help` with `topic=cache`, `docrate_doctor`, and `doccrate://discovery/project-roots`.

DocCrate writes `.doccrate/db-root-key` into that managed working copy so later saves update the same database root instead of creating a second library entry.

Project History

`Project > Project History` opens a split revision browser for the active managed DocCrate. The left side lists saved revisions and identifies the active version; the larger right side previews the selected document's historical Markdown. The list and preview scroll independently.

Recover is available only for an inactive revision with a complete snapshot. Recovery does not move or delete history: DocCrate copies the selected snapshot, adds a restore marker, saves it as a new active revision, and rematerializes the managed checkout. Dirty or stale checkouts retain the existing restore safety checks and must be saved or reconciled before recovery.

Sync Process

The current standard-content sync model should be extended rather than replaced:

1. Ensure the MySQL schema exists using the configured `DC2026_` prefix.
2. Ensure `DC2026_doccrate_library_items` exists. This table is a portable sync envelope for saved roots, documents, assets, shapes, and recent roots.
3. Export normalized local rows to payload items with a stable item-key hash, JSON payload, SHA-256 content hash, and UTC timestamp.
4. Compare payload rows by stable key hash and `updated_utc`.
5. Push newer SQLite payload rows to MySQL and pull newer MySQL payload rows back into the normalized SQLite tables.
6. Keep the existing empty-source guards: never overwrite a populated side with an empty side unless an explicit future force option is added.
7. Rebuild local FTS/search rows and shape registries after a pull.

The first sync of a full local DocCrate library is expected to be large because it pushes every saved document, asset, and shape.

UI Process

The UI should feel like normal DocCrate work:

- **Project > Save Current DocCrate** snapshots the open root and reports the document, asset, and shape counts.
- **Project > Save DocCrate As** prompts for a library title and snapshots the open root as a separate saved root.
- `doc-crate --db-save-as <checkout-path> "<title>" --json` does the same separate-root snapshot non-interactively, which is useful for agents creating a new brochure from an existing managed checkout.
- **Project > Import DocCrate Folder** prompts for a project or collection path, imports it with exact standard-content substitution, and reports project, document, asset, and substitution counts.
- **Project > Saved DocCrates** opens the saved-crate browser. Holding `Shift` opens `database-library.md` from the generated cache folder, with saved root summaries and saved document search entries.
- `doc-crate --db-versions <root-key|checkout-path> --json` lists saved revisions. `active=true` marks the current saved database state, and `restorable=true` means the row includes the full file snapshot required for restore.
- `doc-crate --db-restore-version <root-key|checkout-path> <revision-id>` restores a restorable revision by copying it, adding a hidden dated `{# restored from revision ... #}` authoring comment, saving that copy as the latest revision, and rewriting the managed checkout. Legacy metadata-only revision rows remain visible for history but cannot be restored.
- Open links in that page use `doccrate-db://open/<root-key>`.
- Opening a database root records it in `doccrate_recent_roots`.
- **File > Recent DocCrates** lists up to eight recent brochures, combining saved database DocCrates with local filesystem roots from the legacy recent file.
- The shape editor reads database shapes for the current root, saves changes to `doccrate_shapes`, and refreshes the managed checkout `.shapes/*.shape` copy for agents and file-based tools.

Future refinements can replace the generated page with a fully interactive search panel, but the data model and open links remain the same.

The next-stage storage design is captured in [Database-Primary Brochure Storage](#). That design makes saved DocCrates the default File menu workflow, treats materialized folders as managed checkouts, and keeps explicit local-folder import/export as a secondary workflow.

Database-Primary Brochure Storage

macOS implementation note: This design is implemented on Mac with native AppKit project commands, managed checkouts under Application Support, generated data under Library Caches, and passwords in the macOS Keychain. Older Windows paths or menu names in historical design discussion are not Mac install instructions.

This design makes the replicated DocCrate database the authoritative store for brochures while still giving users and AI agents normal filesystem folders to edit. The local folder becomes a managed checkout/cache, not the source of truth, unless the user explicitly chooses a local-folder workflow.

Goals

- Make **New**, **Open**, **Save**, **Save As**, and **Recent** work against saved database DocCrates by default.
- Let AI agents work on ordinary local filesystem copies so existing MCP, CLI, TCL, editor, export, and validation flows keep working.
- Keep the authoritative brochure in the replicated SQLite/MySQL library.
- Make local-folder import/export explicit and understandable.
- Prevent accidental data loss when a cached working copy has local edits or the database version changed elsewhere.
- Avoid storing generated artefacts, local data, credentials, or secrets in the saved brochure.

User Model

The user should see DocCrate as a document application with a library:

| User action | Primary behavior |
|------------------------------|--|
| New DocCrate | Create a new saved brochure in the local replicated database, materialize a managed checkout, and open it. |
| Open DocCrate | Open the database browser overlay, search saved brochures, and check out the selected brochure to the app-managed cache. |
| Recent DocCrates | Show recently opened database brochures first, with sync/dirty state. |
| Save | Commit the current managed checkout back into the database root. |
| Save As | Create a new database brochure identity from the current checkout. |
| Open Local Project Folder | Explicitly open or import a filesystem project. This is not the default open path. |
| Save Project To Local Folder | Export a copy of the authoritative brochure to a chosen folder for handoff, agent tooling, or archive use. |

The status bar should make the storage mode obvious:

Database DocCrate: LexisPlus Protege Security Brochure | Saved | Synced
Database DocCrate: LexisPlus Protege Security Brochure | Unsaved local edits | Offline

Storage Roles

DocCrate needs four distinct storage roles:

| Role | Location | Purpose |
|------------------------|-----------------------------------|--|
| Authoritative library | Local SQLite, replicated to MySQL | Durable source of saved brochures, recents, search, shapes, assets, and sync metadata. |
| Managed checkout/cache | App-local filesystem folder | Editable source working copy for the viewer, GUI editors, CLI, MCP, TCL, and AI agents. |
| Generated cache | <cache>\generated | Default PDF, Word, SVG, layout PNG, snapshot PNG, database report, and generated browser-page outputs. |
| Explicit local folder | User-chosen path | Import/export/handoff location only, unless the user intentionally opens local mode. |

The viewer and export code continue to operate on a folder. The new rule is that ordinary File menu operations open a database checkout, and Save commits that checkout back to the database.

Cache Location

The Mac managed checkout location is:

```
~/Library/Application Support/Cocoa DocCrate/Managed Checkouts/<sanitized-title>-<root-key-prefix>/
```

Generated outputs live beside those checkouts, not inside them:

```
~/Library/Caches/Cocoa DocCrate/Generated/
```

The exact path is reported by `doc-crate --help cache`, MCP `docrate_help` with `topic=cache`, `docrate_doctor`, and `docrate://discovery/project-roots`.

Each checkout should contain:

```
.docrate/db-root-key
.docrate/db-source-path
.docrate/checkout.json
.docrate/checkout-lock.json
```

`checkout.json` records the root key, root title, database revision, source content hash, materialized time, and the database path used for replication. `checkout-lock.json` records the current process/session owner so a second window or agent can warn before editing the same checkout.

Database Schema Direction

The existing tables already hold the snapshot:

| Table | Current use |
|--------------------------|---|
| doccrate_roots | Saved brochure identity, title, source path, materialized path, counts, hash, timestamps. |
| doccrate_documents | Markdown files under the root. |
| doccrate_assets | .doccrate support files and export assets. |
| doccrate_shapes | Project shape definitions. |
| doccrate_recent_roots | Database recents. |
| doccrate_document_search | Search index over saved documents. |
| doccrate_sync_items | Sync envelope for SQLite/MySQL replication. |

Add a revision/checkpoint layer rather than relying only on `updated_utc` :

| Table | Purpose |
|-------------------------|---|
| doccrate_root_revisions | One row per committed root snapshot, with <code>root_key</code> , <code>revision_id</code> , parent revision, manifest hash, author/source, created UTC, and optional note. |
| doccrate_checkout_state | Local-only app metadata mapping <code>root_key</code> to cache path, base revision, dirty flag, lock/session owner, last opened UTC, and last save UTC. |
| doccrate_conflicts | Optional local-only records for unresolved save conflicts. |

The first implementation can create a revision row at each successful `save_root / save_root_as`. Later, incremental document-level commits can use the same revision table without changing the user model.

Manifest And Dirty Checking

Every checkout should have a manifest of saved source files:

```
rel_path
kind: markdown | asset | shape
content_hash
byte_len
updated_utc
```

On Save, DocCrate compares the current checkout against the manifest:

1. Scan documents, assets, and shapes using the existing inclusion/exclusion rules.
2. Ignore generated exports, sync reports, runtime logs, credentials, `.git`, and local data.
3. Compute current hashes.
4. Save only if at least one source item changed, unless the user explicitly requests a full resnapshot.
5. Record a new root revision and update `doccrate_sync_items`.

This gives agents a simple contract: edit files in the checkout, then call Save or the matching MCP/CLI command.

Save And Conflict Rules

Save from a database checkout should be guarded:

| Condition | Behavior |
|---|---|
| Checkout base revision equals database latest | Commit changed files as a new revision. |
| Checkout has no changed source files | Report "Already saved"; do not create a noisy revision. |
| Database latest differs and checkout has no local edits | Refresh the checkout from the database. |
| Database latest differs and checkout has local edits | Open a conflict overlay. |
| Checkout folder is missing critical marker files | Refuse database save and offer Save As or Import. |

The conflict overlay should offer:

- **Review changes:** show database version, local version, and changed file list.
- **Reload from database:** discard local checkout changes after confirmation.
- **Save as new DocCrate:** preserve local edits under a new database identity.
- **Force save:** available only behind an explicit confirmation, and still recorded as a new revision.

Empty-data guards remain mandatory. An empty checkout must not overwrite a populated database root unless the user chooses a future destructive recovery command with a clear confirmation.

UI Design

File Menu

Primary storage commands should live in the File menu because they are normal document actions:

| Command | Behavior |
|---------------------------------|---|
| New DocCrate... | Create database root, materialize checkout, open it. |
| Open DocCrate... | Open the database browser overlay. |
| Save | Commit current checkout to database. |
| Save As... | Create a new database root from current checkout. |
| Recent DocCrates | Database recents with dirty/offline badges. |
| Open Local Project Folder... | Explicit local workflow. |
| Save Project To Local Folder... | Export current database root or local project to a selected folder. |

The current local-folder command is labelled `Open Local Project Folder...` so users do not think folders are the ordinary storage location.

Library Menu

The Library menu keeps operational database commands:

| Command | Behavior |
|----------------------------------|--|
| Install Signed Credential PAQ... | Verify the delivered PAQ, store the MySQL password in macOS Keychain, test it, and remove the PAQ after success. |
| Test Connection | Test database connectivity. |
| Sync Library | Sync SQLite/MySQL and show only changed rows in the report. |
| Saved DocCrates | Same browser as Project > Open DocCrate, useful for discoverability. |
| Database Status | Show SQLite path, MySQL config source, last sync, pending item count, and cache location. |

Database Browser Overlay

The browser should be the natural front door:

- search by brochure title, document title, document content, tag, and root key;
- show title, last modified UTC, last opened UTC, document count, shape count, sync state, dirty checkout state, and a short content snippet;
- show first-page or index thumbnail when available;
- provide Open, Duplicate, Rename, Export To Folder, and Delete from Library actions;
- support keyboard search and Enter-to-open.

Local Folder Flow

`Open Local Project Folder...` should ask what the user wants:

| Choice | Result |
|-----------------------------|--|
| Import to database and open | Recommended. Creates or updates a database root and opens a managed checkout. |
| Open local folder only | Opens the selected folder without database authority and displays a local-mode status. |

`Save Project To Local Folder...` exports the current source snapshot to the selected folder and warns before overwriting existing source files. It should not change the active project identity unless the user then opens that folder.

Agent Workflow

AI agents should receive and modify managed checkouts, not the database tables directly:

```
agent opens/creates saved DocCrate
-> DocCrate materializes managed checkout
-> agent edits Markdown/assets/shapes in checkout path
-> agent calls save/validate/export
-> DocCrate commits changed source files to the database
-> background sync replicates to MySQL
```

Add MCP/CLI verbs around this workflow:

| Verb | Purpose |
|--------------------------------|---|
| docrate_list_doccrates | Search database roots and return root keys, titles, timestamps, and sync state. |
| docrate_checkout_doccrate | Materialize or reuse a managed checkout and return its path, root key, and base revision. |
| docrate_save_doccrate | Commit a checkout back to the database after conflict and empty-data checks. |
| docrate_doccrate_status | Return dirty files, base revision, latest database revision, sync state, and cache path. |
| docrate_export_doccrate_folder | Save a source-only copy to a user-provided local folder. |
| docrate_import_doccrate_folder | Import a local folder into the database and open a managed checkout. |

Agents should never be asked to edit the database payload rows directly. The folder interface keeps existing tooling reliable and auditable.

Sync Model

The local SQLite database remains the immediate authority for the running app. MySQL is the replicated shared store:

1. UI opens and saves against SQLite.
2. Background or explicit Sync pushes/pulls through the existing `DC2026_` MySQL sync envelope.
3. Pulled roots update local search and recents.
4. If a pulled root is open and the checkout has no local edits, the app can offer to refresh it.
5. If a pulled root is open and the checkout has local edits, the app marks it "database changed elsewhere" and routes Save through the conflict overlay.

Offline work is allowed. The status bar should say `Offline, saved locally` when SQLite has the latest local revision but MySQL has not been reached.

Implementation Plan

Phase 1: Cache Contract

- Rename `materialized_roots_dir` behavior to checkout/cache language.
- Stop deleting an existing checkout unless it is clean or the user confirms.

- Write `checkout.json` with base revision and manifest hash.
- Add a dirty check helper for documents, assets, and shapes.
- Make database Save compare checkout base revision with database latest.

Phase 2: Database-Primary UI

- Add File > New DocCrate, Open DocCrate, Save, Save As, Recent DocCrates.
- Rename the existing folder command to Open Local Project Folder.
- Add Save Project To Local Folder.
- Update the saved-crate overlay with dirty/sync badges and primary Open behavior.
- Display storage mode and sync state in the status bar.

Phase 3: Agent Surface

- Add CLI/MCP verbs for list, checkout, status, save, import, and export.
- Return the managed checkout path in machine-readable JSON.
- Add guards so agent-triggered save refuses conflicts unless the agent passes a documented conflict strategy.

Phase 4: Revisions And Conflict UI

- Add `doccrate_root_revisions` and local `doccrate_checkout_state`.
- Record one revision per successful save.
- Add a conflict overlay with review, reload, save-as-new, and force-save choices.
- Add cache cleanup for old clean checkouts while preserving dirty ones.

Acceptance Criteria

- A normal user can create, open, save, reopen, and sync a brochure without choosing a filesystem folder.
- An agent can get a checkout path, edit files, save back to the database, and export PDF/Word using existing commands.
- Reopening a database brochure never silently deletes unsaved local checkout changes.
- Local-folder commands are explicit and do not confuse the authoritative database identity.
- Generated files, credentials, runtime databases, sync reports, and export artefacts are excluded from saved brochure snapshots.
- The status bar always tells the user whether they are working in database mode or local-folder mode.

macOS Command-Line Client

The notarized DocCrate app embeds the supported headless client at:

```
/Applications/DocCrate.app/Contents/Helpers/doc-crate
```

Add that directory to `PATH` if desired. The CLI, MCP server, and Cocoa app use the same default SQLite library and cache locations described in [Installing DocCrate on macOS](#).

Common commands

```
# Validate, inspect, and export local Markdown.
doc-crate --validate ~/Documents/my-docs/index.md --json
doc-crate --lint ~/Documents/my-docs --json
doc-crate --export-pdf ~/Documents/my-docs --out ~/Desktop/my-docs.pdf

# Inspect saved DocCrates from the local SQLite library.
doc-crate --db-open <root-key> --json
doc-crate --db-save <checkout-path> --json
doc-crate --db-save-as <checkout-path> "New document title" --json

# Governed standard content and library sync.
doc-crate --standard-content-update <reference> --expect-hash <hash> --json
doc-crate --standard-content-sync --json
```

The exact installed command surface is authoritative:

```
doc-crate --help
```

Shared macOS locations

| Purpose | Default location |
|---------------------------|---|
| SQLite library | ~/Library/Application Support/Cocoa DocCrate/security_text.sqlite |
| Saved DocCrate checkouts | ~/Library/Application Support/Cocoa DocCrate/Managed Checkouts/ |
| Signed connection package | ~/Library/Application Support/Cocoa DocCrate/doccrate-db-connection.zip |
| Generated exports/cache | ~/Library/Caches/Cocoa DocCrate/Generated/ |

Use `DOCCRATE_MAC_STORAGE_ROOT` only for an explicitly managed or isolated environment. The Cocoa app, CLI, and MCP then derive their shared storage from that root.

Cocoa GUI control

The headless sidecar deliberately does not implement the old live-viewer control flags. Use `--mac-ui <command>` for its supported Cocoa control bridge, or use [AppleScript and Apple Events](#) for typed

native GUI automation.

Credential safety

`--standard-content-sync` reads public MySQL metadata from the signed connection package and the password from Keychain. Do not provide a password on the command line. For PAQ and manual-recovery details, see [Connection Package and Credential Protection](#).

MCP Stdio Server on macOS

The DocCrate application embeds its MCP server at:

```
/Applications/DocCrate.app/Contents/Helpers/doccrate-mcp
```

It wraps the OS-portable Rust core and headless `doc-crate` helper. It does not automate the native Cocoa window through the legacy Windows viewer channel; use AppleScript for live AppKit control.

Configuration

```
{
  "mcpServers": {
    "doccrate": {
      "command": "/Applications/DocCrate.app/Contents/Helpers/doccrate-mcp",
      "env": {
        "DOCRATE_EXE": "/Applications/DocCrate.app/Contents/Helpers/doc-crate"
      }
    }
  }
}
```

`DOCRATE_EXE` is a historical environment-variable name; on macOS its value is the extensionless embedded helper.

Discovery Resources

Agents should inspect discovery before guessing commands or paths:

```
doccrate://discovery/agent-card
doccrate://discovery/capabilities
doccrate://discovery/project-roots
```

The project-roots resource reports current managed checkouts and generated locations using Mac paths.

Main Tool Families

| Family | Typical tools |
|-----------------|--|
| Help and health | docrate_help, docrate_doctor |
| Project setup | docrate_new_root, docrate_project_config |
| Authoring | docrate_write_doc, docrate_append_doc, docrate_find_replace, docrate_page_break_before |
| Validation | docrate_validate, docrate_lint, docrate_inspect |
| | |

| | |
|-------------------|---|
| Diagrams | docrate_list_diagrams, docrate_diagram_source, diagram help and snapshot tools |
| Pagination/export | PDF analysis, page image, PDF, SVG, Word, editable Word, and diagram-pack tools |
| Standard content | Search, show, expand, validate, create, update, and diagram validation |
| Saved DocCrates | List, open, status, history, restore, save, save-as, and import tools |
| Scripting | docrate_run_tcl |

Tool names and schemas returned by the installed server are authoritative.

macOS Examples

```
docrate_validate { "file": "/Users/example/Documents/project/index.md" }
docrate_list_diagrams { "file": "/Users/example/Documents/project/index.md", "json": true }
docrate_export_pdf { "input": "/Users/example/Documents/project", "out":
"/Users/example/Desktop/project.pdf" }
docrate_standard_content_validate { "file": "/Users/example/Documents/project/whitepaper.md"
}
```

Managed Checkout Contract

For a saved DocCrate:

1. open by root key;
2. use only the checkout path returned by the tool;
3. inspect `.doccrate/checkout.json`;
4. keep generated outputs outside the checkout;
5. validate changes;
6. use the required database save or save-as operation.

Do not invent a checkout path or work directly in internal SQLite files.

Credentials

MCP database operations share the Cocoa app's signed public connection package, RDS CA, and macOS Keychain credential. Never include passwords, PAQ content, or connection secrets in tool arguments or prompts.

Live App Automation

Use [AppleScript and Apple Events](#) for navigation, mode changes, editors, native exports, settings, and agent-referral actions in the running Mac app. The Debug-only local UI test socket is not a release integration.

Back to [Home](#) | [Agent Discovery](#)

Project Config

DocCrate projects use a human-editable Markdown config file for values that repeat across customer documentation.

```
.doccrate/project-config.md
```

The file is project control data, not customer documentation. It is hidden from normal sidebar/export scans and is opened explicitly from:

```
View > Project Config
```

The CLI and MCP can also inspect or create it.

On macOS, choosing **View > Project Configuration...** creates this file when it is missing and immediately opens the generated Variables table. New files use conservative defaults: `docheader`, `doctrailer`, `header_enabled`, `footer`, `spelling_enabled`, and `spelling_mode` are all `off`. `paginate_for_pdf` defaults to `on` and controls the Cocoa viewer's automatic paginated PDF preview. Set it to `off` for online documentation to avoid the background pagination pass. This does not disable an explicit PDF export.

`page_size` accepts `a4` or `letter`, while `page_orientation` accepts `portrait` or `landscape`. If `page_size` is missing, the Mac uses US Letter for the United States region and A4 elsewhere. When DocCrate creates or extends a config through the Mac app, it writes that resolved size into the Variables table so the project remains stable when moved to another computer.

Variables Table

The supported section is `## Variables`.

```
## Variables

name	value	description
product	DMS Connections	Product or documentation set name
customer	Example LLP	Customer display name
docheader	on	Draw the standard document cover banner
docheader_text	{{product}} Security Brochure	Cover banner title
doctrailer	on	Draw the standard vector final-page trailer banner
doctrailer_text	{{product}} Trust Center	Trailer title
header_enabled	on	Draw export header text from page 2 onwards
header_text_left	{{product}}	Optional left header text
header_text_right	{{MONTHYEAR}}	Optional right header text
footer	on	Draw a red footer rule on exported pages except the final page
footer_text	{{product}} customer documentation	Optional text below the rule
spelling_enabled	on	Include spelling checks in lint/export QA
spelling_locale	en-ZA	English spelling dialect
spelling_mode	warn	off, warn, or strict
spelling_allow	DocCrate, LexisNexis	Project-specific accepted terms
```

Rules:

- `name`, `value`, and `description` columns are read case-insensitively.
- Variable names must start with a letter or `_`.
- Variable names may contain letters, numbers, `_`, `-`, and `.`.
- Duplicate names are warnings; the later value wins.
- Missing descriptions are allowed but warned because variables should be understandable to humans and agents.

Substitution

Use variables with double braces:

```
# {{product}} Customer Documentation
```

`DocCrate Project` is accepted as an alias for `DocCrate Project`. If a project defines an explicit `product_name` row, that value is used; otherwise the alias resolves to the configured `product` value.

DocCrate substitutes values before Markdown parsing, so inserted values can affect headings, paragraphs, tables, links, image paths, diagrams produced by expanded standard content, and export output.

Fenced code blocks keep raw tokens. This protects examples, templates, Mermaid source, and graphical-editor round trips from accidentally saving resolved values over authored source.

Authoring-only notes can be written as `{# ... #}`. DocCrate removes these comments from PDF and Word exports while preserving them inside fenced code examples.

Unknown variables remain visible as `{{name}}` and are reported by lint.

Spelling

Enable spelling checks with:

```
spelling_enabled	on	Include spelling checks in lint/export QA
spelling_locale	en-ZA	English spelling dialect
spelling_mode	warn	off, warn, or strict
spelling_allow	DocCrate, LexisNexis	Project-specific accepted terms
```

Supported spelling profiles are `en-US`, `en-GB-ise`, `en-GB-ize`, `en-CA`, `en-AU`, `en-NZ`, `en-ZA`, and `en-EU`. `spelling_mode=strict` reports findings as errors. Project-specific spelling files live under:

```
.doccrate/spelling/allow.txt
.doccrate/spelling/deny.txt
.doccrate/spelling/replacements.tsv
```

Built-In Dynamic Values

| Token | Expansion |
|---------------|--|
| {{MONTHYEAR}} | Current month and year, such as July 2026. |
| {{YEAR}} | Current four-digit year, such as 2026. |

Dynamic values can be used directly in documents or inside project variable values such as `docheader_text`, `doctrailer_text`, `header_text_left`, `header_text_right`, and `footer_text`.

Opening a document in the viewer normalizes literal English month/year values such as `January 2026` to `{{MONTHYEAR}}` in the Markdown source. Inline code, fenced code blocks, and `.doccrate/project-config.md` are not rewritten. For database-backed projects, the normalized document is saved to the managed DocCrate snapshot.

Export text settings also pass through the standard-content lookup used by normal Markdown, so values such as ``# LexisNexis Cloud Infrastructure`

Secure Private Cloud Environments

DocCrate Project services, including production environments, are hosted in LexisNexis secure private cloud environments within Amazon Web Services (AWS) and Google Cloud Platform (GCP).

These cloud environments use industry-standard physical, environmental, and operational safeguards, including facility security, access controls, monitoring, encryption, and independent control-framework audits.

LexisNexis designs and operates its cloud environments with resilient architecture, secure network controls, and continuous monitoring to support availability, security, and scalability.

``` can be used when a managed standard-content database is configured.

## Document Cover And Trailer

Enable the standard cover and vector trailer with:

```
docheader	on	Draw the standard document cover banner
docheader_text	{{product}} Security Brochure	Cover banner title
doctrailer	on	Draw the standard vector final-page trailer banner
doctrailer_text	{{product}} Trust Center	Trailer title
doctrailer_legal	LexisNexis, Lexis, Lexis+ and the Knowledge Burst logo are trademarks	
and Lexis+ AI is a trademark of RELX Inc. Other products may be trademarks or registered		
trademarks of their respective companies. Copyright (c) {{YEAR}} LexisNexis.	Left	
trailer trademark and legal text column		
doctrailer_materials_heading	OTHER MATERIALS	Right trailer materials heading
doctrailer_materials	Available from our Trust Centre Brochures Whitepapers Audit	
reports
trust.lexisnexis.com | Right trailer materials text column |
```

Accepted true values for `docheader` and `doctrailer` include `on`, `true`, `yes`, `1`, and `enabled`. Accepted false values include `off`, `false`, `no`, `0`, `disabled`, `none`, and an empty value. A non-boolean value in `docheader` or `doctrailer` is treated as the displayed title text.

Additional trailer text can be overridden with `doctrailer_legal`, `doctrailer_materials_heading`, and `doctrailer_materials`. Use `<br>` inside a table value to force line breaks. The standard trailer no longer renders a contacts block.

## Export Header

Enable the header with:

```
header_enabled	on	Draw export header text from page 2 onwards
header_text_left	{{product}}	Optional left header text
header_text_right	{{MONTHYEAR}}	Optional right header text
```

Accepted true values for `header_enabled` include `on`, `true`, `yes`, `1`, and `enabled`. Accepted false values include `off`, `false`, `no`, `0`, `disabled`, `none`, and an empty value.

The header is drawn in the top page margin from the second exported page onwards. It does not consume document layout space. Header text appears in PDF output and in page-SVG render paths used by SVG export, layout overview PNGs, and rendered DOCX export.

## Export Footer

Enable the footer with:

```
| footer | on | Draw a red footer rule on exported pages except the final page |
| footer_text | {{product}} customer documentation | Optional text below the rule |
```

Accepted true values for `footer` include `on`, `true`, `yes`, `1`, and `enabled`. Accepted false values include `off`, `false`, `no`, `0`, `disabled`, `none`, and an empty value.

The footer is drawn in the bottom page margin after normal pagination. It does not consume document layout space. It is omitted from the final page so it does not overlap trailer artwork or standard document trailers. The same footer appears in PDF output and in page-SVG render paths used by rendered DOCX export.

PDF and rendered Word exports draw the standard cover from `.doccrate/export-assets/docheader_background.svg` with configurable text overlaid, and draw the standard trailer as vector artwork controlled by `docheader` and `doctrailer`. In a whole-project export, an enabled document header replaces the generated table-of-contents page and the first chapter starts beneath the cover banner. Projects can still provide custom legacy `header.png` and `trailer.png` files under `.doccrate/export-assets/` when those settings are omitted. For older projects, root-level `header.jpg` and `Trailer.png` remain legacy fallbacks for cover and trailer artwork only. The footer logo remains a bitmap asset named `footer_logo.png`; if no project footer logo exists, DocCrate uses its bundled standard footer logo. When a logo is available, the footer text is drawn after the logo.

## Viewer Behavior

`View > Project Config` creates the file if needed, then opens it as an editable Markdown page. The config page renders raw so users can see and edit literal tokens such as `DocCrate Project`.

When an existing config file is opened, DocCrate appends any missing standard variable rows to the `## Variables` table. Existing values are preserved; only absent standard settings are added.

Saving the config reloads the variable dictionary and clears parsed-document caches because one variable can affect many pages.

## CLI

Inspect:

```
doc-crate --project-config ~/Documents/docs --json
```

Create if missing:

```
doc-crate --project-config ~/Documents/docs --init --json
```

`--init` also appends missing standard variable rows to existing config files without overwriting project-specific values.

Single-file commands infer the project root by walking ancestors until they find `.doccrate/project-config.md`. If no config exists, they fall back to the file's parent directory.

## MCP

Use:

```
doccrate_project_config { "root": "/Users/example/Documents/docs", "init": true }
```

The result includes the root, config path, discovered variables, and validation issues.

## Interaction With Standard Content

Standard-content expansion and project substitutions use the same render path. Expanded standard content can itself contain project variables:

```
{{standard:42:"privacy notice"}}
```

If standard item 42 contains `DocCrate Project`, DocCrate expands the standard item and then resolves `DocCrate Project` from the project config.

See also: [Standard Content Library](#), [Command Line](#), and [MCP Server](#).

# Rich Text Markdown Editor Overlay Design

Status: full-workspace synchronized split-view implementation available.

DocCrate should support a richer in-process prose editor for users who think in common word-processor actions rather than Markdown punctuation. The editor still saves Markdown. It should feel like editing the visible document, but it must preserve DocCrate's source-first model, diagram editors, database snapshots, and agent-friendly Markdown files.

## Goals

- Open from a rendered Markdown text block with a right-click command such as `Edit Rich Text`.
- Absorb nearby Markdown text above and below the clicked block on the current page-like editing region until DocCrate reaches a table, diagram, page break, or other hard visual boundary.
- Provide a directly editable rich pane plus familiar controls for headings, bold, italic, strikethrough, inline code, links, lists, block quotes, page breaks, save, and cancel.
- Offer a split view where users can see and edit the generated Markdown beside the rich surface.
- Save changes back to the original Markdown source spans and update the saved DocCrate document store when the current root has a database identity.
- Avoid corrupting unsupported Markdown. When content cannot round-trip safely, keep that segment read-only or fall back to the existing raw Markdown editor.

## Non-Goals

- No browser engine or HTML editor dependency.
- No rich text format stored beside Markdown.
- No table or diagram editing inside this overlay. Tables and diagrams remain their own editor surfaces.
- No arbitrary visual styling that Markdown cannot represent cleanly, such as freeform text colors, custom fonts, margins, or alignment.

## User Experience

The document context menu for a text block should offer:

```
Edit Rich Text...
View Source for Paragraph
Bold Paragraph
Italic Paragraph
Select Text Block
Save as Standard Text...
Delete Item from Document
```

`Edit Rich Text...` opens a full-workspace modal overlay. The background document is hidden behind the editor so the user can use nearly the whole window width for prose editing. The overlay has three regions:

---

| Region        | Purpose                                                   |
|---------------|-----------------------------------------------------------|
| Toolbar       | Icon buttons and menus for formatting and block commands. |
| Rich pane     | Directly editable rendered view of the Markdown island.   |
| Markdown pane | Editable Markdown for the same island.                    |

The toolbar should use icons where possible: bold **B**, italic *I*, code, link, bullet list, numbered list, quote, page break, save, and cancel. Heading buttons currently offer H2 and H3. H1 remains a hard boundary.

The split view shows a directly editable rich pane beside an editable Markdown source pane. Both panes share one Markdown buffer. Typing, paste, delete, and toolbar commands immediately update both panes.

Scroll and focus behavior:

- Clicking or typing in either pane makes that pane active.
- The inactive pane follows the active pane and keeps the same source line or caret neighborhood in view.
- Mouse wheel scrolling in one pane scrolls the other pane to the corresponding Markdown source line.
- Toolbar commands keep the visible text selected where possible, not the hidden Markdown punctuation.
- Bold, italic, strikethrough, and inline code commands keep selected edge whitespace outside the Markdown markers, so the generated Markdown renders cleanly.

Keyboard behavior:

| Input  | Action                                    |
|--------|-------------------------------------------|
| Ctrl+B | Toggle bold on selection or typing state. |
| Ctrl+I | Toggle italic.                            |
| Ctrl+K | Edit link target for selection.           |
| Ctrl+S | Save.                                     |
| Esc    | Cancel.                                   |
| Tab    | Insert spaces in the Markdown pane.       |

## Edit Island Selection

The overlay edits an `EditIsland`, a contiguous range of top-level Markdown source spans. It starts from the clicked `SourceBlockAnchor`.

Selection algorithm:

1. Locate the clicked source block anchor and its source span.
2. Build an ordered list of top-level source spans from the current parsed document and layout anchors.

3. Classify each span as editable prose, soft boundary, or hard boundary.
4. Walk upward and downward from the clicked span while spans remain editable prose and remain in the same page-like region.
5. Stop at hard boundaries: Mermaid or blockgraph diagram fences;
6. Markdown tables;
7. fenced code blocks;
8. toolbar/navigation tables;
9. manual page breaks;
10. H1 chapter headings unless the clicked block is that heading;
11. source spans DocCrate cannot classify safely.

The "same page-like region" should initially be based on visual order between hard boundaries in the current viewer layout. When the PDF layout view is active or page mapping is available, it can use the actual page index as an additional limit.

The island stores:

```
struct EditIsland {
 document_index: usize,
 rel_path: PathBuf,
 source_start_line: usize,
 source_end_line: usize,
 source_start_span: usize,
 source_end_span: usize,
 original_text: String,
 original_hash: String,
 segments: Vec<EditIslandSegment>,
}

struct EditIslandSegment {
 anchor_id: String,
 source_line: usize,
 source_start: usize,
 source_end: usize,
 kind: RichBlockKind,
 original_hash: String,
}
```

The save operation should replace the full contiguous island range, not each block independently. The per-segment anchors are still useful for conflict checks, caret restoration, diagnostics, and future block-level history.

## Target Rich Document Model

The current implementation edits a Markdown buffer directly and projects the rendered rich pane back to Markdown source character ranges. A later richer WYSIWYG pass can add a small Markdown editing model rather than relying on the projection map alone.

```
struct RichEditDocument {
 blocks: Vec<RichBlock>,
}
```



```

enum RichBlock {
 Heading { level: u8, inlines: Vec<RichInline> },
 Paragraph { inlines: Vec<RichInline> },
 BulletList { items: Vec<RichListItem> },
 OrderedList { start: usize, items: Vec<RichListItem> },
 BlockQuote { blocks: Vec<RichBlock> },
 PageBreak,
 UnsupportedRaw { markdown: String, read_only: bool },
}

struct RichInline {
 text: String,
 marks: RichMarks,
 link: Option<String>,
}

struct RichMarks {
 bold: bool,
 italic: bool,
 strike: bool,
 code: bool,
}

```

The model should be parsed from the island Markdown with `pulldown-cmark` plus DocCrate's existing source span rules. Serialization should produce normalized Markdown:

- ATX headings for headings.
- Blank line between blocks.
- `-` for bullet lists.
- `1.` style ordered lists unless preserving a non-1 start.
- Markdown links for links.
- Backticks for inline code.
- `**`, `*`, and `***` for bold, italic, and bold-italic.
- `~~` for strikethrough.
- Existing hard wrapping preserved for untouched paragraphs where practical; edited paragraphs may serialize as soft-wrapped Markdown lines.

Unsupported inline HTML should be preserved as raw Markdown if it is not touched. If touched, the Markdown pane should show that DocCrate will normalize it.

## Target Overlay State

The current implementation keeps a focused overlay state with the island, Markdown buffer, pane focus, scroll offsets, rich/Markdown drag state, dirty flag, and status line. A later model-based editing pass should keep the richer model outside the existing raw `EditState` so the two editors can coexist cleanly:

```

struct RichTextEditorState {
 island: EditIsland,
 document: RichEditDocument,
}

```

```

markdown_buffer: String,
view_mode: RichTextViewMode,
active_pane: RichTextPane,
selection: RichSelection,
typing_marks: RichMarks,
undo: Vec<RichEditSnapshot>,
redo: Vec<RichEditSnapshot>,
dirty: bool,
status: String,
}

```

This should live in a new module, for example `src/rich_text.rs` for parsing, model, commands, and serialization. Rendering and input dispatch can either start in `src/render.rs` or move into a focused `src/rich_text_editor.rs` module once the state grows.

## Rendering And Editing

The rich pane should use DocCrate's portable text measurement and native AppKit selection drawing patterns:

- Render blocks with the same theme tokens as the normal viewer.
- Draw the caret and selection using the existing text edit caret style.
- Use stable block rectangles so toolbar state, hover, and selection do not cause layout shifts.
- Re-layout only the overlay document on each keystroke, not the whole project.
- Re-render the underlying document only after save.

The rich pane and Markdown pane share the same multiline `TextEditCursor`. The rich pane uses a projection map from rendered runs to Markdown character ranges for click hit testing, drag selection, caret painting, copy, paste, typing, and visual arrow-key movement. A later model-based pass can parse the buffer back into a `RichEditDocument` after a short debounce or on pane switch. If parsing fails, keep the Markdown pane active, show the error in the overlay status line, and disable save until the source is valid or the user switches to raw editor mode.

## Formatting Commands

Formatting commands apply to the shared source selection, whether it was made in the rich pane or Markdown pane. A later model-based editing pass should apply the same commands to the rich model:

| Command                       | Selection behavior                                        | Empty selection behavior |
|-------------------------------|-----------------------------------------------------------|--------------------------|
| Bold / Italic / Strike / Code | Toggle mark on selected inline runs.                      | Toggle typing mark.      |
| Link                          | Prompt for URL or local target and apply to selection.    | Insert link placeholder. |
| Heading menu                  | Convert selected block(s).                                | Convert current block.   |
| Bullet / Numbered list        | Convert paragraphs to list items or unwrap existing list. | Start new list item.     |
| Quote                         | Wrap or unwrap selected blocks in a blockquote.           | Quote current block.     |

The toolbar should show mixed state when a selection contains multiple mark states. A simple first pass can show active only when every selected run has the mark.

## Save Flow

Save must be transactional from the user's point of view:

1. Read the current Markdown file from disk.
2. Recompute the island source span from the saved `source_start_line`.
3. Compare the current island hash with `original_hash`.
4. If it differs, stop and show: The document changed on disk. Reload or copy your edited Markdown from the split pane.
5. Use the edited Markdown buffer as the replacement source.
6. Replace the exact contiguous island source range.
7. Write the Markdown file.
8. Clear the overlay and reload the current document.
9. If the root has `.doccrate/db-root-key`, refresh the saved root snapshot in the local database.

A future optimization can replace the full root snapshot with a row-level helper:

```
pub fn save_document_for_open_root(
 docs_dir: &Path,
 rel_path: &Path,
 content: &str,
) -> Result<(), String>
```

If the root is not database-backed, save only the file and show normal status. The existing `Project > Save Current DocCrate` command remains the explicit way to snapshot a filesystem-only root into the library.

## Document Store Changes

The current `doccrate_documents` table is sufficient for the first version. It already stores `rel_path`, `content`, `content_hash`, `byte_len`, `updated_utc`, and sync metadata through `doccrate_sync_items`.

Add helpers in `doccrate_db.rs`:

- resolve current root key from `.doccrate/db-root-key`;
- resolve root id;
- upsert one document row for a relative path;
- rebuild the matching `doccrate_document_search` row;
- update root aggregate counts and hash, or at minimum update root `updated_utc` and content hash from current child rows;
- update `doccrate_sync_items` for the document and root.

Do not add a block-level table unless future requirements need audit history, comments, or collaborative merges. The source file remains the source of truth for anchors.

## Conflict And Recovery

The overlay should protect users from invisible source churn:

- Stale island hash: do not write. Offer reload, cancel, or copy Markdown.
- Parse error in Markdown pane: disable save and keep the pane active.
- Unsupported block touched: show the raw Markdown block and require source editing for that block.
- Database update failure after file save: keep the file save, show a warning, and let `Project > Save Current DocCrate` repair the document store.
- Sync conflict later: the existing timestamp/hash sync process should report the document row as a conflict rather than silently overwriting it.

## Implementation Status

Implemented:

- `Edit Rich Text...` is available from rendered text-block context menus.
- The editor extracts a Markdown island and stops at H1 headings, diagram fences, tables, thematic breaks, and manual page breaks.
- The modal overlay renders an editable rich pane beside an editable Markdown pane.
- Click/drag selection, caret painting, typing, paste, copy/cut, and visual arrow movement work in the rich pane through a Markdown projection map.
- Toolbar commands can apply bold, italic, strikethrough, inline code, link, bullet list, numbered list, quote, H2, H3, and manual page break Markdown.
- `Ctrl+S`, `Esc`, caret movement, selection, copy, cut, paste, and mouse-wheel scrolling are handled while the overlay is open.
- Save replaces the original source island after checking that the source has not changed on disk. Database-backed roots refresh their saved root snapshot.
- `Open Standard Text Item...` is available for prose and table cells that contain one unambiguous editable standard-text token. It opens that database record in the same rich/Markdown editor and saves it with a content-hash conflict check.
- Regression tests cover island extraction and core Markdown commands.

Remaining design work:

- Undo and redo stacks.
- Link target dialog instead of inserting `url`.
- List indent/outdent shortcuts.
- Row-level saved-document update metadata, if later needed for more granular database sync.

## Acceptance Criteria

- A user can right-click a paragraph, open rich edit, apply bold/italic/list/link formatting, save, and see the rendered document update.
- The resulting Markdown is readable and uses normal Markdown syntax.
- The editor stops before diagrams and tables, preserving their existing editor flows.
- Split Markdown pane and rich pane remain synchronized.
- Database-backed roots refresh their saved root snapshot on save.
- Stale source edits are detected before writing.
- Existing raw source editing still works for diagrams, tables, code blocks, and unsupported Markdown.

# Spelling on macOS

DocCrate combines its project-aware Rust spelling engine with native macOS text editing. The Rust engine decides which Markdown text is prose and applies the brochure's regional, terminology, allow-list, deny-list, and replacement rules. AppKit displays the resulting ranges with familiar spelling underlines and context-menu actions.

This division is deliberate. Running the standard macOS continuous checker over raw Markdown would also inspect code fences, URLs, reference tokens, diagram source, and other authoring syntax that is not brochure prose.

## Project Settings

Open **View > Project Configuration...** and edit these Variables rows:

```
name	value	description
spelling_enabled	on	Enable project-aware live spelling and export QA
spelling_locale	en-GB-ise	Brochure English profile
spelling_mode	warn	off, warn, or strict
```

Underscore locale forms such as `en_GB` and `en_ZA` are accepted and normalised to DocCrate's canonical hyphenated profile names.

| Setting              | Effect                                                                                                     |
|----------------------|------------------------------------------------------------------------------------------------------------|
| spelling_enabled=off | No live project underlines and no automatic spelling QA. Manual Spell Check Document... remains available. |
| spelling_enabled=on  | Run a short, debounced project-aware review after opening or editing source.                               |
| spelling_mode=off    | Disable automatic spelling even if spelling_enabled says on.                                               |
| spelling_mode=warn   | Report findings as warnings during QA and export workflows.                                                |
| spelling_mode=strict | Report findings as errors where validation requires clean output.                                          |

New configurations keep spelling disabled until the author chooses a profile and enables it.

## Supported English Profiles

| Profile   | Intended policy                                                             |
|-----------|-----------------------------------------------------------------------------|
| en-US     | US English: color, center, analyze, organize.                               |
| en-GB-ise | British English with colour, centre, analyse, organise.                     |
| en-GB-ize | British spelling with Oxford-style analyze and organize.                    |
| en-CA     | Canadian English, including colour, centre, and common Canadian -ize usage. |
| en-AU     | Australian English.                                                         |
|           |                                                                             |

|       |                                                                                  |
|-------|----------------------------------------------------------------------------------|
| en-NZ | New Zealand English.                                                             |
| en-ZA | South African English.                                                           |
| en-EU | DocCrate European-English policy based on British -ise plus project style rules. |

Choose a profile from the **Language** menu to update `spelling_locale` for the project. The selected value is stored with the brochure rather than as a global Mac preference.

## Live Assistance in Source

When spelling is enabled, DocCrate reviews source after a short pause in typing:

1. Rust extracts checkable prose and ignores fenced code and protected inline authoring syntax.
2. It applies the selected regional profile and project terminology rules.
3. The source editor adds native macOS spelling underlines to the returned source ranges.
4. Right-clicking an underlined word offers a governed replacement when one is available, plus **Review Project Spelling....**

Changing the document invalidates old ranges immediately. Results are applied only if the same project, document, locale, and source are still current, so a late background review cannot mark newly edited text.

DocCrate deliberately disables automatic spelling correction in Markdown source. Product names, legal wording, security terminology, code, and customer names must never be silently rewritten.

## Native macOS Dictionaries

DocCrate also selects the closest `NSSpellChecker` language so standard Mac editing can supply familiar dictionary suggestions:

| DocCrate profile            | macOS dictionary |
|-----------------------------|------------------|
| en-US                       | en_US            |
| en-CA                       | en_CA            |
| en-AU                       | en_AU            |
| en-NZ                       | en_NZ            |
| en-ZA                       | en_ZA            |
| en-GB-ise, en-GB-ize, en-EU | en_GB            |

If a regional macOS dictionary is unavailable, DocCrate falls back to US English for `en-US` and British English for the other profiles. The native dictionary is an assistance layer, not the policy authority: macOS does not distinguish DocCrate's British `-ise` and `-ize` profiles or its European house style. Rust continues to enforce those differences.

# Manual Review and Corrections

Choose **Edit > Spell Check Document...** or press `Command-;` to start a governed review. Manual review works even when automatic spelling is disabled.

For each correctable finding DocCrate:

- reveals the exact Markdown source range;
- explains the regional or terminology rule;
- proposes the Rust engine's replacement;
- allows Replace, Skip, or Stop;
- saves accepted corrections through the normal document workflow.

The Mac does not silently accept every suggestion from its personal learned-word dictionary. A word learned globally in macOS could be inappropriate for another customer or brochure, so project terminology stays inside the DocCrate project.

## Project Terminology

Inline project terms can be listed in the Variables table:

```
| spelling_allow | DocCrate, LexisNexis | Project-specific accepted terms |
```

Larger governed lists live under:

```
.doccrate/spelling/allow.txt
.doccrate/spelling/deny.txt
.doccrate/spelling/replacements.tsv
```

- `allow.txt` contains product names, trademarks, acronyms, customer names, and approved technical terms.
- `deny.txt` identifies dictionary-valid words that are wrong for this project; a row may include its preferred replacement.
- `replacements.tsv` supplies locale-sensitive house-style corrections.

Example replacements:

|           |              |               |
|-----------|--------------|---------------|
| en-GB-ise | localization | localisation  |
| en-US     | authorise    | authorize     |
| en-EU     | US           | United States |

Separate those columns with actual tab characters. A two-column row applies to every profile; a three-column row applies only to its named locale.

These files travel with the project and are shared by the Mac app, CLI, MCP, TCL, validation, and export QA.



## Standard Text and Dialect Variants

Standard text is governed in the library and is never silently rewritten by the spelling checker. Existing references remain valid:

```
{{standard:575:"Vault Expiration"}}
```

An unqualified reference uses the brochure's `spelling_locale`. For a British brochure DocCrate first looks for the `en_UK` variant in the same standard-text family. If no British variant exists, it displays the approved US version and raises a **Standard Text Dialect Fallback** warning; the source token is not changed.

New non-US references can pin the intended library variant explicitly:

```
{{standard:575:"Vault Expiration":en_UK}}
{{standard:575:"Vault Expiration":en_CA}}
{{standard:575:"Vault Expiration":en_AU}}
```

The standard-content locale tags deliberately use `en_UK`, rather than the spelling engine's separate `en-GB-ise` and `en-GB-ize` profiles. Both British spelling profiles resolve governed wording through `en_UK`. US is the legacy default, so `:en_US` may be omitted.

**Find Standard Text** follows the same rule. Matching project-locale entries sort first. US fallback entries remain available, but their details show the dialect warning and the inserted token retains the requested non-US locale tag. That means a subsequently approved regional variant will be selected without editing the brochure reference.

## Important note for the Windows team

The Windows implementation must accept the optional final locale field in both labelled and unlabelled tokens, normalize hyphen/underscore aliases, and use the same resolution order: explicit token locale, otherwise project `spelling_locale`, then approved `en_US` fallback with a visible warning. The Windows standard-text finder must also prefer the requested locale, identify US fallback results before insertion, and preserve the requested locale in the inserted token. Do not silently convert or overwrite governed standard text.

## Automation

The embedded helper supports the same engine:

```
/Applications/DocCrate.app/Contents/Helpers/doc-crate \
--spellcheck ~/Documents/my-docs --locale en-GB-ise --json

/Applications/DocCrate.app/Contents/Helpers/doc-crate \
--spelling-profiles --json
```

AppleScript and Apple Events expose profile listing, profile selection, current spelling status, and manual document checking. MCP exposes `docrate_spellcheck` and `docrate_spelling_profiles`.

## Recommended Brochure Workflow

1. Set the brochure's `spelling_locale` before substantial editing.
2. Add product, trademark, legal, and customer terms to the project allow list.
3. Use `warn` while drafting.
4. Review native underlines and run **Spell Check Document...** before export.
5. Use `strict` for a final validation gate where the publication process requires zero unresolved spelling findings.

See also: [Project Configuration](#) | [User Guide](#) | [macOS Command Line](#)

# Standard Content Library on macOS

DocCrate can search, edit, create, validate, and expand governed reusable text and diagrams. The Cocoa app, embedded CLI, and MCP server share the same local SQLite library by default.

## macOS Storage

```
~/Library/Application Support/Cocoa DocCrate/security_text.sqlite
```

Administrators can select another approved SQLite file in Settings. For an isolated test environment, `DOCCRATE_MAC_STORAGE_ROOT` changes the shared root used by the Mac app and helpers.

## Content Model

| Field                      | Purpose                                               |
|----------------------------|-------------------------------------------------------|
| Reference number           | Stable identity used by <code>{{standard:n}}</code> . |
| Topic                      | Human-readable title.                                 |
| Product, region, and scope | Search and governance facets.                         |
| Content                    | Markdown expanded into documents.                     |
| Hash and provenance        | Concurrency, diagnostics, and traceability.           |

Use a labelled token in Markdown:

```
{{standard:42:"privacy notice"}}
```

For a non-US governed-text variant, add the optional locale after the readable label:

```
{{standard:42:"privacy notice":en_UK}}
```

US is the backwards-compatible default. When no locale is written, DocCrate uses the project's `spelling_locale`, tries the corresponding standard-text variant first, and falls back to approved US wording with a visible warning. The standard-text finder uses the same preference and marks fallback results. See [Spelling on macOS](#).

The numeric reference is authoritative; the label is an author-readable hint. Expansion happens before Markdown parsing, so approved content can include normal Markdown and diagrams. Unknown references remain visible and are reported by validation. Expansion is skipped inside fenced code blocks.

## Cocoa Library Windows

| Library command | Purpose |
|-----------------|---------|
|                 |         |

|                                |                                                                                                                          |
|--------------------------------|--------------------------------------------------------------------------------------------------------------------------|
| Find and Edit Standard Text    | Search, preview, create, and edit governed Markdown text.                                                                |
| Find and Edit Standard Diagram | Search, preview, create, and edit governed diagrams.                                                                     |
| Sync Library with MySQL        | Synchronise the local SQLite library over the approved mandatory-TLS connection.                                         |
| Install Signed Credential PAQ  | Verify a .paq or .txt PAQ, store the password in Keychain, test the connection, and remove the staged PAQ after success. |

Editors compare stored content hashes before update and refuse to overwrite a record changed by another process. Standard tokens stay as tokens in document source and table cells.

## Secure MySQL Sync

The Mac desktop flow resolves:

1. signed public connection metadata from `~/Library/Application Support/Cocoa DocCrate/doccrate-db-connection.zip`;
2. the password from the macOS Keychain;
3. the approved CA certificate from `~/Library/Application Support/Cocoa DocCrate/rds-ca.pem` or an approved bundled fallback.

The connection ZIP signature is mandatory. The ZIP contains host, database, user, port, table prefix, and CA metadata, but not the password. Connections use finite timeouts, mandatory TLS, DNS/CNAME trust checks, and disabled `LOAD DATA LOCAL INFILE`.

Database failures are classified as credential, resolver/network, TLS, login, database access, or test-query problems and are shown in a readable dialog with **Copy to Clipboard**. Reports do not include the password. An unsuccessful PAQ install leaves the PAQ available for a corrected retry.

Successful sync writes `database-sync-report.md` under the generated cache and opens it in DocCrate. Empty-source guards prevent an accidentally empty SQLite or MySQL side from overwriting a populated side.

## macOS CLI

```
doc-crate --standard-content-search --query privacy --product "DMS Connections" --region
Global --json
doc-crate --standard-content-show '{{standard:42:"privacy notice"}}' --json
doc-crate --standard-content-expand '{{standard:42:"privacy notice"}}' --json
doc-crate --standard-content-validate docs/whitepaper.md --json
doc-crate --standard-content-parse-diagrams --json
doc-crate --standard-content-create --topic "Privacy" --content-file privacy.md --json
doc-crate --standard-content-show 42 --json
doc-crate --standard-content-update 42 --content-file privacy.md --expect-hash <content-hash>
--json
```

Use the helper embedded in `/Applications/DocCrate.app/Contents/Helpers/` or add that directory to `PATH`. `doc-crate --help` is authoritative for the installed command syntax.

## MCP and TCL

MCP exposes equivalent search, show, expand, validate, create, update, and sync operations for agents. TCL exposes the repeatable local workflow. Explicit database overrides should be used only for approved or isolated data.

## Security Rules

- Never store passwords in Markdown, project config, shell history, SQLite content, connection ZIPs, sync reports, or agent prompts.
- Do not copy customer or governed library material into public examples.
- Preserve hash checks on managed updates.
- Treat the connection ZIP, signature, CA, PAQ, and Keychain entry as separate operational components.

Back to [Home](#) | [Connection Protection](#)

# Theming

DocCrate has two theme systems:

- **Viewer themes** control the application chrome, Markdown colours, diagram card defaults, and theme-aware diagram defaults.
- **Syntax highlighting themes** control token colours inside fenced code blocks. Configure those in `.doccrate/highlight.toml` and `.doccrate/themes/*.tmTheme`.

## Viewer Themes

DocCrate ships with `dark`, `light`, and `lexis-red`. The `lexis-red` theme is a light, print-preview style theme that uses the same LexisNexis PMS 185 red heading treatment as PDF export.

A docs root can add named viewer themes in:

```
<docs>/doccrate/themes.json
```

`theme.json` is accepted as a compatibility alias.

Example:

```
{
 "active": "midnight",
 "themes": {
 "midnight": {
 "base": "dark",
 "colors": {
 "bg": "#101418",
 "sidebar_bg": "#151b22",
 "text": "#d7e2ea",
 "link": "#6bb6ff",
 "code_bg": "#0b1015",
 "diagram_card_bg": "#111a24",
 "diagram_card_stroke": "#38506a",
 "mermaid_node_fill": "#162233",
 "mermaid_group_title": "#ffd166",
 "arch_group_header": "#1c2633",
 "arch_icon_fill": "#16334a"
 },
 "raw": {
 "#202020": "#1c2633"
 }
 }
 }
}
```

`active` selects the startup theme for that docs root. You can override it when launching:

```
doc-crate --theme midnight docs
```

Switch an already-open viewer from the CLI or MCP:

```
doc-crate --set-theme midnight
```

MCP:

```
docrate_set_theme { "theme": "midnight" }
```

Ctrl+T cycles through `dark`, `light`, `lexis-red`, and configured custom themes.

## Theme Inheritance

Each custom theme has a `base` of `dark` or `light`. DocCrate first applies the base mapping, then overlays your `colors` and `raw` entries.

`colors` maps named DocCrate tokens. `raw` maps a literal source colour to another literal colour. Use `raw` for diagram-type semantic palettes that intentionally use fixed colours and do not have a named token.

## Supported Colour Keys

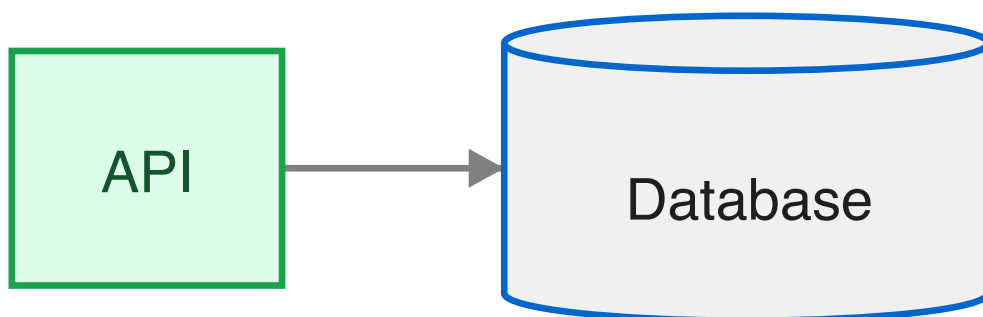
| Key                                                     | Purpose                            |
|---------------------------------------------------------|------------------------------------|
| <code>bg</code>                                         | Main viewer background             |
| <code>sidebar_bg</code>                                 | Sidebar and some panel backgrounds |
| <code>sidebar_sel</code>                                | Selected sidebar row               |
| <code>sidebar_hvr</code> / <code>sidebar_hover</code>   | Hovered sidebar row                |
| <code>border</code>                                     | Divider and grid lines             |
| <code>scrollbar</code>                                  | Scrollbar track                    |
| <code>scrollthumb</code> / <code>scrollbar_thumb</code> | Scrollbar thumb                    |
| <code>text</code>                                       | Body text                          |
| <code>text_dim</code> / <code>muted</code>              | Muted text and markers             |
| <code>text_bright</code>                                | Bright text and selected labels    |
| <code>h1</code> through <code>h6</code>                 | Markdown heading colours           |
| <code>link</code>                                       | Link and primary accent colour     |
| <code>link_hvr</code> / <code>link_hover</code>         | Hovered link colour                |
| <code>code_fg</code>                                    | Default code text colour           |
| <code>code_bg</code>                                    | Code block background              |
| <code>blockquote</code>                                 | Blockquote accent bar              |
| <code>rule</code>                                       | Horizontal rule                    |
|                                                         |                                    |

|                      |                                        |
|----------------------|----------------------------------------|
| sidebar_dir          | Sidebar directory labels               |
| diagram_card_bg      | Diagram card background                |
| diagram_card_stroke  | Diagram card border                    |
| mermaid_node_fill    | Default Mermaid node fill              |
| mermaid_node_stroke  | Default Mermaid node stroke            |
| mermaid_node_text    | Default Mermaid node text              |
| mermaid_edge         | Default Mermaid edge line              |
| mermaid_edge_label   | Default Mermaid edge label             |
| mermaid_group_fill   | Default Mermaid group fill             |
| mermaid_group_stroke | Default Mermaid group stroke           |
| mermaid_group_title  | Default Mermaid group title            |
| arch_group_header    | Architecture diagram group header fill |
| arch_icon_fill       | Architecture diagram icon fill         |

Colours use `#RRGGBB` strings. Unknown token names or malformed colour strings stop startup with an error so mistakes are visible.

## Diagram-Specific Overrides

Viewer themes set defaults. Diagram annotations still win for individual diagrams:



Use annotations when one diagram needs special emphasis; use `themes.json` when the whole docs root should share a palette.



# Menu And Editing Usability Design

DocCrate has grown from a Markdown viewer into a brochure editor, project checkout tool, governed content library, diagram editor, and export tool. The menus should now describe those jobs directly instead of exposing implementation details or scattering related actions across `File`, `View`, and `Database`.

## Goals

- Make project lifecycle, export, visual editing, source editing, and governed library work obvious from the top menu.
- Keep `View` for display mode and display preferences only.
- Replace object-specific "edit as text" commands with one whole-document `View Source` action that jumps to the selected source range.
- Keep visual editors for text, tables, diagrams, shapes, and standard-content records.
- Use contextual labels in right-click menus so the command says what will be edited, inserted, copied, or deleted.
- Make it clear when a command edits a database artefact rather than the local document copy.

## Recommended Top-Level Menus

| Menu     | Purpose                                                                                                                    |
|----------|----------------------------------------------------------------------------------------------------------------------------|
| File     | Application-level commands: new window if added later, run TCL script, exit.                                               |
| Project  | Open, create, save, import, close, recent DocCrates, project config, and saved DocCrate lifecycle.                         |
| Edit     | Normal editing commands: undo/redo when available, cut/copy/paste, find, replace, spell check, selection actions.          |
| Insert   | Discoverable document insertion commands for text, table, page break, standard text, diagram, and standard diagram.        |
| View     | Display controls: document/source mode, sidebar, themes, zoom where applicable, and visual finders.                        |
| Library  | Governed database artefacts: standard text snippets, standard diagrams, KB import/search, sync, and connection status.     |
| Export   | PDF, Word, editable Word, SVG, layout overview, diagram pack, current page, and whole document exports.                    |
| Language | Spelling/dialect selection.                                                                                                |
| Editor   | Appears only while an editor is active and contains editor-specific save, exit, zoom, pan, add, link, and delete commands. |

`Project` and `Export` should become main menus. The current `File` menu is overloaded: users opening a brochure, importing a folder, saving a database checkout, and exporting a PDF are doing different jobs. Separating those jobs reduces hunting and makes the product feel more deliberate.

Do not add a generic top-level `Update` menu. "Update" is too ambiguous: it could mean update the current document, update the database, update a standard snippet, update a checkout, update the release folder, or update DocCrate itself. Use `Library` for governed artefacts and use contextual verbs such as `Edit Standard Text...`, `New Standard Text...`, `Save Standard Text`, and `Sync Library`.

## Proposed Menu Contents

### File

| Command           | Notes                                                       |
|-------------------|-------------------------------------------------------------|
| Run TCL Script... | Automation belongs here unless a later Tools menu is added. |
| Exit              | Close the application.                                      |

### Project

| Command                      | Notes                                                                     |
|------------------------------|---------------------------------------------------------------------------|
| New Project...               | Create a local docs root.                                                 |
| Open DocCrate...             | Open the saved database DocCrate browser.                                 |
| Open Local Project Folder... | Clearer than "Open Project From Local Folder".                            |
| Recent DocCrates             | Saved database and local folder recents.                                  |
| Close Project                | Return to bundled docs.                                                   |
| Project Config               | Move from View; config is project data, not display.                      |
| Save Current DocCrate        | Save the current checkout under its database identity.                    |
| Save DocCrate As...          | Save as a separate database entry.                                        |
| Import DocCrate Folder...    | Import one project or a collection.                                       |
| List Versions...             | Open version history for the current saved DocCrate when available.       |
| Restore Version...           | Copy a historical version as the latest version without deleting history. |

### Edit

| Command            | Notes                                                                                                        |
|--------------------|--------------------------------------------------------------------------------------------------------------|
| Undo / Redo        | Enable in source and rich editors first; grey out elsewhere.                                                 |
| Cut / Copy / Paste | Work on source text, rich editor text, table cells, selected blocks, and diagram selections where supported. |
| Find...            | Project/document find.                                                                                       |
|                    |                                                                                                              |

|                         |                                                                                             |
|-------------------------|---------------------------------------------------------------------------------------------|
| Find and Replace...     | Source and rich editor aware.                                                               |
| Spell Check Document... | Current behavior.                                                                           |
| Cut Selection           | Visible when rendered blocks, tables, diagrams, or another movable object selection exists. |
| Copy Selection          | Visible when a selection exists.                                                            |
| Paste After Selection   | Visible when the paste buffer contains a document object.                                   |
| Delete Selection...     | Uses the red confirmation state for rendered document blocks.                               |
| Shape Editor...         | Could move to Library > Shapes later, but can remain here until shape workflows mature.     |

## Insert

| Command             | Notes                                                              |
|---------------------|--------------------------------------------------------------------|
| Text                | Insert below the current caret/block and open rich text editing.   |
| Table               | Insert a starter table and open the table editor.                  |
| Page Break          | Insert a page break marker.                                        |
| Standard Text...    | Open the standard text finder in insertion mode.                   |
| Diagram             | Insert a starter diagram and open the visual editor when possible. |
| Standard Diagram... | Open the standard diagram finder in insertion mode.                |

## View

| Command                               | Notes                                                                                   |
|---------------------------------------|-----------------------------------------------------------------------------------------|
| Document                              | Rendered document mode.                                                                 |
| Source                                | Whole-document source editor. Reparse on return to Document.                            |
| Toggle Sidebar                        | Current behavior.                                                                       |
| Find Diagram...                       | Visual navigation aid can remain in View; editing standard diagrams belongs in Library. |
| Next Theme / Dark / Light / Lexis Red | Current behavior.                                                                       |

**Source** replaces the many object-level source-edit commands. When invoked from a right-click menu, DocCrate should switch the entire main window to source mode, scroll to the object's source line, and select or highlight the source range. Returning to **Document** reparses the full Markdown file. Parse errors should keep the user in source mode, show the error, and move the caret to the problem line.

## Library

| Command                     | Notes                                                  |
|-----------------------------|--------------------------------------------------------|
| Find Standard Text...       | Compact finder for insertion/copy.                     |
| Manage Standard Text...     | Full search/edit/create panel.                         |
| Open Standard Text by ID... | Accepts 272, #272, or a full token.                    |
| New Standard Text...        | Create a governed snippet.                             |
| Find Standard Diagram...    | Search/copy/insert standard diagrams.                  |
| Manage Standard Diagrams... | Open/edit/delete/rename governed diagrams.             |
| Import KB Spreadsheet...    | Ensure local schema and import SafeBase-style KB data. |
| Search KB...                | Search imported KB content.                            |
| Sync Library                | Bidirectional local/MySQL sync.                        |
| Connection Settings...      | Save credential and connection package status.         |
| Test Connection             | TLS MySQL test.                                        |

This menu can still use the existing local SQLite and MySQL sync code. The menu label should describe the user's mental model: they are working with an approved content library, not with raw database tables.

## Export

| Command                | Notes                                                      |
|------------------------|------------------------------------------------------------|
| Current Page to PDF    | Current Ctrl+E.                                            |
| Document to PDF        | Whole project/brochure.                                    |
| PDF Layout Overview    | Pagination contact sheet.                                  |
| Current Page to Word   | Rendered DOCX.                                             |
| Document to Word       | Rendered DOCX.                                             |
| Editable Word Document | Native editable DOCX where possible.                       |
| SVG                    | Whole document or current page, depending current support. |
| Diagram Pack           | Existing command.                                          |

Export commands should no longer live in `File`. Export is central to DocCrate and important enough to be visible as its own workflow.

## Context Menu Design

Right-click menus should be short, contextual, and named for the object under the mouse. A command should not say "edit source" when it actually opens a whole-document source view, and it should not say "edit diagram" when the selected object is a standard database diagram.

### Text Block

| Command                                              | Rule                                                                                                     |
|------------------------------------------------------|----------------------------------------------------------------------------------------------------------|
| Edit Paragraph...                                    | Opens the rich Markdown editor for the selected prose island.                                            |
| View Source for Paragraph                            | Switches to whole-document source mode at the paragraph range.                                           |
| Open Standard Text #272...                           | Show only when the source contains one unambiguous standard token. Include the key/label when available. |
| Save Paragraph as Standard Text...                   | Creates a new standard text item.                                                                        |
| Select Paragraph                                     | Starts or updates block selection.                                                                       |
| Cut Paragraph / Copy Paragraph / Delete Paragraph... | Object-specific labels for single-block actions.                                                         |
| Insert After >                                       | Text, Table, Page Break, Standard Text, Diagram, Standard Diagram.                                       |

### Multi-Selection

| Command                                 | Rule                                                                                             |
|-----------------------------------------|--------------------------------------------------------------------------------------------------|
| Cut Selected Blocks                     | Works for continuous block selections.                                                           |
| Copy Selected Blocks                    | Copies Markdown source for the selected range.                                                   |
| Move Selection Up / Move Selection Down | Same behavior as Alt+Up / Alt+Down; disabled with a clear message for non-contiguous selections. |
| Delete Selected Blocks...               | Turns the selected blocks red; Enter or Delete confirms, Esc cancels.                            |
| View Source for Selection               | Switches to source mode and selects the source range.                                            |

### Table

| Command                                  | Rule                                             |
|------------------------------------------|--------------------------------------------------|
| Edit Table...                            | Opens the visual table editor.                   |
| View Source for Table                    | Whole-document source mode at the table range.   |
| Insert Standard Text in Cell...          | Only when a cell is under the mouse or selected. |
| Open Standard Text #272...               | Only when a cell contains one unambiguous token. |
| Cut Table / Copy Table / Delete Table... | Table-aware object commands.                     |

## Diagram

| Command                                        | Rule                                            |
|------------------------------------------------|-------------------------------------------------|
| Edit Diagram Visually...                       | Only for diagram families with a visual editor. |
| View Source for Diagram                        | Whole-document source mode at the fenced block. |
| Copy Diagram as SVG                            | Current behavior.                               |
| Save Diagram as Standard Diagram...            | Creates or updates governed diagram content.    |
| Cut Diagram / Copy Diagram / Delete Diagram... | Diagram-aware object commands.                  |

If the diagram is a standard diagram reference or the editor opens a database diagram record, the menu and editor banner must say:

Editing standard diagram #123, not the document copy

## Empty Document Space

| Command                           | Rule                                            |
|-----------------------------------|-------------------------------------------------|
| Insert Here > Text                | Insert below the nearest block above the click. |
| Insert Here > Table               | Insert a starter table.                         |
| Insert Here > Page Break          | Insert a page break marker.                     |
| Insert Here > Standard Text...    | Open the finder in insertion mode.              |
| Insert Here > Diagram             | Insert a starter diagram.                       |
| Insert Here > Standard Diagram... | Open the finder in insertion mode.              |

## Source Editing Policy

Source editing should be one high-quality mode:

- `View > Source` switches the whole main window to source text.
- `View > Document` reparses and returns to the rendered document.
- Right-click `View Source for ...` enters the same source mode and navigates to the selected object's source range.
- The source editor keeps normal native macOS text editing commands: cursor movement, selection, shift-selection, word movement, page movement, `Ctrl+A`, `Ctrl+X/C/V`, `Ctrl+Z/Y`, `Home`, `End`, `Delete`, `Backspace`, search, and replace.
- Long lines should wrap by default for readability, with a toggle for horizontal pan if exact source layout is needed.
- Parse errors do not discard edits. They keep the user in source mode, show the first blocking error, and move the caret to the relevant line. Best-effort rendered preview can remain visible only if it is

clearly marked stale.

Inline "diagram source" or "table Markdown source" editors should be removed from right-click menus once full source mode can jump to the relevant range. The visual editors remain because they provide a different editing experience, not just a smaller text box.

## Standard Database Artefact Editing

Standard text, standard diagrams, imported KB records, and saved DocCrate versions are governed artefacts. Their UI should always expose four facts:

| Fact          | Example UI text                                                                                               |
|---------------|---------------------------------------------------------------------------------------------------------------|
| Artefact type | Standard Text, Standard Diagram, KB Entry, Saved DocCrate Version.                                            |
| Stable ID     | #272, diagram id, KB object id, version id.                                                                   |
| Scope/key     | anz_local_deployment, product, region, or path.                                                               |
| Save target   | Saving updates the standard library, Saving updates this checkout, or Restoring creates a new latest version. |

The save button should use contextual labels:

- Save Standard Text
- Save Standard Diagram
- Save KB Entry
- Save DocCrate Version
- Save Document

This removes the current ambiguity around whether the user is editing a local document copy or a shared database item.

## Migration Plan

1. Add top-level Project, Export, Insert, and Library menus while keeping old command IDs where possible.
2. Move menu items without changing behavior first.
3. Rename context-menu labels so they describe the selected object.
4. Route all object source commands to whole-document source mode with source range navigation.
5. Remove inline source-only editors after full source mode handles paragraph, table, diagram, and selection range navigation reliably.
6. Add contextual editor banners for standard database artefacts.
7. Update docs/user-guide.md, docs/shortcuts.md, CLI/MCP help, and screenshots after the menu structure lands.

## Acceptance Criteria

- Users can distinguish project work, export work, view changes, and library maintenance from the top menu alone.
- No right-click menu contains both "source" and "visual" commands that open competing text editors for the same object.
- View Source for Paragraph/Table/Diagram/Selection always opens the same source editor and highlights the relevant range.
- Standard database artefact editors always identify the artefact and whether save writes to the library or the current document.
- Export commands are discoverable without opening File .
- Standard text and standard diagram maintenance is discoverable without opening View .